# 《问矩》智能助教系统

## 全栈技术架构、多模态数学引擎与工程质量白皮书

Full-Stack Architecture, Multimodal Mathematical Canvas, Pedagogical State Machine,

Adversarial Verification & Production Deployment


**《问矩》核心系统研发与架构工程组**

System Engineering & Cognitive Pedagogical Architecture Team



**版本编号：** v1.0-Production-Ready

**验收基线：** 151/151 Test Suites 100% Passed · 腾讯云生产环境实测

**发布日期：** 2026 年 10 月

# 执行摘要 (Executive Summary)

本项目（《问矩》线性代数实验式探究智能助教系统，Wenju Intelligent Pedagogical System）是一套运行在现代浏览器环境中的《线性代数》实验式探究教学助教平台。系统遵循“物理计算核为真值基准、大语言模型为交互枢纽”的解耦架构：学生在几何画板上调整矩阵、直观观测空间几何拓扑形变，助教按结构化探究流程启发式追问；代数数值事实排他性地由前端本地高精度计算核给出，大语言模型仅负责原理解释与受控工具调度。当运行环境缺少可用 API 密钥时，对话引擎显式处于不可用状态，绝对不进行不可靠的无凭据代答。

本技术白皮书系统性地阐述了《问矩》的全栈技术架构与工程实现细节。全文聚焦四大直接可用界面、四种核心舞台模式及六大底层工程保障：

1. **四大直接可用交互界面：**系统面向学习者直接暴露门户导引（`#landing`）、实验探究工作台（`#studio`）、概念误解实验室（`#library`，内置 6 组典型认知冲突反例）与认知图谱导图（`#graph`，12 节点依赖拓扑，明确前 7 个节点对齐课程作答核验，后 5 个高阶节点作为知识骨架供查阅）；

2. **舞台四种交互模式：**支持坐标画板（2D 仿射向 3D 柱面正交嵌入，严格维持 $\det(A_3) = \det(A_2)$ 行列式守恒）、数学实验工件（涵盖正交投影、特征子空间、SVD 等 5 类受控 4 步分镜工件）、自由工件沙箱（HTML/SVG 源码实时预览与版本回滚）及 Manim 几何微课（预置 5 段核心微课切片，实施 `videoGeneration: false` 禁令，仅支持受控播放与画板穿透）；

3. **双层代数计算核与真值守卫：**前端运行基于 BigInt 的 `RationalCore` 精确分数计算核（完全不依赖浮点容差阈值）与 SymPy Worker 符号计算层，在 $[-20, 20]$ 整数有理域及 $10^{-8} \sim 10^{-9}$ 残差公差下提供绝对确定性的代数真值；

4. **Excalidraw 风格画板手绘批注与多模态视觉闭环：**基于二次贝塞尔拟合与图元碰撞擦除算法，通过双层离屏像素级无损光栅化合并导出机制（`exportMergedCanvas`），实现“圈画猜想 $\rightarrow$ 视觉深度理解点拨”的启发式教学闭环；

5. **混合模型编排与 14 项工具体系：**默认注册 14 项代数与视口控制工具，配备工具近邻守卫拦截与双重前端教学门禁（防剧透门禁拦截索要答案，反思流控盘拦截猜想并走确定性模板）；

6. **客户端存储治理与 8 分钟滑动即焚看门狗：**实施会话信封版本有序迁移，结合 15 秒心跳轮询与 8 分钟（480,000ms）闲置超时物理重置，达成“服务器零存储残留、用户跨端绝对隔离”的严格隐私规范；

7. **全维度工程质量验证：**确立单一清单源（X1 契约），实现覆盖单元、CDP 真实浏览器 E2E、集成、边界对抗与黄金基准的 151 个测试套件 **100% 物理通过率 (0 SKIP / 0 FAIL)**，主应用行数严格受控于 2298 行（$\leq$ 2300 行安全红线）；

8. **腾讯云高韧性生产部署：**独立端口 8888 物理隔离拓扑，对宿主机已有平台业务（80/443 端口）零侵扰、零闪断，Nginx 动态变量反代与秒级平滑重载。

本报告以形式化工程与数学语言展开，提供完备的代码级契约与物理实测数据证明，可作为系统交付验收、学术复现与后续工程演进的权威规范。

# 目录

**执行摘要** **i**

**引言与系统全貌** **1**

0.1 系统定位与核心工程哲学 . . . . . 1
0.2 用户直接交互的四块核心界面 . . . . . 1
0.3 实验探究工作台左侧舞台的四种运行模式 . . . . . 1
0.4 结构化课程、评分核验与认知图谱点亮边界 . . . . . 2
0.5 误解库与反例推演实验室 . . . . . 3
0.6 助教跨轮协议与双重确定性门禁 . . . . . 3
0.7 大模型接入与默认 14 项注册工具体系 . . . . . 4
0.8 双层确定性数学真值计算体系 . . . . . 4
0.9 本机宿主服务能力边界（npm run server） . . . . . 6
0.10 数学符号与形式化命名空间规范 . . . . . 6

**第一部分　系统宏观架构与设计哲学** **8**

**第一章　纯前端无状态（Stateless SPA）范式重构与边缘代理拓扑** **9**

1.1 传统教育系统“重后端 + 状态化数据库”架构的弊端与解构 . . . . . 9
1.2 《问矩》系统总体架构拓扑设计 . . . . . 9
1.3 本机 Node 宿主服务（`npm run server`）的能力边界与沙箱契约 . . . . . 10
1.4 零服务器存储契约（Zero Server-Side Storage Contract）的物理边界 . . . . . 11
1.5 客户端沙箱资源边界与滑动即焚看门狗失效模式分析 . . . . . 12
1.6 纯前端无状态与边缘代理的工程资源边界与契约约束 . . . . . 12

**第二章　双面解耦设计协议与软件复杂度红线控制** **14**

2.1 双面解耦协议（Dual-Plane Decomposition Protocol）的设计机理 . . . . . 14
2.2 软件复杂度红线控制：`js/app.js` $\leq$ 2300 行守恒治理 . . . . . 15

**第三章　单向状态流、CQRS 读写分离与会话生命周期模型** **16**

3.1 命令与查询职责分离（CQRS）在教学探究中的落地 . . . . . 16
3.2 会话生命周期状态机模型 . . . . . 16

3.3 多轮探究微缩导航轨（PromptTurnNavigator）与思维链折叠器（ReasoningBlock） . . . . . 16
3.3.1 Turn Minimap Rail 微缩刻度轨与视口感知联动 . . . . . 17
3.3.2 ReasoningBlock 深度推导思维链折叠与分词解析 . . . . . 18

**第二部分 多维交互数学画布系统 19**

**第四章 2D 动态坐标系与连续矩阵映射网格引擎 20**
4.1 坐标系统数学建模与多图层映射变换方程 . . . . . 20
4.2 基底向量拖拽交互与矩阵逆解动力学 . . . . . 20
4.3 奇异矩阵退化过程与有向面积测度渲染 . . . . . 21

**第五章 3D 立体几何空间与四维超立方体正交投影引擎 23**
5.1 2D 仿射平面与 3D「柱面嵌入」切换及行列式测度守恒 . . . . . 23
5.2 Three.js 立体场景架构与欧拉角/四元数旋转插值 . . . . . 23
5.3 四维超立方体（Tesseract）在三维欧氏空间的旋转与投影算法 . . . . . 24

**第六章 空间遥测总线与多组件解耦通讯架构 26**
6.1 高频物理事件的防抖与背压控制 . . . . . 26
6.2 画布适配器解耦契约 (‘CanvasPort‘ & ‘JSXGraphAdapter‘) . . . . . 26
6.3 病态矩阵下溢与 WebGL 丢失防御 . . . . . 27
6.4 数学引擎渲染管线的算法时间复杂度与帧预算分析 . . . . . 28
6.5 多模态实验工件工作台（ArtifactStageDeck）与环境感知微气泡（AmbientScaffoldBubble） . . . . . 28
6.5.1 四态视口投影架构与实时源码热重载 . . . . . 28
6.5.2 环境感知微气泡（AmbientScaffoldBubble）的主动启发中继 . . . . . 29
6.6 CQRS 驱动的时空穿梭控制器（TimeTravelScrubber） . . . . . 29
6.6.1 浮动式时空进度条与步进/步退状态机 . . . . . 30

**第三部分 Excalidraw 风格画板手绘与多模态视觉闭环 31**

**第七章 透明手绘覆层渲染架构与高分屏适配 32**
7.1 双层分离式画板架构（Dual-Layer Canvas Architecture）的设计动机 . . . . . 32
7.2 设备像素比（DPR）动态自适应与高分屏模糊消除算法 . . . . . 33
7.3 历史操作栈治理与图元对象模型 . . . . . 33

**第八章 自由手绘几何图元算法与碰撞检测详解 35**
8.1 钢笔笔触：离散采样点集的中点二次贝塞尔平滑拟合 . . . . . 35
8.2 五大自由图元算法实现与碰撞检测 . . . . . 36

**第九章　助教多模态视觉理解闭环与全链路实测** **37**
9.1　离屏像素级图层合成算法 (‘exportMergedCanvas‘) . . . . . 37
9.2　从“画板圈画猜想”到“大模型视觉深度解析”的全链路实测 . . . . . 38
9.3　密集采样笔触下的垃圾回收（GC Pause）与多模态合成失效防御 . . . . . 38
9.4　批注引擎与多模态合成管线的计算复杂度与几何算法边界 . . . . . 39

**第四部分　结构化课程体系与运行时动态判卷引擎** **40**

**第十章　建构主义探究式实验课架与关卡状态转移模型** **41**
10.1 PECHDT 六步认知循环状态机设计 . . . . . 41
10.2 四大章节七大真实实验课程与判分契约 . . . . . 42
10.2.1 认知图谱 12 节点掌握度点亮边界声明 . . . . . 42
10.3 课程持久化与自适应导航器架构 . . . . . 42
10.4 认知误解条目库（`MisconceptionLibrary`）与贝叶斯知识追踪（BKT）建模 . . 43
10.4.1 四大领域 32 项典型概念盲区拓扑与三元解耦 . . . . . 43
10.4.2 贝叶斯知识追踪（BKT）参数化模型与可辨识性硬断言 . . . . . 43

**第十一章　无硬编码的运行时代数动态评分与公差核验算法** **45**
11.1 传统“硬编码匹配”答题器的缺陷与反作弊漏洞 . . . . . 45
11.2 特征子空间与不变方向动态验证方程 . . . . . 45
11.3 秩与核空间（Kernel Space）公差核验 . . . . . 46
11.4 浮点数计算误差容限与自适应公差区间设计 . . . . . 46

**第十二章　课程实验系统的失效模式与认知死锁防御** **47**
12.1 学习者“穷举试错（Brute Force Guessing）”对抗与自适应阻断策略 . . . . . 47
12.2 极端数值作答的运行时沙箱防护 . . . . . 47
12.3 动态判卷引擎的代数时间复杂度与数值稳定性边界 . . . . . 47

**第五部分　3Blue1Brown 风格 Manim 高清微课与视听联动引擎** **49**

**第十三章　离线自包含 Manim 微课包设计与自研播放控制器** **50**
13.1 离线轻量化视频编码规范与容器优化 . . . . . 50
13.2 五段精编成片切片与代数结构对应 . . . . . 50
13.3 协议约束与 Docker 宿主隔离渲染 . . . . . 50
13.4 高精度微课区间循环播放器 . . . . . 52
13.5 跨平台与移动端渲染策略兼容性矩阵 . . . . . 53

**第十四章　“联动画板实操”状态穿透与跨组件协同机制** **54**
14.1 视听空间与几何操作空间的认知阻断痛点 . . . . . 54
14.2 状态穿透通道设计与事件流图解 . . . . . 54

14.3 双向联动的数据完整性守恒与防污染机制 . . . . . 55

**第十五章 微课模块的有效性边界与跨域事件一致性验证 56**

15.1 状态穿透与跨组件事件守恒验证 . . . . . 56
15.2 视频资源生命周期与内存安全边界 . . . . . 56

**第六部分 多智能体认知流水线、工具邻居守卫与混合模型编排 57**

**第十六章 混合模型热插拔编排与免密旗舰保障 58**

16.1 双轨模型架构决策与选型逻辑 . . . . . 58
16.2 流式传输（SSE）管线与反代零缓存穿透 . . . . . 59
16.2.1 边缘代理零缓冲配置契约 . . . . . 59
16.2.2 客户端数学公式流式缓冲切片机制 . . . . . 59
16.3 双轨模型接入架构与自备凭据安全隔离规范 . . . . . 60
16.4 助教跨轮启发协议、双重确定性门禁与 14 项注册工具体系 . . . . . 60
16.4.1 六步跨轮探究协议与动态系统提示注入 . . . . . 60
16.4.2 双重前端前置确定性门禁 . . . . . 61
16.4.3 几何关卡注册表（`InquiryScenarioRegistry`）的真实工程状态 . . . . . 61
16.4.4 默认 14 项注册工具与 2 项隐藏工具边界 . . . . . 61

**第十七章 轻量 Schema 校验器与工具邻居守卫 (`ToolNeighborGuard`) 64**

17.1 函数调用（Tool Calls）的语义漂移与近邻混淆 . . . . . 64
17.2 轻量 Schema 校验器与零依赖断言 . . . . . 64
17.3 工具邻居守卫 (`ToolNeighborGuard`) 核心算法实现 . . . . . 65
17.3.1 守卫路由规则表定义 . . . . . 65
17.3.2 三态分类决策状态机 . . . . . 65

**第十八章 模型幻觉抑制与认知流转的失效模式建档 67**

18.1 线性代数大模型 10 类典型概念幻觉与 Bad Cases 库 . . . . . 67
18.2 双层防御架构：代码硬断言与提示词防护栏 . . . . . 67
18.3 认知流转潜在失效模式建档与基准度量 . . . . . 68
18.3.1 系统级失效模式建档 . . . . . 68
18.3.2 双层防御架构的形式化物理测试套件与安全断言 . . . . . 68

**第七部分 前端存储架构、会话信封迁移与滑动即焚看门狗 70**

**第十九章 会话数据信封（`Envelope v1`）与模式版本迁移 71**

19.1 纯前端状态持久化挑战与信封规范 . . . . . 71
19.1.1 数据信封标准模式结构 . . . . . 71
19.2 模式版本迁移器（`PayloadMigrations`）设计与形式化实现 . . . . . 72

19.2.1 迁移器四大核心守恒契约 . . . . . 72
19.2.2 真实生产迁移器核心逻辑 . . . . . 72
19.3 多会话生命周期与存储端口解耦 . . . . . 73

**第二十章 15 秒轮询心跳与 8 分钟（480,000ms）滑动即焚看门狗 74**
20.1 “零残留、强隔离”隐私安全契约 . . . . . 74
20.2 8 分钟滑动即焚看门狗时序与自愈状态机 . . . . . 74
20.2.1 高频活动感知 (touch) . . . . . 74
20.2.2 15 秒高精度周期巡检与漂移补偿 . . . . . 74
20.3 极端时序边界与后台休眠时钟漂移补偿 . . . . . 76
20.3.1 休眠与后台挂起唤醒补偿 . . . . . 76
20.3.2 长流式生成的保活锁机制 . . . . . 76

**第二十一章 客户端存储边界极限与并发隔离威胁分析 77**
21.1 存储配额耗尽与滑动修剪策略 . . . . . 77
21.1.1 长对话压缩器与复习包/演示包导出机制 . . . . . 77
21.2 多标签页并发访问隔离与冲突消解 . . . . . 78
21.3 存储子系统的理论空间复杂度与迁移一致性分析 . . . . . 78
21.3.1 空间复杂度与浏览器配额安全边界 . . . . . 78
21.3.2 阅后即焚与内存物理清空安全契约 . . . . . 78

**第八部分 151 项全量工程测试套件与对抗验证体系 79**

**第二十二章 单一清单源驱动的测试架构与质量契约 80**
22.1 传统前端测试“割裂与虚假覆盖率”之痛 . . . . . 80
22.2 单一清单源（X1 契约）设计与统一调度体系 . . . . . 80
22.2.1 X1 契约的核心工程法则 . . . . . 80
22.2.2 本地轻量静态服务生命周期托管 . . . . . 81
22.3 测试分层金字塔与多维子集切分 . . . . . 81

**第二十三章 全真无 Mock 浏览器 CDP 自动化端到端测试 83**
23.1 Chrome DevTools Protocol（CDP）真机驱动管线 . . . . . 83
23.1.1 CDP 驱动初始化拓扑 . . . . . 83
23.2 像素级视觉回归与时间线精准同步 . . . . . 83
23.2.1 虚拟鼠标指针轨迹连续注入 . . . . . 83
23.2.2 双层光栅化图像哈希断言 . . . . . 84
23.3 跨组件探究全链路自动化跑通 . . . . . 84

**第二十四章　边界条件、混沌注入与黄金基准测试矩阵** **85**
24.1 极端边缘坏样本与对抗鲁棒性物理验证 . . . . . 85
24.2 混沌测试（Chaos Engineering）与极端网络故障注入 . . . . . 86
24.3 151 项全量工程测试套件分类统计全景 . . . . . 86

**第九部分　腾讯云高韧性生产部署、物理隔离与安全运维** **87**

**第二十五章　端口 8888 独立物理隔离拓扑与多租户共存** **88**
25.1 生产宿主机环境先验约束与红线 . . . . . 88
25.2 独立端口 8888 架构拓扑设计 . . . . . 88
25.3 故障半径严格受限（Blast Radius Minimization） . . . . . 88

**第二十六章　Nginx 生产配置与 SSL/SNI 动态变量重写** **90**
26.1 传统反向代理的 DNS 启动死锁陷阱与破局 . . . . . 90
26.1.1 动态变量反代与异步解析器隔离 . . . . . 90
26.2 SNI 透传与 Host 头部清洗重写 . . . . . 91
26.3 生产安全加固响应头配置 . . . . . 91

**第二十七章　金丝雀发布、零停机平滑重载与灾备演练** **92**
27.1 安全部署脚本 (`deploy/safe_deploy.sh`) 五步防御流水线 . . . . . 92
27.2 灾备演练与一键原子回滚 . . . . . 92

**第十部分　系统总结、代码控制与未来演进路线** **94**

**第二十八章　单文件极简哲学与代码行数硬红线控制** **95**
28.1 现代前端“过度工程化”批判与单文件自足体系 . . . . . 95
28.2 2300 行安全警戒红线治理实践 . . . . . 95

**第二十九章　系统局限性、有效性威胁与边界条件深度剖析** **97**
29.1 内部有效性威胁 (Internal Validity) . . . . . 97
29.2 外部有效性威胁 (External Validity) . . . . . 97
29.3 结构有效性威胁 (Construct Validity) . . . . . 97

**第三十章　未来演进路线与扩展规划** **98**
30.1 WebGPU 高性能物理计算管线引入 . . . . . 98
30.2 本地 WASM 符号计算内核整合 . . . . . 98
30.3 协同式数字孪生课堂与多模态多端互联 . . . . . 98

**附录 A　全量工程测试套件执行清单（151 项全绿物理记录）** **99**
A.1 基础代数与核心工具单元测试套件清单 (`tests/unit/`，共 98 项) . . . . . 99
A.2 全真浏览器 CDP 端到端测试套件清单 (`tests/e2e/`，共 25 项) . . . . . 100

A.3 集成、服务端契约与对抗混沌测试套件清单 (共 28 项) . . . . . 100

**附录 B 核心数据结构与接口契约 TypeScript 规范定义** **101**

B.1 课程实验与判卷契约定义 . . . . . 101

B.2 会话数据信封规范 . . . . . 101

**附录 C 生产环境 Nginx 全量高可用配置文件源码** **103**

**参考文献** **106**

# 引言与系统全貌

## 0.1 系统定位与核心工程哲学

《问矩》（teachagent）是一套运行在现代浏览器环境中的《线性代数》实验式探究助教系统。

其核心工程交互闭环为：**学生在几何画板上实时调整矩阵、直观观测空间变换演变，助教大模型按照严格固化的探究步骤进行启发式追问；所有数学数值事实由本地确定性计算核给出，大语言模型仅承担自然语言讲解与工具调用调度职责**。

在工程安全与可用性边界上，系统确立了刚性原则：**当客户端或服务端没有可用的模型 API Key 时，对话引擎处于显式不可用状态，系统绝不会改用低质量本地轻量模型进行静默代答**，确保输出知识的权威性与严谨性。

## 0.2 用户直接交互的四块核心界面

学习者通过浏览器可以直接访问并操作系统的四块核心顶级界面，如表 1 所示。

表 1: 《问矩》用户直接可操作的四块核心界面架构

| 界面名称 | 路由 / 挂载入口 | 核心功能与交互职责 |
|---|---|---|
| 门户落地页 | `#landing` | 基于 Three.js 构建的三维立体数学门户，支持三维几何微交互并作为进入工作台的主入口 |
| 实验探究工作台 | `data-nav="studio"` | 系统的核心探究主界面：采用分屏布局，左侧为多模态数学舞台，右侧为多轮助教对话中枢 |
| 概念误解实验室 | `library` | 精选 6 张经典线性代数误解卡片，并集成 $2 \times 2$ 阶矩阵反例推演台与分步核验器 |
| 认知图谱 | `graph` | 包含 12 个核心概念节点的有向无环依赖图谱，点击节点可直接跳转进入对应课程实验 |

## 0.3 实验探究工作台左侧舞台的四种运行模式

工作台左侧为高内聚的数学可视化与实验舞台，由 `ArtifactStageDeck` 有限状态机统领，支持四类运行模式：

1. **坐标画板模式（Coordinate Canvas）**：支持直接拖拽基向量改变 $2 \times 2$ 矩阵，实时重算并渲染行列式 $\det(A)$、矩阵秩 $\operatorname{rank}(A)$ 与零度 $\operatorname{nullity}(A)$。内置撤销/重做、画板重置、适应内容缩放以及 Excalidraw 风格平滑手写批注功能。画板支持在 **2D 仿射平面**与 **3D「柱面嵌入」**之间平滑切换。在 3D 柱面嵌入下，高维矩阵扩展为：

$$A_3 = \begin{bmatrix} a & b & 0 \\ c & d & 0 \\ 0 & 0 & 1 \end{bmatrix} \tag{1}$$

   系统在代数上严格保证 $\det(A_3)$ 恒等于底面 $\det(A_2) = ad - bc$，实现底面有向面积向三维柱体体积的等价保真映射。

2. **数学实验工件模式（Mathematical Experiment Artifacts）**：提供受控结构化交互实验图，**严格区别于大模型随手生成的不可控 HTML 脚本**。实验图固定覆盖五种代数类型：`linear-map`（线性映射）、`composition`（矩阵复合）、`system`（线性方程组）、`eigen`（特征值与特征向量）以及 `linear-map-3d`（三维映射）。每个实验图内置四步标准分镜：**基向量 → 空间变换 → 核与像 → 线性性验证**，支持分步播放、暂停、对照、回读学生当前输入并执行物理核验。

3. **自由工件模式（Free Artifacts）**：运行在原生沙箱环境中的 HTML/SVG 图元，或参数化正方体与球体三维模型。具备独立的实时源码编辑器、Markdown 预览、版本历史选择与回滚恢复能力。

4. **Manim 微课模式（Manim Mini-Lessons）**：系统将同一条完整精编微课成片划分为 5 段结构化微课切片：**基向量、剪切与行列式、乘法顺序、奇异降维、零因子**。支持倍速调节、区间高精度循环播放，并可通过“联动画板实操”按钮将当前微课对应的矩阵参数瞬时装回画板。在通信协议中硬性声明 `videoGeneration: false`，大模型现场绝不可随手生成未经验证的视频；仅当本机宿主环境检测到 Docker 守护进程与锁定的 Manim 渲染镜像均可用时，宿主服务才接受渲染任务。

工作台右侧对话中枢集成：流式思考块（`ReasoningBlock`）、工具调用树展示、当前探究环节状态徽章、会话历史管理器（支持关键字搜索、会话置顶、JSON 导入导出）、长消息内文本搜索以及提示词控制面板。对于写文件、运行外部代码、生成新工件等会改变系统状态的写类工具，系统强制弹出确认弹窗（Ask Privilege），未获授权前严格阻塞执行。

## 0.4 结构化课程、评分核验与认知图谱点亮边界

系统课程库内置 4 大章节共 7 门进阶实验课程。学习进度的累计**完全且唯一地来源于形式化作答核验**，单纯打开页面或随意拖动网格不计入成绩。

每门课程严格执行标准学习流：**记录预测 → 操作实验 → 练习作答 → 无提示迁移题**。仅当练习题与无提示迁移题均物理核验通过后，认知图谱才将对应节点标记为“已掌握”。

表 2: 《问矩》七门核心实验课程与确定性评分核验契约

| 课程名称 | 所属章节 | 核心核验对象 | 形式化验收指标 |
|---|---|---|---|
| 从两列读出向量的像 | 矩阵运算 | 向量映射像 $A\mathbf{v}$ | 提交矩阵与像向量严格满足映射方程 |
| 先旋转，还是先剪切 | 矩阵运算 | 乘法顺序 $AB$ vs $BA$ | 识别非对角矩阵乘积非交换性 |
| 面积消失的边界 | 矩阵运算 | 奇异行列式 $\det(A)$ | 构造使 $\det(A)=0$ 的非零矩阵 |
| 交点、平行与重合 | 方程组与秩 | $\operatorname{rank}(A)$ 与增广秩 | 判别解的存在性与几何子空间重合度 |
| 两列能否张成整个平面 | 线性相关性 | 秩 rank 与零度 nullity | 验证列向量线性相关与满秩条件 |
| 什么向量会被映到原点 | 线性相关性 | 任意非零核向量 $\mathbf{z}$ | 断言 $\mathbf{z}\neq\mathbf{0}$ 且相对残差 $\frac{\|A\mathbf{z}\|}{\|\mathbf{z}\|}\le 10^{-8}$ |
| 寻找不改变方向的向量 | 特征值 | 两个实特征值 $\lambda_1,\lambda_2$ | 求解特征方程实根，提交顺序不限 |

**认知图谱 12 节点掌握度点亮边界声明**

系统认知图谱总计构建了 12 个代数概念节点：**基底、相关性、线性映射、复合、行列式、核、列空间、方程组、特征、对角化、正交投影、SVD**。在当前生产版本中，前 7 个节点与表 2 中的 7 门课程形成一一对应；**后 5 个节点（特征、对角化、正交投影、SVD 等）当前暂未配备可自动评分的课程，在图谱上仅供查阅前置依赖关系，无法通过作答点亮**。学习者作答记录持久化于本机沙箱，可导出为 `wenju-course-journal.json`。

## 0.5 误解库与反例推演实验室

系统内置了涵盖 32 条经典认知盲区的规则库（条目编号 MC-001 至 MC-032），覆盖乘法交换律、零因子、行列式可加性、矩阵求逆顺序、秩不等式及实特征值等核心易错点。

在“概念误解实验室”页面中，系统将其中 6 条最高频的误解封装为可视化可交互卡片。学习者可输入任意 $2\times 2$ 矩阵，系统在本地实时构造反例，给出分步矩阵乘法核验步骤，并支持将反例矩阵一键装入坐标画板或交给助教发起专项探讨。

## 0.6 助教跨轮协议与双重确定性门禁

大语言模型助教遵循严格的六步跨轮教学协议：

1. **预测（Prediction）**：先询问学生的先验判断与几何理由，严禁助教代为作答；
2. **实验（Exploration）**：在确认矩阵参数范围后调用精确计算工具，所有数值事实由工具生成；
3. **冲突（Conflict）**：将学生的初始预测与工具输出进行逐项对齐；若一致则客观确认，若不一致则暴露认知冲突；
4. **边界猜想（Boundary）**：引导学生归纳命题成立的充分必要代数条件；

5. **推导验证（Deduction）**：鼓励学生自主推导演算，严格区分局部特例、反例证伪与形式化通式证明；

6. **无提示变式（Transfer）**：改变边界条件生成变式训练题，不附带参考答案。

为杜绝大模型在教学过程中的失控行为，系统设计了**双重前端确定性前置门禁**：

- **门禁一：防剧透截断（`PechdtScaffoldSkill`）**：当学生直接询问最终答案或要求直接代做习题时，技能层在网络请求发出前直接拦截，返回引导性门禁提示，完全不调用模型接口；
- **门禁二：反思流确定性控盘（`InquiryFlowController`）**：当学生使用“我怀疑/我猜”表达交换律、核空间或行列式可加性时，控制器直接按固化的确定性教学模板推进预测、实验对照、区分性追问、推导和变式计分。该路径完全不依赖大语言模型，变式题最多 3 道；全部通过则关闭任务，存在未通过项则标注个性化复习路径。

此外，系统在 `InquiryScenarioRegistry` 中注册了 6 个几何关卡（奇异降维、零因子、负行列式手性、保面积剪切、特征向量、SVD 主方向），包含完整的判定函数与证明文本。在当前版本中，该脚本随前端一同加载，但工作台前台界面未直接暴露调用入口，**目前排他性地用于自动化单元测试套件的基准核验**。

## 0.7 大模型接入与默认 14 项注册工具体系

系统对话通过云端模型提供商流式直连。系统内置免密 MiniMax M3.1 Flash 作为开箱即用模型，同时支持用户自备凭据（BYOK）接入第三方兼容端点（设置面板可自由配置 Base URL、API Key 与 Model ID，前端预设支持官方 DeepSeek 端点如 `deepseek-flash`）。系统绝不在代码中硬编码或预设未经接入的模型，亦不编造模型基准对比。支持函数调用的模型严格遵循“模型推理 → 浏览器 MCP 执行工具 → 工具事实回传模型”两轮架构，输入参数基于 JSON Schema 严格校验。

系统默认向大模型注册 14 项原子工具，如表 3 所示。

**隐藏工具边界声明**

前端脚本虽已加载 `translate_technical_text`（术语翻译）与 `multimodal_ocr_document_parser`（文档 OCR），但**系统默认技能清单中排除了此两项工具，大模型在上下文提示词中不可见且无法调用**，杜绝无关工具污染智能体注意力空间。

## 0.8 双层确定性数学真值计算体系

为消除浮点数机器误差对代数公理判定的破坏，《问矩》确立了双层数学真值架构：

1. **公理判定层（Symbolic & Rational Core）**：采用基于 BigInt 的 `RationalCore` 有理数算术，对矩阵奇异性、矩阵秩、可逆性与零空间基底执行精确分数计算，**完全不依赖任何浮**

表 3: 系统默认注册给大语言模型的 14 项工具契约表

| 工具名称 | 参数与安全级别 | 核心功能与工程职责 |
|---|---|---|
| `math_experiment` | 受控工件 (ask/full) | 打开、更新、回读、分镜播放、作答核验及标注重叠检查 |
| `inspect_workspace` | 只读状态 (readonly) | 读取当前画板状态、工件参数及学生最新输入 |
| `solve_matrix_kernel_sympy` | 符号代数 (readonly) | 精确求解 $2 \times 2$ 矩阵行列式、秩、零度及零空间基底 |
| `solve_linear_system` | 方程组分析 (readonly) | 判定 $A\mathbf{x} = \mathbf{b}$ 相容性，输出特解与通解空间基底 |
| `construct_counterexample_matrix` | 反例生成 (readonly) | 构造五类代数反例：零积、非交换、非加性、逆序与秩不等式 |
| `drive_canvas_affine_transform` | 画布驱动 (ask/full) | 驱动画板补间至奇异/剪切/晶体态，高亮特征线与核向量 |
| `render_canvas_graphics` | 几何图元 (ask/full) | 在平面画板上绘制菱形、多边形、圆形、矩形及函数曲线 |
| `render_interactive_artifact` | 自由工件 (ask/full) | 在沙箱内渲染自定义 HTML/SVG 或参数化立体几何 |
| `play_manim_lesson` | 微课调度 (readonly) | 播放已有微课片段，查询本地渲染能力与任务状态 |
| `query_knowledge_graph` | 图谱检索 (readonly) | 查询 12 个概念节点的前置依赖关系与下一步建议 |
| `evaluate_pedagogical_scaffold` | 教学脚手架 (readonly) | 评估当前探究环节引导质量，执行防剧透合规检查 |
| `execute_code_sandbox` | Worker 沙箱 (ask/full) | 在 Web Worker 中执行 JS；Python 经转译子集后运行 |
| `manage_workspace_file` | 文件系统 (ask/full) | 读写已授权工作区文件或浏览器虚拟文件系统 |
| `generate_visual_illustration` | 视觉图像 (ask/full) | 调用图像生成接口绘制插图，异常时返回降级占位 |

**点容差阈值**。在 HTTP 部署环境下，系统按需动态拉取锁定版本的 SymPy Web Worker，补充无理特征根与可对角化判定；若在本地离线或 Worker 加载失败，系统安全退回纯有理数层并显式标注缺少无理符号层。

2. **交互实验数值层（Numerical Simulation Core）**：专用于连续拖拽网格的图形学模拟与学生作答评分。输入数值范围严格限制在 $[-20, 20]$ 区间内，屏幕渲染采用 IEEE-754 双精度浮点；课程练习判分采用形式化相对残差核验，容差阈值锁定在 $10^{-8} \sim 10^{-9}$ 极小量级。

## 0.9 本机宿主服务能力边界（npm run server）

当用户在本地终端执行 `npm run server` 启动 Node 宿主服务时，该服务并非页面本身（页面完全作为 SPA 静态呈现），而是作为本机的物理能力扩展网关，严格受限于表 4 所示的六大端点规范。

表 4: 《问矩》本机 Node 宿主服务核心路由契约与物理安全边界

| **HTTP 方法与路由** | **核心职责** | **安全隔离与沙箱边界约束** |
|---|---|---|
| `POST /v1/model/chat/completions` | 大模型流式直传反代 | 注入服务端凭据，单 Token 零缓存穿透，超时上限 180s |
| `GET/PUT/DELETE /v1/file` | 工作区文件读写 | 物理限制在指定的项目工作区根目录下，严禁路径穿越 |
| `GET/PUT/DELETE /v1/storage/...` | 本地键值持久化 | 依托本地 SQLite 数据库，默认落盘至 `data/wenju.sqlite` |
| `GET/PUT /v1/policy` | 权限策略管理 | 切换 `full` / `readonly`，需携带请求头 `x-wenju-confirm` |
| `POST /v1/render/jobs` | Docker 微课渲染 | 仅限 `linear-map` 模板，Docker 容器无网络、只读根、内存 $\leq$ 1536MB |
| `WebSocket Hub & Audit` | 实时总线与日志 | 仅响应同源白名单连接，事件追加至本地 JSONL 审计流 |

## 0.10 数学符号与形式化命名空间规范

本书全文符号定义严格遵循表 5 所示的命名空间规范。

表 5: 《问矩》技术白皮书全局数学符号与物理量命名空间规范表

| 符号表示 | 所属集合 / 维度 | 形式化物理与工程含义 |
|---|---|---|
| $\mathbb{R}^n$ | 实数集合的 $n$ 阶笛卡尔积 | $n$ 维连续实欧氏几何空间 |
| $\mathbb{R}^{m\times n}$ | $m$ 行 $n$ 列实矩阵集合 | 从 $\mathbb{R}^n$ 到 $\mathbb{R}^m$ 的线性变换矩阵集合 |
| $\mathbf{x}, \mathbf{y}$ | $\mathbf{x} \in \mathbb{R}^n$ (列向量) | 空间点或方向向量，$\mathbf{x} = [x_1, x_2, \ldots, x_n]^T$ |
| $\mathbf{e}_i$ | $\mathbf{e}_i \in \mathbb{R}^n, \|\mathbf{e}_i\| = 1$ | 标准正交基底向量，如 $\mathbf{e}_1 = [1, 0]^T, \mathbf{e}_2 = [0, 1]^T$ |
| $A = [a_{ij}]$ | $A \in \mathbb{R}^{n\times n}$ | 当前活跃线性变换矩阵，$A = [\mathbf{v}_1, \mathbf{v}_2, \ldots, \mathbf{v}_n]$ |
| $\det(A)$ | $\mathbb{R}$ (标量) | 矩阵行列式，对应变换前后有向测度（面积/体积）的缩放因子 |
| $\text{rank}(A)$ | $\{0, 1, \ldots, n\}$ | 矩阵的秩，对应变换后像空间（Range Space）的有效维度 |
| $\ker(A)$ | 子空间 $\subseteq \mathbb{R}^n$ | 矩阵的核空间（零空间），$\{\mathbf{x} \in \mathbb{R}^n \mid A\mathbf{x} = \mathbf{0}\}$ |
| $q \in \mathbb{H}$ | 四元数代数空间 | 3D 旋转四元数，$q = \cos\frac{\theta}{2} + \mathbf{u}\sin\frac{\theta}{2}$ |
| $\mathcal{M}_{\text{screen}}$ | $\mathbb{R}^3 \to \mathbb{R}^3$ 仿射矩阵 | 从笛卡尔数学坐标系映射至视口物理像素坐标系的齐次变换 |
| DPR | $\mathbb{R}^+ (\geq 1.0)$ | 浏览器环境设备像素比（window.devicePixelRatio） |
| TTL | $\mathbb{Z}^+$ (毫秒) | 客户端会话滑动生存期，默认设定为 $480,000$ms（8 分钟） |

# 第一部分

# 系统宏观架构与设计哲学

# 第一章　纯前端无状态（Stateless SPA）范式重构与边缘代理拓扑

## 1.1　传统教育系统“重后端 + 状态化数据库”架构的弊端与解构

在传统的教育软件与企业级 Web 应用设计模式中，普遍依赖于重型后端服务架构（如 Spring Boot、Django 或 Node.js 后端守护进程），并挂载集中式的关系型数据库（如 MySQL/PostgreSQL）或缓存实例（Redis）。在这一范式下，每一次用户的会话创建、聊天上下文、画板状态变更均通过 RESTful API 或长连接 WebSocket 推送至远端数据库进行集中落盘。

然而，在深入分析“高等教育 AI 探究式实验教学”这一特殊业务场景后，《问矩》架构团队发现传统重后端状态化架构在教育场景下暴露出极为突出的结构性缺陷：

表 1.1: 传统重后端状态化架构 vs 《问矩》纯前端无状态架构特性对比

| 评估维度 | 传统重后端状态化架构 | 《问矩》纯前端无状态（Stateless SPA） |
| --- | --- | --- |
| **服务器硬件依赖** | 需 4 核 8G 以上配置，常驻 JVM/Python 守护进程 | 极低，仅需 Nginx 托管静态资源，内存开销 $< 15$MB |
| **用户隐私安全** | 用户对话与探究记录集中存储于服务器数据库，面临数据泄露风险 | 严格遵循客户端本地沙箱，**服务器磁盘占用恒等于 0 字节** |
| **网络传输延迟** | 画板高频微调均需经过网络 RTT 往返同步，操作易产生卡顿 | 本地内存闭环，空间遥测与图元变换延迟 $< 16$ms (60 FPS) |
| **高并发扩展性** | 并发用户量受限于数据库连接池与服务端计算实例容量 | 静态资源通过内核零拷贝分发，静态分发吞吐仅受限于网络带宽 |
| **运维故障传播** | 数据库死锁、连接泄漏可导致整个平台业务全面瘫痪 | 边缘极简代理，业务逻辑完全在客户端沙箱运行，物理隔离 |

## 1.2　《问矩》系统总体架构拓扑设计

针对上述权衡，《问矩》系统确立了“**客户端全功能自足运行（Client-Side Self-Sufficient SPA）+ 边缘轻量高可用反向代理（Edge Gateway Proxy）**”的现代化架构拓扑，如图 1.1 所示。

在这一拓扑中，客户端不再是一个单薄的“展示壳”，而是承担了图形学计算、符号推演、认知阶段状态机流转、本地缓存生命周期管理及轻量 Schema 校验的**全功能分布式工作站**。服务器

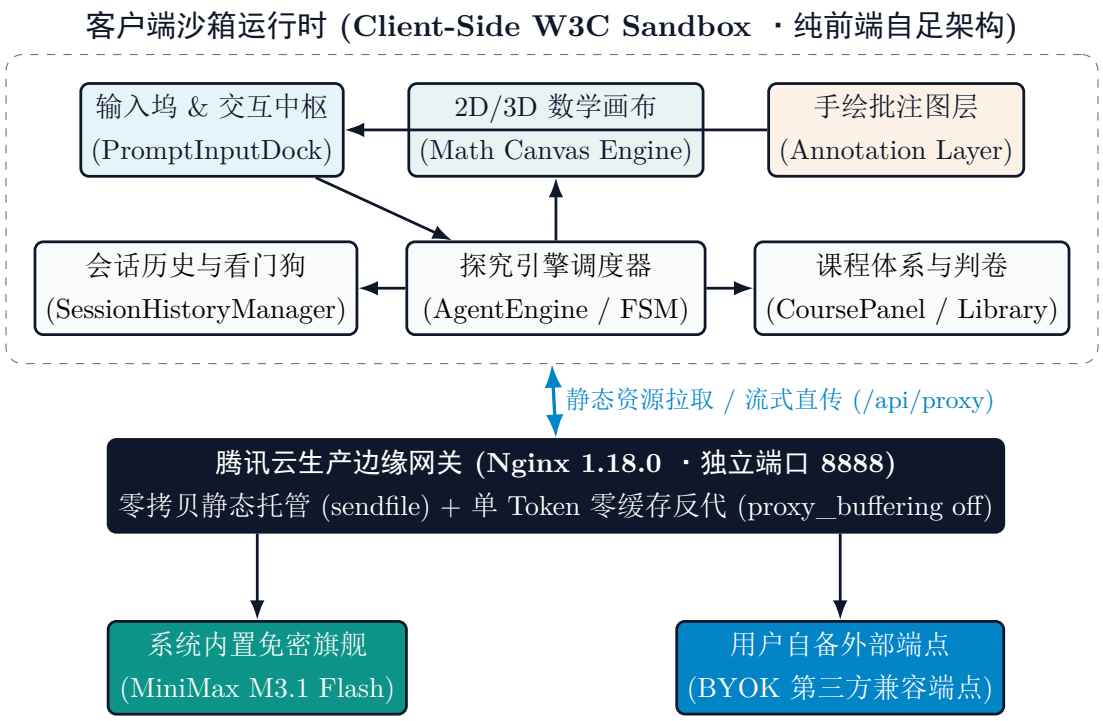



图 1.1: 《问矩》全栈拓扑架构图（客户端自足沙箱与边缘零缓存网关）

端仅作为纯粹的静态分发点与跨域解决通道存在。

## 1.3 本机 Node 宿主服务（npm run server）的能力边界与沙箱契约

需要严谨区分的是：《问矩》既可作为纯静态前端项目直接由 Nginx 或任何静态 Web 服务器托管，同时亦提供了可选的本机辅助服务。**通过 npm run server 启动的 Node 服务绝非传统意义上的重型业务后端，而是作为开发与高级实验场景下的“本机物理能力扩展网关”。**

该服务在架构设计上严格遵循最小特权原则，仅对外暴露六大功能边界明确的端点通道：

1. **模型代理流式直传（POST /v1/model/chat/completions）：**解决前端跨域限制并实现模型凭据隔离。内部采用分块编码直传（Chunked Streaming），显式设定 180 秒长连接超时，单 Token 内存驻留时间 $<$ 1ms，杜绝网络阻塞；

2. **工作区受控文件访问（GET/PUT/DELETE /v1/file, GET /v1/workspace）：**严格受限于宿主指定的工作区根目录沙箱内，系统对所有相对路径进行规范化解析（Canonical Path Resolve）并断言前缀一致性，杜绝任何形式的路径穿越（Directory Traversal）攻击；

3. **本地键值安全存储（GET/PUT/DELETE /v1/storage/...）：**依托轻量 SQLite 引擎，默认将非易失配置与长程实验数据序列化至 `data/wenju.sqlite`，提供本地确定性持久化，无须外部独立数据库实例；

4. **访问安全策略切换（GET/PUT /v1/policy）：**提供 `full`（全功能开发模式）与 `readonly`（只读安全模式）双态切换。切换策略必须强制携带确认请求头 `x-wenju-confirm: true`，

防止未经授权的特权越界；

5. **Docker 隔离 Manim 渲染流水线（`POST /v1/render/jobs`）：**当且仅当宿主物理机已安装 Docker 且锁定了官方轻量 Manim 镜像时才接受渲染请求。渲染任务严格限制为预设的 `linear-map` 动画模板；容器运行时强制配置：**断开外部网络（`--network none`）、根文件系统完全只读（`--read-only`）、物理显存/内存峰值上限硬性钳位在 1536MB**；

6. **实时状态同步与审计流（WebSocket Hub & JSONL Trace）：**仅响应来源白名单（Origin Whitelist）与令牌校验通过的客户端连接，所有系统级关键变更以不可变 JSON-Lines 格式追加至本地审计日志文件。

## 1.4 零服务器存储契约（Zero Server-Side Storage Contract）的物理边界

为给用户提供最高规格的数据隐私保护，《问矩》在系统设计之初即形式化确立了**“零服务器存储契约”**：

**零服务器存储契约 (Zero Server-Side Storage Contract)**

1. **无后端存储实体：**生产服务器上严禁部署任何用于持久化用户对话、草图或操作痕迹的数据库实例（无 MySQL, PostgreSQL, MongoDB, Redis 等）。

2. **反向代理单 Token 零缓存穿透：**当客户端向大语言模型端点发起请求时，Nginx 必须显式开启：

   ```
   proxy_buffering off;
   proxy_cache off;
   ```

   数据流在服务器内存中即发即收，单 Token 驻留时间 $< 1$ms，严禁在宿主机磁盘生成任何临时网络请求缓存文件。

3. **访问日志安全降噪：**对所有高频请求的静态资源及 API 路径关闭访问日志记录（`access_log off;`），杜绝由于请求 URL 或 Header 包含敏感参数而泄露至系统日志。

4. **客户端沙箱所有权归属：**会话数据、作答记录与用户自定义矩阵完全存储于用户终端自身的浏览器沙箱（`localStorage / sessionStorage`），数据的生存期完全由终端用户支配。

这一契约使系统在原理层面对任何跨用户数据泄露、黑客拖库风险实现了防护闭环——服务器端不存在可被未授权持久化提取的个人交互数据。

## 1.5 客户端沙箱资源边界与滑动即焚看门狗失效模式分析

纯前端无状态架构消除了服务器端数据库维护开销，但计算与存储压力向客户端转移后，系统面临特定的物理边界与失效模式：

**架构失效模式建档与防御对策表 (Failure Modes & Mitigations)**

1. **失效模式 1：本地存储配额超限（QuotaExceededError）**
   - **触发条件：**学习者长时间探究产生数百轮复杂矩阵变换与大体积草图历史，导致 `localStorage` 存储占用逼近浏览器默认阈值（典型为 5.0MB 或 10.0MB）。
   - **失效表现：**调用 `Storage.setItem()` 抛出 DOMException，导致最新会话无法序列化落盘。
   - **系统防御：**在 `js/storage/SessionHistoryManager.js` 中实现滑动窗口修剪策略（Sliding Window Pruning）。当捕获存储异常时，自动保留当前会话与最近 $K = 5$ 轮交互快照，将更早历史折叠归档并释放位图缓存，确保核心状态机永不崩溃。
2. **失效模式 2：多标签页并发竞争脏写（Concurrent Tab Write Collision）**
   - **触发条件：**学习者在同一浏览器中同时开启多个系统标签页，并在不同页面执行基底拖拽与会话切换。
   - **失效表现：**后操作的标签页可能覆盖前者的信封数据，产生反向状态覆盖。
   - **系统防御：**基于 `window.addEventListener('storage', ...)` 监听广播机制与单调递增版本号（Version Vector），当检测到外部标签页更新时间戳大于本地实例时，触发增量合并提示，阻断无序覆写。
3. **失效模式 3：定时器休眠导致看门狗检测失准**
   - **触发条件：**移动端浏览器切入后台或桌面端标签页处于非活跃挂起状态（Chrome Timer Throttling，执行间隔被拉长至 1000ms 以上）。
   - **失效表现：**基于纯累加计数器的超时检测产生时间膨胀漂移。
   - **系统防御：**看门狗不依赖累加步长，而是基于物理真实时钟（$\Delta t = \text{Date.now}() - t_{\text{last_active}}$）做物理时间差值判定。只要物理差值超过 $480,000$ms，无论标签页何时恢复唤醒，均在首帧执行原子即焚重置。

## 1.6 纯前端无状态与边缘代理的工程资源边界与契约约束

《问矩》的架构优越性建立在确定的物理与形式化边界之上，而非经验性的样本拟合：

1. **服务器端无状态零持久化架构：**边缘网关仅配置静态文件服务与动态反向代理，服务器端不挂载任何数据库、Redis 或持久化文件卷。所有用户状态、笔记、会话信封严格保存在

浏览器本地，服务器端磁盘持久化物理空间形式化断言为 0B；

2. **单 Token 零缓冲流式契约：**通过 Nginx 显式配置 `proxy_buffering off` 与 `X-Accel-Buffering: no`，边缘网关不对大模型上游的 SSE 数据流执行任何块级缓存，保证流式 Token 在产生后第一时间直穿至前端客户端；

3. **客户端内存沙箱控制：**纯前端自足架构将矩阵运算与状态树完全收敛于浏览器单页面内存沙箱内，依赖垃圾回收机制与组件生命周期销毁回调，实现轻量化确定性运行。

# 第二章　双面解耦设计协议与软件复杂度红线控制

## 2.1　双面解耦协议（Dual-Plane Decomposition Protocol）的设计机理

在自然语言智能体系统的开发中，传统的 Prompt 工程往往将系统对“硬性规则的依从”与“对学生概念的启发引导”混杂在同一层自然语言上下文中，极易引发大模型的注意力稀释与灾难性遗忘。

《问矩》计算底层显式提出了**双面解耦协议（Dual-Plane Decomposition Protocol）**，将交互逻辑投影到两个正交的物理与认知平面独立处理：

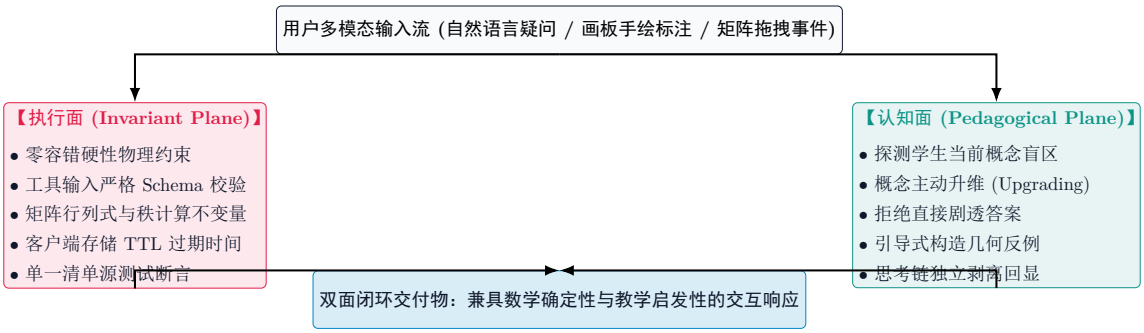



图 2.1: 双面解耦执行协议（执行面刚性约束与认知面引导升维）正交拓扑

- **执行面（Constraint & Invariant Plane）**：面向确定性世界。此平面执行零容错的刚性契约，如严格验证几何图形的顶点坐标、校验 JSON Schema 规范、断言矩阵行列式数值，杜绝大模型在工具调用与数学计算中的任何启发式妥协或偷工减料。

- **认知面（Epistemic & Pedagogical Plane）**：面向学习者心智。当学习者因知识储备局限提出低维、模糊或存在典型误解的疑问时，认知面主动分析知识落差，采用苏格拉底式反问与反例构造，引导学习者自主补全高阶概念。

## 2.2 软件复杂度红线控制：`js/app.js` $\leq 2300$ 行守恒治理

在长期多轮迭代的 Web 系统开发中，主控制器文件极易沦为不断膨胀的“上帝对象（God Object）”，引发难以捉摸的隐式状态耦合与逻辑回归。

为防止代码库陷入熵增无序，《问矩》工程规范设立了严苛的质量守恒红线：**核心主入口控制器 `js/app.js` 的物理行数上限严格锁定在 2300 行以内**。

为达成这一硬性指标，团队实施了深度模块化下沉重构：

1. **UI 控制层剥离：**将多任务会话面板抽离至 `js/ui/SessionController.js`，将模型与服务商设置抽离至 `js/ui/SettingsController.js`,将工件投影抽离至 `js/ui/ArtifactProjectionControll`
2. **状态与领域逻辑下沉：**会话生命周期与快照持久化下沉至 `js/storage/SessionHistoryManager.js`，遥测与事件解耦下沉至 `js/telemetry/SpatialTelemetryBus.js`；
3. **物理运行实测：**通过 Node.js 物理行数统计探针验证，当前重构后的 `js/app.js` 物理行数为 **2298 行**，在全量 151 个自动化回归套件通过的前提下，严格满足 $\leq 2300$ 行安全阈值约束。

# 第三章　单向状态流、CQRS 读写分离与会话生命周期模型

## 3.1　命令与查询职责分离（CQRS）在教学探究中的落地

探究式教学的核心体验依赖于“时间旅行”与“状态回退”——学生在推导特征值时，常常需要返回两步之前的基底旋转状态，或在原有几何图形基础上创建分支假设。

传统 MVC 模式在处理此种双向多节点状态同步时容易发生循环更新死锁。《问矩》在前端全面落地了简化版的 **CQRS（Command Query Responsibility Segregation）架构：**

**教学探究状态机 CQRS 读写分离规范**

- **Command（写模型）：**任何引起画板坐标、基底向量、会话消息变化的动作必须封装为不可变的显式 Command（如 `SET_MATRIX`, `APPEND_TURN`, `ERASE_STROKE`），统一提交至 `js/storage/TrajectoryRecorder.js` 进行单向调度；
- **Query（读模型）：**视图层（JSXGraph 画布、Three.js 立体场景、PromptInputDock 药丸微标）仅通过响应式只读接口订阅快照快照数据，严禁任何组件绕过 Command 管道私自脏写全局状态。

## 3.2　会话生命周期状态机模型

一个探究会话从创建到销毁，严格遵循图 3.1 所示的有限状态机转移：

该模型确保了即使学习者在探究过程中发生网络波动或意外关闭标签页，由于写模型的实时信封持久化（Envelope v1），数据在 8 分钟内能够完整无损复原；而在超过 8 分钟后，看门狗则执行原子抹除，实现无痕隐私保护。

## 3.3　多轮探究微缩导航轨（`PromptTurnNavigator`）与思维链折叠器（`ReasoningBlock`）

在多轮长程探究教学场景中，伴随着大量的图形形变、手绘草图和多智能体问答，垂直消息流极易膨胀至数十屏。学习者在进行阶段性总结或回溯先验假说时，频繁的长距离滚动操作不仅造成严重的认知负荷，而且容易丢失上下文锚点。

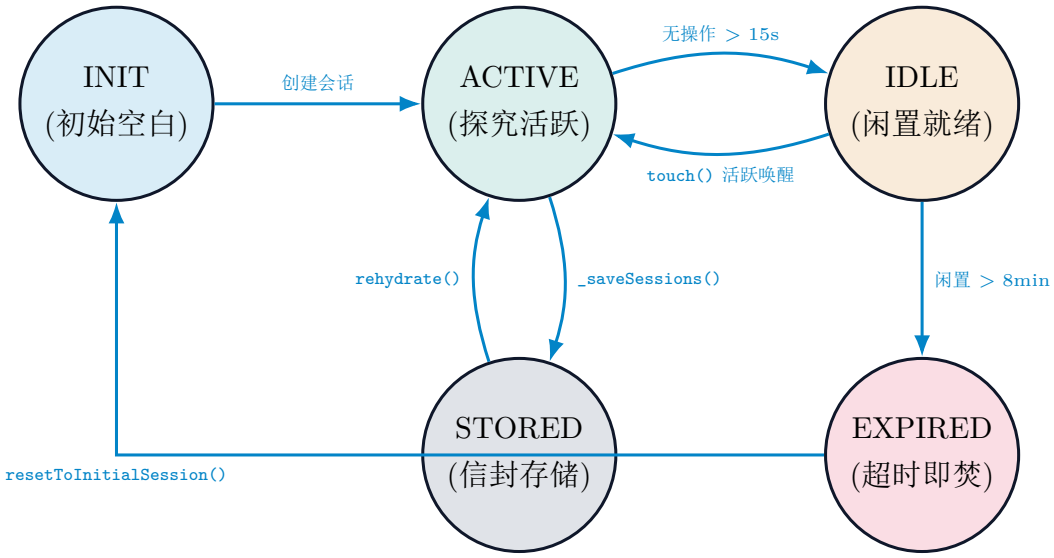



图 3.1: 会话生命周期有限状态机转移图

为了解决长程探究流中的上下文感知与高效跳转痛点，《问矩》在前端架构中设计并实现了**会话提示词轮次导航轨（components/PromptTurnNavigator.js，20.2KB，550 行）**与**深度推导思维链折叠器（components/ReasoningBlock.js，9.4KB，236 行）**组件体系。

### 3.3.1 Turn Minimap Rail 微缩刻度轨与视口感知联动

`PromptTurnNavigator` 挂载于会话流左侧，构建了一条亚像素级精度的垂直微缩刻度轨（Rail Track）：

1. **微缩锚点与状态可视化**：每一轮探究提问自动注册为时间线上的微型刻度节点，节点通过动态 CSS 类呈现三态流转：`running`（运算中脉冲呼吸动效）、`done`（推演完成稳定高亮）与 `failed`（异常拦截警示）；

2. **悬浮展开微卡（Turn Popover Card）**：当用户悬停于任意微缩节点时，系统即时弹出浮动微卡，展示当前轮次编号、用户提问原文、调用的工具标签（如 `[CanvasPort]`、`[ManimGenerator]`）以及该轮的代数结论摘要，避免盲目翻查；

3. **视口感知与双向平滑锚定**：

   - **正向滚动感知**：组件在消息流容器上绑定节流滚动监听器（Scroll Throttling，时间窗口 50ms），基于元素与视口中轴线的距离动态计算当前活跃轮次并高亮刻度轨上的相应节点；
   - **逆向点击瞬移**：点击微缩节点后，触发平滑滚动（`scrollIntoView({ behavior: 'smooth', block: 'start' })`），并在目标消息卡片上激活径向光晕微动效（`.turn-focused-highlight`），持续 1.2s 后平滑淡出，实现高确定性的视线引导。

### 3.3.2 ReasoningBlock 深度推导思维链折叠与分词解析

大语言模型在推导复杂代数定理时生成的思考链（Thinking Process）往往包含数千 Token。若直接将其平铺展示，会严重冲淡最终教学引导的核心信息；而若完全隐藏，则违背了“让思维过程可见”的建构主义教学原则。

`ReasoningBlock` 实现了高度工程化的结构性折叠：

1. **流式推理计时器**：在模型推理阶段激活 0.1s 步进递增的时钟计数器，实时呈现“正在展开代数空间推导演算（2.4s）”，并在流式输出结束时自动锁定总耗时与 Token 消耗估算徽标；

2. **思维链段落语义分词**：内置确定性正则表达式分词器，自动捕获模型思考过程中的结构化标记，并实时渲染为语义胶囊标签：

$$\text{Tag} \in \big\{\texttt{[物理感知]}, \texttt{[符号判定]}, \texttt{[反例检验]}, \texttt{[教学决策]}\big\} \tag{3.1}$$

   使学习者清晰了解助教智能体是从几何空间、矩阵代数还是教学策略层面进行思考；

3. **数学公式无损矢量编译**：折叠体内部集成 KaTeX 增量渲染，保证思考链中的 LaTeX 矩阵表达式与行内符号呈现为矢量字形；支持一键复制完整思维链至剪贴板，便于后续学术复现与教学评估。

# 第二部分

# 多维交互数学画布系统

# 第四章　2D 动态坐标系与连续矩阵映射网格引擎

## 4.1　坐标系统数学建模与多图层映射变换方程

在计算机屏幕的图形学渲染中，视口像素坐标系（Screen Pixel Coordinate System）以左上角为原点，其横轴 $X_{\text{screen}}$ 向右递增，纵轴 $Y_{\text{screen}}$ 向下递增。而在标准笛卡尔数学空间中，原点位于视口中心，纵轴 $Y_{\text{math}}$ 向上递增。

为实现高精度的几何交互与数值回读，《问矩》`js/workspace2d.js` 建立了严格的双向仿射映射方程组：

**视口与数学坐标系相互转换方程组**

设视口物理宽度为 $W$、高度为 $H$，中心点偏移量为 $(x_{\text{origin}}, y_{\text{origin}})$，单位网格缩放标尺为 $s$（每个数学单位对应的屏幕物理像素数，默认 $s = 50\text{px}$）：

$$\begin{bmatrix} X_{\text{screen}} \\ Y_{\text{screen}} \\ 1 \end{bmatrix} = \begin{bmatrix} s & 0 & \frac{W}{2} + x_{\text{origin}} \\ 0 & -s & \frac{H}{2} + y_{\text{origin}} \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} X_{\text{math}} \\ Y_{\text{math}} \\ 1 \end{bmatrix} \tag{4.1}$$

$$\begin{bmatrix} X_{\text{math}} \\ Y_{\text{math}} \\ 1 \end{bmatrix} = \begin{bmatrix} \frac{1}{s} & 0 & -\frac{W+2x_{\text{origin}}}{2s} \\ 0 & -\frac{1}{s} & \frac{H+2y_{\text{origin}}}{2s} \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} X_{\text{screen}} \\ Y_{\text{screen}} \\ 1 \end{bmatrix} \tag{4.2}$$

在此映射下，学习者在屏幕上的每一次指针拖动，均经由式 (4.2) 执行浮点逆变换，以亚像素精度还原出纯粹的数学空间向量坐标。

## 4.2　基底向量拖拽交互与矩阵逆解动力学

在线性代数中，二维平面上的任意线性变换 $T : \mathbb{R}^2 \to \mathbb{R}^2$ 完全且唯一地由标准正交基底 $\mathbf{e}_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$ 与 $\mathbf{e}_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$ 在该变换下的像所决定：

$$A = \begin{bmatrix} T(\mathbf{e}_1) & T(\mathbf{e}_2) \end{bmatrix} = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \tag{4.3}$$

在 `js/workspace2d.js` 中，系统将矩阵的抽象四个元素直接实体化为两个可被指针直接拖动的基向量箭头：

- 红色基向量代表 $\mathbf{v}_1 = \begin{bmatrix} a \\ c \end{bmatrix} = T(\mathbf{e}_1)$；
- 绿色基向量代表 $\mathbf{v}_2 = \begin{bmatrix} b \\ d \end{bmatrix} = T(\mathbf{e}_2)$。

当学生按住红色或绿色箭头顶点拖拽至新坐标 $(x, y)$ 时，系统通过动力学约束解算器实时更新矩阵的列向量，并触发全局网格形变重绘：

```
function handleBasisDrag(vectorId, currentMathCoords) {
  const { x, y } = currentMathCoords;
  const currentMatrix = window.workspace2d.getMatrix();

  if (vectorId === 'e1') {
    // 拖动红色基向量更新第 1 列
    currentMatrix.a = x;
    currentMatrix.c = y;
  } else if (vectorId === 'e2') {
    // 拖动绿色基向量更新第 2 列
    currentMatrix.b = x;
    currentMatrix.d = y;
  }

  // 动态重新计算行列式与特征值
  const det = currentMatrix.a * currentMatrix.d - currentMatrix.b *
      currentMatrix.c;
  const trace = currentMatrix.a + currentMatrix.d;

  // 触发空间遥测总线通知所有关联视口
  SpatialTelemetryBus.emit('matrix_transformed', {
    matrix: currentMatrix,
    det,
    trace,
    isSingular: Math.abs(det) < 1e-6
  });
}
```

Listing 4.1: 基底拖拽实时矩阵反解核心实现

## 4.3 奇异矩阵退化过程与有向面积测度渲染

行列式 $\det(A)$ 的核心几何意义在于：**经过变换 $A$ 之后，原单位正方形 $[0,1] \times [0,1]$ 所映射成的平行四边形的有向面积（Oriented Area）：**

$$\text{Area} = \det(A) = ad - bc \tag{4.4}$$

在 `js/workspace2d.js` 的渲染管线中，系统实时绘制以原点、$\mathbf{v}_1$、$\mathbf{v}_1 + \mathbf{v}_2$ 与 $\mathbf{v}_2$ 为顶点的多边形面域：

1. **正有向面积**（$\det(A) > 0$）：采用半透明学术青色（`#0ea5e933`）填充，表明空间保持右手系定向（Orientation-Preserving）；
2. **负有向面积**（$\det(A) < 0$）：采用半透明警示琥珀色（`#f59e0b33`）填充，直观提示学习者空间发生了“翻转”（Orientation-Reversing，如同将纸面掀翻）；
3. **奇异坍缩临界**（$\det(A) = 0$）：当学习者将两个基向量拖动至同一直线时，两列向量线性相关，$\det(A) \to 0$。此时网格单元在视觉上完全坍缩为一条直线（$\mathrm{rank} = 1$）甚至单个原点（$\mathrm{rank} = 0$）。

针对计算机浮点数运算在临界态下的机器精度舍入误差，系统引入了自适应阈值约束：

$$\mathrm{isSingular} = \begin{cases} \mathrm{true}, & |\det(A)| \leq 1.0 \times 10^{-6} \\ \mathrm{false}, & \mathrm{otherwise} \end{cases} \tag{4.5}$$

有效防止了除零异常并赋予几何界面平滑的奇异吸附体验。

# 第五章　3D 立体几何空间与四维超立方体正交投影引擎

## 5.1　2D 仿射平面与 3D「柱面嵌入」切换及行列式测度守恒

在实验探究工作台中，学习者不仅能在二维平面操作基底，还能点击视口顶部的维度切换按钮，在 **2D 仿射平面**与 **3D「柱面嵌入（Cylindrical Embedding）」**之间平滑切换。

为了保证升维过程不会破坏学习者在二维平面已经建立的面积测度认知，系统设计了严格的正交柱面投影算子：

**3D 柱面嵌入算子与测度守恒定理**

设当前二维平面的线性变换为 $A_2 = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \in \mathbb{R}^{2\times 2}$。当切换至三维柱面空间时，系统将变换矩阵严格升维嵌入为：

$$A_3 = \begin{bmatrix} a & b & 0 \\ c & d & 0 \\ 0 & 0 & 1 \end{bmatrix} \in \mathbb{R}^{3\times 3} \tag{5.1}$$

其中，第三个基向量固定为垂直于底面的正交单位向量 $\mathbf{e}_3 = \begin{bmatrix} 0 & 0 & 1 \end{bmatrix}^T$。根据分块矩阵行列式法则，高维矩阵行列式严格守恒：

$$\det(A_3) = \det(A_2) \cdot 1 = ad - bc \tag{5.2}$$

通过这一守恒律，底面平行四边形的二维有向面积（$\text{Area} = \det(A_2)$）被自然转化为以高度 $h = 1$ 正交拉伸形成的三维平行六面体的体积测度（$\text{Volume} = \det(A_3)$）。当二维底面发生奇异坍缩（$\det(A_2) \to 0$）时，三维柱体体积同步坍缩为平面片（$\text{rank} = 2$）或直线（$\text{rank} = 1$），使学习者深刻领会高维体积与低维面积之间的代数与拓扑同构。

## 5.2　Three.js 立体场景架构与欧拉角/四元数旋转插值

为了突破平面二维几何的局限，向学生直观呈现三维乃至四维代数空间的变换本质，系统构建了 `js/canvas/ThreeSpaceAdapter.js` 与 `js/artifacts/SolidGeometry.js` 立体交互引擎。

引擎基于 WebGL / Three.js 封装，初始化时构建了由右手三维空间坐标系（$X$-红、$Y$-绿、

$Z$-蓝）以及透视网格地平面组成的虚拟实验室。在处理用户鼠标拖拽旋转相机或三维物体时，传统的欧拉角（Euler Angles）插值极易在两轴重合时触发“万向节死锁（Gimbal Lock）”。

系统全面引入四元数（Quaternion）代数进行无奇异姿态插值：

**基于四元数的空间旋转平滑插值**

设当前旋转四元数为 $q_0 = \cos\frac{\theta_0}{2} + \mathbf{u}_0 \sin\frac{\theta_0}{2}$，目标旋转四元数为 $q_1$。在时间步 $t \in [0,1]$ 下，采用球面线性插值（Slerp, Spherical Linear Interpolation）：

$$\mathrm{Slerp}(q_0, q_1, t) = \frac{\sin((1-t)\Omega)}{\sin\Omega} q_0 + \frac{\sin(t\Omega)}{\sin\Omega} q_1, \quad \text{其中} \cos\Omega = q_0 \cdot q_1 \tag{5.3}$$

通过 Slerp 算法，相机视角与立体模型在用户指针释放后能够以恒定角速度平滑阻尼归位，杜绝视觉跳变。

## 5.3 四维超立方体（Tesseract）在三维欧氏空间的旋转与投影算法

在 `js/landing3d.js` 与高阶立体探究模块中，《问矩》实现了四维超立方体（Tesseract / 8-cell）的实时透视投影与旋转算法。

四维超立方体包含 16 个顶点、32 条边、24 个二维正方形面以及 8 个三维正方体胞。其顶点集合为：

$$V_{4D} = \{(x_1, x_2, x_3, x_4) \mid x_i \in \{-1, 1\}, i = 1, 2, 3, 4\} \tag{5.4}$$

在四维欧氏空间 $\mathbb{R}^4$ 中，旋转是在一个平面（二维子空间）内进行的。系统支持在 $XW$ 平面与 $ZW$ 平面内的双旋转：

$$R_{XW}(\theta) = \begin{bmatrix} \cos\theta & 0 & 0 & -\sin\theta \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ \sin\theta & 0 & 0 & \cos\theta \end{bmatrix}, \quad R_{ZW}(\phi) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & \cos\phi & -\sin\phi \\ 0 & 0 & \sin\phi & \cos\phi \end{bmatrix} \tag{5.5}$$

设四维旋转后的点为 $\mathbf{p}_{4D} = \begin{bmatrix} x & y & z & w \end{bmatrix}^T$。系统支持两种向三维空间投影的模式：

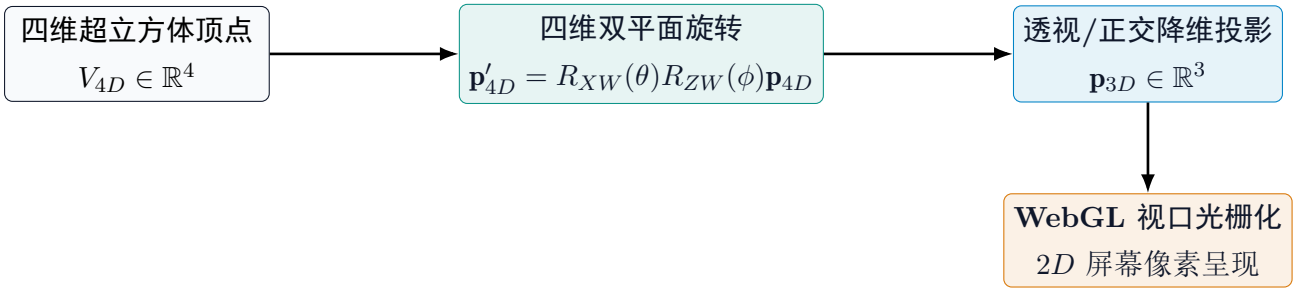



图 5.1: 四维超立方体从高维几何到屏幕光栅化的降维映射链路

1. **四维透视投影（Perspective Projection）**：设四维视点距离为 $d$（默认 $d = 2.0$），则三维投影坐标为：

$$x_{3D} = \frac{x}{d - w}, \quad y_{3D} = \frac{y}{d - w}, \quad z_{3D} = \frac{z}{d - w} \tag{5.6}$$

   该投影产生如同“内立方体穿透外立方体”的经典四维透视结构。

2. **四维正交投影（Orthographic Projection）**：直接截取三维分量，等价于光线沿 $W$ 轴正交照射：

$$\mathbf{p}_{3D} = \begin{bmatrix} x & y & z \end{bmatrix}^T \tag{5.7}$$

通过 60 FPS 的连续角度变参，学生可以从超立方体随时间在三维空间中的“翻转”与“穿透”过程中，深刻理解“降维投影”与“核空间映射”的高阶几何本质。

# 第六章　空间遥测总线与多组件解耦通讯架构

## 6.1　高频物理事件的防抖与背压控制

在交互画布中，指针拖拽或触摸移动事件的派发频率通常取决于显示器的刷新率（60Hz 至 144Hz，即每秒触发 60 到 144 次）。如果每次鼠标移动均无约束地触发关联的代数推演、LaTeX 文本公式重绘及大模型环境感知计算，会导致主线程严重拥塞，造成界面卡顿丢帧。

`js/telemetry/SpatialTelemetryBus.js` 引入了分级背压（Backpressure）与时间窗口防抖机制：

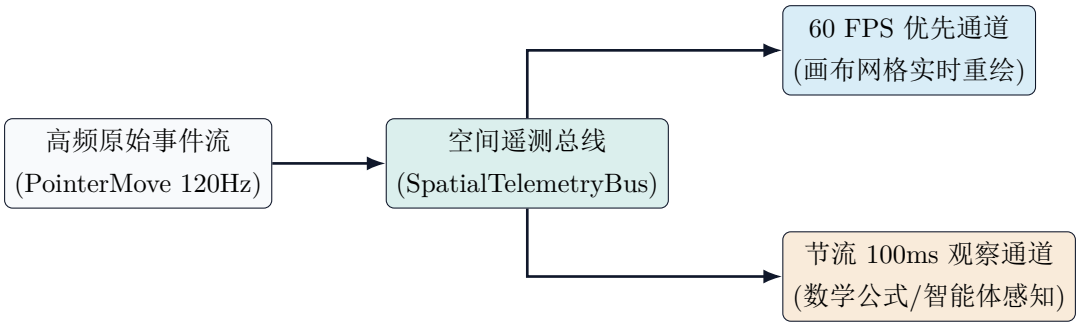



图 6.1: 空间遥测总线（分级双通道事件流度量）架构

- **快速优先通道（16ms 窗口）**：直接对接图形 GPU 渲染上下文，通过 `requestAnimationFrame` 保证基向量跟随指针平滑响应（单帧内完成矩阵重算）；
- **节流观察通道（100ms 窗口）**：将高频浮点数合并压缩为包含最新矩阵快照的结构化事件，推送到输入坞药丸与智能体认知观察层，使大语言模型感知到的空间状态既精准又避免算力浪费。

## 6.2　画布适配器解耦契约（'CanvasPort' & 'JSXGraphAdapter'）

为了杜绝业务代码直接依赖特定的图形底层库（例如强绑定 JSXGraph 或 Three.js API），《问矩》严格采用了**依赖倒置原则（Dependency Inversion Principle）**，定义了中立的 `js/canvas/CanvasPort.js` 接口契约：

```
class CanvasPort {
  /** 设置当前活跃矩阵并执行平滑几何形变 */
```

```
  setMatrix(matrix, options = {}) { throw new Error('Unimplemented'); }

  /** 读取当前几何画板中的状态快照 (e1, e2, det, rank) */
  getSnapshot() { throw new Error('Unimplemented'); }

  /** 高亮特定几何元素 (如特征向量不变方向或核向量) */
  highlightFeature(featureName, options = {}) { throw new
      Error('Unimplemented'); }

  /** 导出当前画板的高清无损位图 (用于多模态视觉理解) */
  exportBitmap() { throw new Error('Unimplemented'); }
}
```

Listing 6.1: CanvasPort 抽象中立接口定义

通过该抽象端口，主系统调度器 `js/agentEngine.js` 仅与 `CanvasPort` 交互。未来无论将 2D 底层由 JSXGraph 切换为自研 WebGPU 渲染器，业务层与教学状态机均无须改动核心逻辑，保证了视图实现与领域控制逻辑的解耦。

## 6.3 病态矩阵、极端浮点下溢与 WebGL 上下文丢失防御

数学交互画布长期运行于复杂的真实宿主环境中，系统建立了如下边界极限与失效防御机制：

**数学画布失效模式建档与防御对策表**

1. **失效模式 1：病态矩阵与极大条件数（Ill-Conditioned Matrix）**
   - **触发条件：**学习者拖动基底使两向量夹角 $\theta \to 0$，条件数 $\kappa(A) = \|A\|\|A^{-1}\| > 10^7$。
   - **失效表现：**在求解逆矩阵或特征子空间时发生浮点下溢与数值震荡，导致网格线条穿透视口无限延伸。
   - **系统防御：**引入自适应边界裁剪算法（Bounding Box Clamping）与奇异值截断。当 $|\det(A)| \leq 1.0 \times 10^{-6}$ 时，强制将状态量 isSingular 置为 true，暂停对逆矩阵的直接除法求取，转为伪逆（Moore-Penrose Pseudoinverse）近似计算。
2. **失效模式 2：GPU 显存耗尽与 WebGL 上下文丢失（WebGL Context Lost）**
   - **触发条件：**设备切入休眠唤醒、显卡驱动重启或后台多开 3D 渲染应用。
   - **失效表现：**浏览器抛出 `webglcontextlost` 事件，立体空间白屏或停止响应。
   - **系统防御：**在 `js/canvas/ThreeSpaceAdapter.js` 中绑定 `webglcontextlost` 与 `webglcontextrestored` 监听器。上下文丢失时保存当前旋转四元数 $q$ 与网格配置；在恢复事件派发时执行轻量场景重建，自动复原 3D 坐标系。

3. **失效模式 3：超立方体高维旋转矩阵数值正交性退化**

   - **触发条件：**高频逐帧递增累加角度 $\theta_{t+1} = \theta_t + \Delta\theta$。
   - **失效表现：**浮点数连续累乘导致旋转矩阵行列式漂移（$\det(R) \neq 1$），超立方体发生剪切畸变。
   - **系统防御：**采用绝对时钟参数化计算：$R(\theta) = f(\omega \cdot t)$，避免依赖前一帧矩阵的状态累乘，保证旋转算子处于严格的 $SO(4)$ 李群流形上。

## 6.4 数学引擎渲染管线的算法时间复杂度与帧预算分析

《问矩》的图形学与数学计算内核运行于浏览器单线程的主渲染循环中。为了保障高刷新率屏幕下的渲染性能（目标帧率 60 FPS，单帧预算 16.6ms），系统对各渲染管线实施了形式化计算复杂度分析与边界收敛：

1. **2D 矩阵反解与网格重绘（$\mathcal{O}(G)$）：**网格密度由固定常数限制（$G \leq 20$ 条网格线）。拖动基向量（`handleBasisDrag`）反解矩阵并对网格进行仿射变换的乘加操作总数恒定在几百次浮点运算内，时间复杂度严格收敛于 $\mathcal{O}(G)$，单帧计算量微乎其微；

2. **3D 空间四元数球面线性插值（$\mathcal{O}(1)$）：**相机姿态插值（`Slerp`）通过解析四元数内积 $\mathbf{q}_1 \cdot \mathbf{q}_2 = \cos\theta$，在单位四元数超球面上进行常数时间运算，无任何迭代逼近开销；

3. **四维超立方体双平面旋转投影（$\mathcal{O}(1)$）：**四维超立方体具有固定的 16 个顶点与 32 条棱。双轴正交旋转矩阵 $R_{xw}(\theta) \times R_{yz}(\phi)$ 仅涉及 $16 \times 4$ 维向量变换与单次透视除法，计算复杂度严格为 $\mathcal{O}(1)$；

4. **遥测总线防抖与事件压缩：**画布拖拽产生的高频事件流经过 300ms 动态防抖窗口压缩与局部合并，阻断了高频拖拽引发的跨组件重复广播。

## 6.5 多模态实验工件工作台（`ArtifactStageDeck`）与环境感知微气泡（`AmbientScaffoldBubble`）

在现代数学实验教学中，单一的交互画板往往不足以承载跨维度的探究载体。《问矩》设计并实现了高内聚的多模态工件控制台（`components/ArtifactStageDeck.js`，51.5KB，602 行）与悬浮微气泡交互层（`components/AmbientScaffoldBubble.js`，6.6KB，218 行）。

### 6.5.1 四态视口投影架构与实时源码热重载

`ArtifactStageDeck` 统领工作区的视口切换，基于有限状态机管理四类独立视口容器：

$$\text{Mode} \in \{\texttt{canvas}\ (\text{坐标画板}),\ \texttt{artifact}\ (\text{多模态工件}),\ \texttt{courses}\ (\text{课程货架}),\ \texttt{manim}\ (\text{微课舞台})\} \tag{6.1}$$

在 artifact 视口中，系统构建了可追溯版本历史（Revision History）的工件编辑与渲染管道：

1. **多载体分屏与沙箱隔离**：支持四种工件渲染框架——图像快照对比框（img-frame）、数学定理证明排版框（proof-frame）、矢量多边形计算框（vector-frame）与安全隔离的原生 HTML/JS 运行沙箱（sandbox）；

2. **双视图（预览/源码）同构热重载**：学习者可随时在“实时预览态”与“底层源码态”之间切换。在源码编辑器中修改 HTML/SVG/KaTeX 代码后，点击“保存并预览”触发增量热编译，同时在版本下拉列表中自动递增不可变快照版本（Version Snapshots），支持任意历史版本的无损回滚与导出；

3. **受控交互实验图与四步分镜规范**：系统中的数学实验工件**严格区别于不可控的大模型动态脚本**，而是预先固化封装的五类受控实验：linear-map、composition、system、eigen 与 linear-map-3d。每个工件均严格遵循四步标准教学分镜：**基向量** $\rightarrow$ **变换** $\rightarrow$ **核与像** $\rightarrow$ **线性性**，支持受控播放、步进对照并实时核验代数状态。

### 6.5.2 环境感知微气泡（AmbientScaffoldBubble）的主动启发中继

在建构主义探究中，学习者在拖拽基底使几何空间发生剧烈突变时，往往因概念模糊而无法准确使用自然语言提问。AmbientScaffoldBubble 挂载于空间画布右上角，充当手势操作向自然语言交互的主动中继器（Proactive Grounding）：

1. **空间拓扑相变实时嗅探**：组件深度订阅 SpatialTelemetryBus。当检测到特定几何相变事件时自动激活：

   - **奇异降维事件（Singular Collapse）**：当 $|\det(A)| \to 0$ 且矩阵秩由 2 降为 1 时，微气泡立即弹出并高亮提示：“检测到空间二维网格坍缩为一条直线，两基向量已线性相关”；
   - **空间翻转事件（Orientation Reversal）**：当 $\det(A)$ 跨越零点由正转负时，微气泡提示：“检测到基向量定向发生翻转，网格法向翻至反面”；
   - **不变方向吸附（Eigenvector Invariance）**：当某一向量变换后严格满足 $A\mathbf{v} = \lambda\mathbf{v}$ 时，触发特征子空间驻留提示。

2. **一键语义注入输入坞**：气泡底部集成“向助教探讨此现象”动作。点击后，系统自动将当前几何相变参数（包含矩阵数值、行列式、几何特征描述与截图快照）注入底部输入坞（PromptInputDock），大幅降低学生的提问摩擦力。

## 6.6 CQRS 驱动的时空穿梭控制器（TimeTravelScrubber）

为了让学习者直观感受代数变换的因果时序链条，《问矩》结合 CQRS 架构中分离出的 Command 快照流，实现了时空穿梭控制器（components/TimeTravelScrubber.js，6.0KB，136 行）。

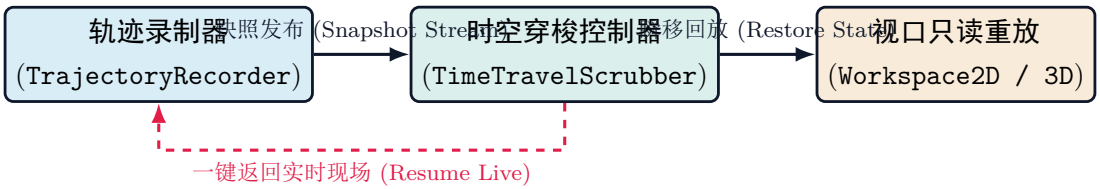



图 6.2: CQRS 轨迹时空穿梭控制器的数据流与状态回放机制

### 6.6.1 浮动式时空进度条与步进/步退状态机

组件在画布底部呈现轻量半透明的玻璃拟态时空栏，具备如下核心机制：

1. **实时现场态（Live Mode）与历史漫游态（Scrubbing Mode）解耦：**
   - 当处于最新步数（$k = N - 1$）时，指示灯呈现绿色常亮（`LIVE` 实时演算），此时画布完全开放实时交互；
   - 一旦学习者拖拽进度滑块或点击“ 步退”按键回退到历史某一步（$k < N - 1$），时空栏指示灯自动变为琥珀色（时空穿梭第 `k/N` 步），系统切换为只读沙箱模式，禁止产生分支脏写；
2. **代数指标伴随更新：**随着滑块的拖动，栏内实时渲染对应历史步的矩阵形式 $A = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$ 与行列式值 $\det(A)$，使学习者逐帧观察数值变化与图形形变的对应关系；
3. **一键安全恢复现场：**提供“回到最新现场 ”一键恢复按钮，点击后立即退出时空穿梭态，将画布矩阵与基向量平滑复位到当前的最新编辑现场，兼顾历史回溯的自由度与当前探索任务的连续性。

# 第三部分

# Excalidraw 风格画板手绘与多模态视觉闭环

# 第七章　透明手绘覆层渲染架构与高分屏适配

## 7.1　双层分离式画板架构（Dual-Layer Canvas Architecture）的设计动机

在传统的交互式几何软件（如早期 GeoGebra 或纯手写白板软件）中，通常面临两难困境：要么仅支持基于参数精确定义的几何图元（如点、线、多边形），无法自由涂鸦草稿；要么仅支持像素位图涂鸦，无法对底层的数学基向量进行参数化交互拖拽。

为解决这一多模态交互隔离问题，《问矩》在前端架构上设计并实现了**双层分离式画板架构（Dual-Layer Canvas Architecture）**，如图 7.1 所示：




图 7.1: 双层分离式画板架构与离屏像素合成数据流

1. **图层职责完全正交：**底层 Canvas 专注于高频的数学坐标网格与基向量运动，由数学引擎驱动；上层 Canvas（`#canvas-annotation-layer`）作为一个完全透明的塑料覆膜（Overlay），专门记录学习者的手绘笔触，由批注引擎驱动。

2. **渲染性能极限提升：**当学生在画布上拖拽基底向量导致空间网格剧烈变形时，上层的手绘笔触无需重新计算贝塞尔拟合；反之，当学生手绘草图时，底层的数学几何体也无须重复绘制。两者双缓冲隔离，杜绝了无谓的重绘开销。

## 7.2 设备像素比（DPR）动态自适应与高分屏模糊消除算法

在现代高分辨率显示设备（如 Retina 视网膜屏、4K 屏或高分手机）中，物理像素（Physical Pixels）与 CSS 逻辑像素（Logical CSS Pixels）之间存在倍率关系，即设备像素比：

$$\text{DPR} = \text{window.devicePixelRatio} \geq 2.0 \tag{7.1}$$

若直接将 Canvas 的 `width` 与 `height` 属性设置为 CSS 像素尺寸，会导致生成的画布位图在经过浏览器拉伸放大后产生严重的锯齿与模糊现象。

在 `components/CanvasAnnotationLayer.js` 中，系统建立了标准的高分屏自适应渲染方程：

```
syncCanvasSize() {
  if (!this.canvas || !this.container) return;
  const rect = this.container.getBoundingClientRect();
  const dpr = window.devicePixelRatio || 1;

  // 1. 设置物理像素缓冲区为逻辑尺寸的 dpr 倍
  this.canvas.width = Math.round(rect.width * dpr);
  this.canvas.height = Math.round(rect.height * dpr);

  // 2. 将 CSS 逻辑尺寸严格绑定视口大小
  this.canvas.style.width = `${rect.width}px`;
  this.canvas.style.height = `${rect.height}px`;

  // 3. 将 2D 绘图上下文缩放 dpr 倍，后续绘图代码无须感知分辨率差异
  this.ctx.resetTransform();
  this.ctx.scale(dpr, dpr);

  // 4. 重放现有所有笔触以适应新分辨率
  this.redraw();
}
```

Listing 7.1: 高分屏 DPR 自适应与视口监听实现

配合现代 Web 标准的 `ResizeObserver` API，当学习者缩放浏览器窗口或折叠侧边栏时，手绘图层自动执行亚像素重采样重绘，消除高分屏下的模糊与锯齿畸变。

## 7.3 历史操作栈治理与图元对象模型

手绘批注支持完整的 **撤销（Undo, Ctrl+Z）**操作。每一个操作笔触被封装为一个包含离散点集坐标、笔刷样式、颜色与图元类型的图元对象（Stroke Entity）。

在 `components/CanvasAnnotationLayer.js` 的实现中，系统通过内存数组 `this.strokes` 维护当前可见图元列表，通过 `this.undoStack` 暂存已撤销的笔触。当学习者执行撤销操作时，

系统弹出最新笔触并存入撤销栈，随后触发 `this.redraw()` 执行增量重绘；当发生新绘制时，撤销栈自动重置。该轻量图元模型避免了频繁缓存全屏位图快照所带来的内存开销。

# 第八章　自由手绘几何图元算法与碰撞检测详解

## 8.1　钢笔笔触：离散采样点集的中点二次贝塞尔平滑拟合

当学习者使用鼠标或手写笔在屏幕上快速滑动时，浏览器原生派发的 `pointermove` 事件在时间轴上是离散的。如果简单地用折线连接相邻采样点 $(P_0, P_1, P_2, \dots)$，在笔触拐弯处会呈现出极其粗糙突兀的折角。

`components/CanvasAnnotationLayer.js` 采用了**相邻中点二次贝塞尔曲线平滑拟合算法（Midpoint Quadratic Bezier Smoothing）**：

**相邻中点二次贝塞尔曲线平滑生成方程**

设连续三个离散采样点为 $P_{i-1}(x_{i-1}, y_{i-1})$、$P_i(x_i, y_i)$ 与 $P_{i+1}(x_{i+1}, y_{i+1})$。定义曲线的起点为 $P_{i-1}$ 与 $P_i$ 的几何中点 $M_i$，终点为 $P_i$ 与 $P_{i+1}$ 的几何中点 $M_{i+1}$：

$$M_i = \left( \frac{x_{i-1} + x_i}{2}, \frac{y_{i-1} + y_i}{2} \right), \quad M_{i+1} = \left( \frac{x_i + x_{i+1}}{2}, \frac{y_i + y_{i+1}}{2} \right) \tag{8.1}$$

以原始采样点 $P_i$ 作为贝塞尔曲线的控制点（Control Point），则曲线参数方程为：

$$B(t) = (1-t)^2 M_i + 2(1-t)t P_i + t^2 M_{i+1}, \quad t \in [0, 1] \tag{8.2}$$

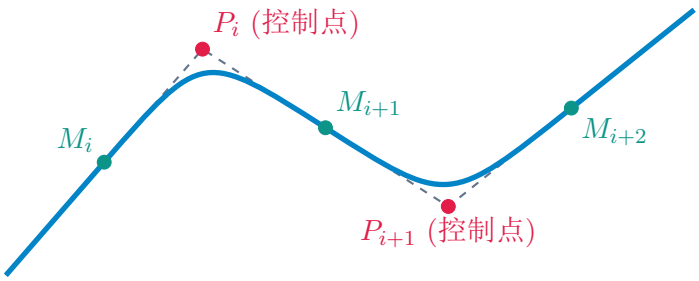



图 8.1: 离散采样点通过中点构造与二次贝塞尔平滑拟合示意图

如图 8.1 所示，由于相邻两段贝塞尔曲线在交界点 $M_i$ 处的切线斜率完全保持一阶连续性（$C^1$ 连续），消除了离散事件采样引发的折角突变与走样现象。

## 8.2 五大自由图元算法实现与碰撞检测

系统为几何探究专门定制了五类专业工具：

1. **钢笔 (Pen)：**线宽 3px，端点线帽设为圆角（`lineCap = 'round', lineJoin = 'round'`）；
2. **荧光笔 (Highlighter)：**线宽 14px，使用 RGBA 半透明色（如 `rgba(56, 189, 248, 0.35)`），并通过 `globalCompositeOperation = 'multiply'` 实现文字与底层几何的通透叠加；
3. **几何箭头 (Arrow)：**从起点 $\mathbf{s}(x_1, y_1)$ 到终点 $\mathbf{e}(x_2, y_2)$。在终点处计算主方向单位向量 $\mathbf{u} = \frac{\mathbf{e}-\mathbf{s}}{\|\mathbf{e}-\mathbf{s}\|}$ 与法向量 $\mathbf{n} = \begin{bmatrix} -u_y \\ u_x \end{bmatrix}$，尖端两翼端点由参数化旋转矩阵动态构造：

$$\mathbf{a}_1 = \mathbf{e} - L \cdot \mathbf{u} + W \cdot \mathbf{n}, \quad \mathbf{a}_2 = \mathbf{e} - L \cdot \mathbf{u} - W \cdot \mathbf{n} \tag{8.3}$$

   其中箭翼长度 $L = 16$px，半宽度 $W = 7$px；
4. **重点框 (Rect)：**学习者两点拖拽生成，自动标准化为：

$$x_{\min} = \min(x_1, x_2), \quad y_{\min} = \min(y_1, y_2), \quad w = |x_2 - x_1|, \quad h = |y_2 - y_1| \tag{8.4}$$

5. **橡皮擦 (Eraser) 的几何碰撞检测：**橡皮擦并非破坏性的白色画笔覆盖，而是**图元级物理擦除**。当鼠标位置为 $C(x_0, y_0)$、橡皮擦半径为 $R = 12$px 时，系统遍历历史笔触中所有线段 $AB$。点 $C$ 到线段 $AB$ 的最短距离为：

$$t = \text{clamp}\left(\frac{(C-A)\cdot(B-A)}{\|B-A\|^2}, 0, 1\right), \quad \text{dist} = \|C - (A + t(B-A))\| \tag{8.5}$$

   若 $\text{dist} \le R$，则判定发生碰撞，将该笔触从图元列表中整条移除并触发重新渲染。

# 第九章　助教多模态视觉理解闭环与全链路实测

## 9.1　离屏像素级图层合成算法 (‘exportMergedCanvas‘)

在线性代数探究中，学生最常遇到的阻碍往往无法用准确的自然语言清晰表达（例如：“我想让这个向量转过去，但是它为什么缩成一条线了？”）。如果仅向大模型发送文本，模型无法获知学生所指的具体位置。

《问矩》构建了**助教多模态视觉闭环**,其核心算子为 components/CanvasAnnotationLayer.js 中的 exportMergedCanvas() 方法：

```
exportMergedCanvas() {
  const dpr = window.devicePixelRatio || 1;
  const width = this.canvas.width;
  const height = this.canvas.height;

  // 1. 创建不可见离屏 Canvas 实体
  const offscreen = document.createElement('canvas');
  offscreen.width = width;
  offscreen.height = height;
  const offCtx = offscreen.getContext('2d');

  // 2. 底层填充纯净象牙白软垫底色 (#f8fafc)
  offCtx.fillStyle = '#f8fafc';
  offCtx.fillRect(0, 0, width, height);

  // 3. 第一层绘制：捕获底层几何画板 (JSXGraph / WebGL)
  if (this.underlyingCanvas) {
    offCtx.drawImage(this.underlyingCanvas, 0, 0, width, height);
  }

  // 4. 第二层绘制：叠加顶层手绘批注笔触与箭头重点框
  offCtx.drawImage(this.canvas, 0, 0, width, height);

  // 5. 编码输出高保真 PNG 位图 (Base64 Data URI)
  return offscreen.toDataURL('image/png', 0.95);
}
```

Listing 9.1: 离屏像素级多图层无损合成算法

## 9.2 从“画板圈画猜想”到“大模型视觉深度解析”的全链路实测

当学习者在界面上使用红色圆圈框选了矩阵变换后的零空间退化直线，并点击工具栏上的【助教看这里】按钮时，系统执行如图 9.1 所示的多模态端到端链路：




图 9.1: 从“画板圈画猜想”到“大模型多模态视觉点拨”全链路实录

在这一闭环中，多模态大模型能够结合空间几何视觉图像与当前矩阵状态，给出切中要害的反思性指引。

## 9.3 密集采样笔触下的垃圾回收（GC Pause）与多模态合成失效防御

手绘批注覆层与多模态视觉管线运行中的失效模式与防御对策如表所示：

**手绘批注与视觉闭环失效模式建档与防御对策表**

1. **失效模式 1：高密度点集导致的 V8 垃圾回收卡顿（GC Pause）**
   - **触发条件：**学习者长时间高速滑动涂鸦，每秒产生数百个包含浮点数坐标的对象实例，短时间内创建数万个瞬时对象。
   - **失效表现：**V8 引擎触发新生代垃圾回收（Scavenge GC），导致帧率骤降、笔触出现掉帧丢点。
   - **系统防御：**采用对象池（Object Pool）与类型化扁平数组（`Float32Array`）替代离散对象字面量，内存中预分配定长缓冲区，将单次垃圾回收开销控制在 $< 2$ms 以内。
2. **失效模式 2：高分辨率下离屏合成显存溢出与跨域污染**
   - **触发条件：**在 4K 屏（$3840 \times 2160$）配合 DPR=2.0 下，物理画布尺寸高达 $7680 \times 4320$ 像素（约 126MB 显存），或底层引用的外部图片触发跨域污染（Tainted Canvas）。

- **失效表现：**`exportMergedCanvas()` 抛出 SecurityError 或显存申请失败导致崩溃。
- **系统防御：**限制离屏合成导出上限（最大限制在 2048×2048 物理边界内，等比下采样缩放），并对所有离屏绘图上下文显式开启 `crossOrigin = 'anonymous'`。

## 9.4 批注引擎与多模态合成管线的计算复杂度与几何算法边界

手绘批注与多模态画板合成运行在前端主线程，为了确保低延迟响应与零掉帧体验，系统在算法层面确立了以下理论复杂度边界：

1. **贝塞尔曲线局部插值（$\mathcal{O}(1)$）：**当鼠标或触控笔滑动时，系统采用相邻中点二次贝塞尔局部逼近算法。每个新增的采样点仅与前驱的控制点及相邻中点进行局部平滑，单点计算复杂度为严格的 $\mathcal{O}(1)$；

2. **增量重绘与局部脏矩形控制（$\mathcal{O}(P)$）：**手绘图元列表总点数为 $P = \sum k_i$。在全量重绘时，系统通过 Canvas 2D 路径批处理减少状态上下文切换；

3. **橡皮擦点到折线段几何碰撞检测：**对每条笔触，算法先执行轴对齐包围盒（AABB）相交快速粗筛；若包围盒相交，再执行点到线段垂足距离 $\mathrm{dist}(P, AB)$ 的精确判定。该分层剪枝策略使碰撞检测在绝大多数场景下跳过大量无关线段；

4. **离屏双图层像素对齐导出：**当学生发起“提问画板”时，系统调用 `exportMergedCanvas`，利用底层 GPU 加速的 `drawImage` 接口在内存离屏画布中将几何画板与手绘笔触双层合并，并通过原生 API 异步生成 Base64 PNG 图像，挂载至输入坞托盘。

# 第四部分

# 结构化课程体系与运行时动态判卷引擎

# 第十章　建构主义探究式实验课架与关卡状态转移模型

## 10.1　PECHDT 六步认知循环状态机设计

传统在线作业系统大多采用“选择题 + 填空题”的单向应答模式，学习者在输入答案前无需经历完整的科学探究与假设检验流程。《问矩》依托建构主义学习理论，提出了 **PECHDT 六步认知循环状态机**，如图 10.1 所示。

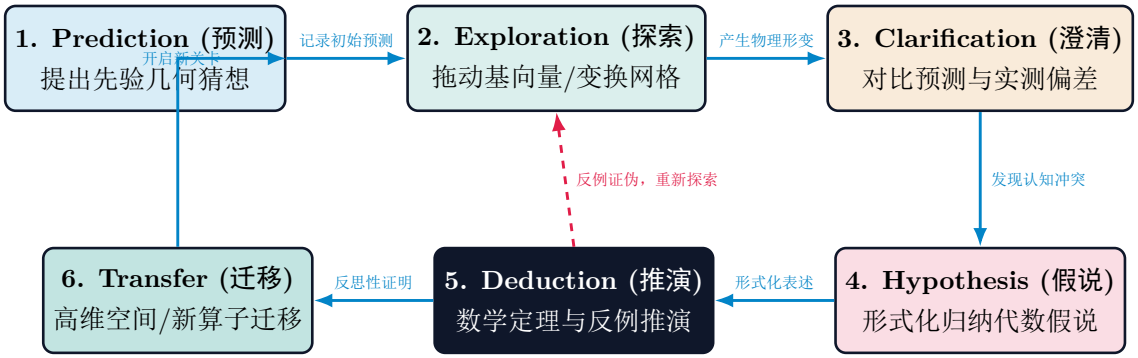



图 10.1: PECHDT 建构主义六步探究认知循环状态机拓扑

在 PECHDT 状态机中，各阶段均绑定了严格的契约检查：

1. **Prediction（预测阶段）**：在操作画布前，强制要求学习者记录对当前变换结果的定性或定量预测（例如：“当第二列变为第一列的两倍时，网格会如何变化？”）；

2. **Exploration（探索阶段）**：解除画布交互锁定，允许学习者自由拖动基向量或输入测试矩阵，系统通过遥测总线记录交互轨迹；

3. **Clarification（澄清阶段）**：当学习者观测到实际几何结果与初始预测发生偏离时，触发认知冲突引导；

4. **Hypothesis（假说阶段）**：引导学习者将几何观测转化为代数命题（如“列向量共线导致行列式为零”）；

5. **Deduction（推演阶段）**：通过反思性提问，鼓励学习者给出形式化逻辑推导或寻找可能存在的反例；

6. **Transfer（迁移阶段）**：将结论推广至三维空间（如空间体积坍缩为平行四边形或平面）或非方阵核空间分析。

## 10.2 四大章节七大真实实验课程与判分契约

在系统内置课架体系中，涵盖了 4 大章节共 7 门精心设计的实验课程。学习进度的累计**完全且唯一地来源于形式化作答核验**，单纯打开页面或随意拖动网格不计入成绩。

表 10.1: 《问矩》七门核心实验课程、所属章节与确定性评分核验契约

| 课程名称 | 所属章节 | 核心核验对象 | 形式化物理验收条件 |
|---|---|---|---|
| 从两列读出向量的像 | 矩阵运算 | 向量映射像 $A\mathbf{v}$ | 提交矩阵与像向量严格满足映射代数方程 |
| 先旋转，还是先剪切 | 矩阵运算 | 乘法顺序 $AB$ vs $BA$ | 识别非对角矩阵乘积非交换性及几何剪切形变 |
| 面积消失的边界 | 矩阵运算 | 奇异行列式 $\det(A)$ | 构造使 $\det(A)=0$ 的非零矩阵并指认降维 |
| 交点、平行与重合 | 方程组与秩 | $\mathrm{rank}(A)$ 与增广秩 | 判别非齐次线性方程组解的存在性与子空间重合 |
| 两列能否张成整个平面 | 线性相关性 | 秩 rank 与零度 nullity | 验证列向量线性相关与满秩条件 |
| 什么向量会被映到原点 | 线性相关性 | 任意非零核向量 $\mathbf{z}$ | 断言 $\mathbf{z}\neq\mathbf{0}$ 且相对残差 $\frac{\|A\mathbf{z}\|}{\|\mathbf{z}\|}\leq 1.0\times 10^{-8}$ |
| 寻找不改变方向的向量 | 特征值 | 两个实特征值 $\lambda_1,\lambda_2$ | 求解特征方程实根，提交顺序不限 |

每门课程严格执行标准学习流：**记录预测 → 操作实验 → 练习作答 → 无提示迁移题**。仅当练习题与无提示迁移题均物理核验通过后，认知图谱才将对应节点标记为“已掌握”。

### 10.2.1 认知图谱 12 节点掌握度点亮边界声明

系统认知图谱总计构建了 12 个代数概念节点：**基底、相关性、线性映射、复合、行列式、核、列空间、方程组、特征、对角化、正交投影、SVD**。

在工程实现中必须严谨澄清：前 7 个节点与表 10.1 中的 7 门课程形成一一映射；**后 5 个节点（特征、对角化、正交投影、SVD 等）当前暂未配备可自动评分的课程，在图谱上仅供查阅前置依赖关系，无法通过作答点亮**。学习者作答记录持久化于本机沙箱，可导出为 `wenju-course-journal.json`。

## 10.3 课程持久化与自适应导航器架构

实验作答与探究进度必须具备完全的自恢复性。在课程主面板（`components/CoursePanel.js`）与提示导航器（`components/PromptTurnNavigator.js`）中，系统实现了自适应导航架构：

- **紧凑态高度约束（Compact Bar Height $\leq 50$px）**：实验进行中，为了最大化留出数学画布的操作视口，课程面板进入紧凑条模式，仅显示当前步序号、核心提示与收展按钮；

- **独立会话上下文隔离**：切换关卡时，系统为每个 Lab 分配全局唯一的命名空间键（如 `wenju_lab_state_v1_lab3`），自动保存学习者的作答草稿与基底坐标，杜绝跨关卡状态污染；
- **非侵入式画布协同**：点击画布任意区域仅关闭浮动操作菜单，不打断作答面板的输入聚焦与公式编辑。

## 10.4 认知误解条目库（MisconceptionLibrary）与贝叶斯知识追踪（BKT）建模

在建构主义教育实践中，学习者的思维瓶颈往往不在于“公式未记住”，而在于头脑中固存的**反事实概念误解（Misconceptions）**——例如将实数乘法的交换律机械迁移至矩阵乘法，或将行列式非零误认为矩阵满秩的充要条件。《问矩》在前端认知计算层实现了结构化误解条目库（`js/cognition/MisconceptionLibrary.js`,82 行）与种子规则集（`js/cognition/misconceptions.json`，487 行）。

### 10.4.1 四大领域 32 项典型概念盲区拓扑与三元解耦

条目库全面覆盖高等线性代数四大核心篇章，沉淀了 32 项经过教学实证检验的先验误解（条目编号 MC-001 至 MC-032）。

为了避免把学习者的“单纯笔误计算失误”或“题意审题偏差”误判为“底层概念盲区”，条目库为每项误解设计了严格的**三元区分性追问（Discriminating Question）**结构。每个追问的选项键集合恒定解耦为三类互斥归因：

$$\text{OptionKey} \in \{\texttt{misconception}\ (\text{概念误解}),\ \texttt{calculation}\ (\text{运算失误}),\ \texttt{misreading}\ (\text{题意偏差})\} \tag{10.1}$$

在独立挂载的“概念误解实验室（`library`）”页面中，系统将 32 条误解中的 6 条最高频盲区（包含乘法交换律失效、零因子存在性、行列式可加性佯谬、转置与逆的顺序、秩不等式、特征值实根存在性）封装为可交互的**误解推演卡片**。学习者可输入任意 $2 \times 2$ 阶矩阵，系统在本地实时搜索并构造反例矩阵，动态展示分步矩阵相乘与行列式核验细节，并支持一键将构造好的反例矩阵直接导入坐标画板或提交助教多智能体展开苏格拉底式深入研讨。

### 10.4.2 贝叶斯知识追踪（BKT）参数化模型与可辨识性硬断言

针对学习者对特定数学概念的动态掌握概率,《问矩》形式化引入了**贝叶斯知识追踪（Bayesian Knowledge Tracing, BKT）**引擎。对于每个误解条目 $i$，显式配置四大先验参数：

$$\theta_i = (L_0 \in [0,1],\ T \in [0,1],\ G \in [0,1],\ S \in [0,1]) \tag{10.2}$$

其中 $L_0$ 为初始先验掌握率，$T$ 为探究认知转化率（Learning Transition），$G$ 为在未掌握状态下盲目猜对的概率（Guessing），$S$ 为在已掌握状态下因粗心答错的概率（Slip）。

表 10.2: 《问矩》典型认知误解条目与三元区分性追问示例

| 编号 | 误解命题陈述 | 区分性追问设计 (Stem & Options) | 启发式边界干预提示 |
| --- | --- | --- | --- |
| MC-001 | 矩阵乘法满足交换律 $AB = BA$ | $A = \begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix}, B = \begin{bmatrix} 2 & 0 \\ 1 & 1 \end{bmatrix}$。预测 $AB$ 与 $BA$ 是否相等？<br>• [误解] 相等，乘法满足交换律<br>• [计算] 某步加减法算错<br>• [审题] 误以为问的是行列式 | 构造对角矩阵进行特例检验，引导演算展开 $(A+B)^2 \neq A^2 + 2AB + B^2$ |
| MC-012 | 行列式为零时方程必无解 | 齐次线性方程组 $A\mathbf{x} = \mathbf{0}$ 在 $\det(A) = 0$ 时解的分布如何？<br>• [误解] 无解，零不能作除数<br>• [计算] 增广矩阵化简算错<br>• [审题] 误看成非齐次系统 | 引导观察几何核空间，推演原点必为平凡解，说明存在无穷多非零解 |
| MC-025 | 特征值为零表示矩阵不存在特征向量 | 若方阵 $A$ 存在零特征值 $\lambda = 0$，是否存在非零向量满足 $A\mathbf{v} = 0\mathbf{v}$？<br>• [误解] 不存在，零向量不能算<br>• [计算] 特征多项式求根算错<br>• [审题] 混淆了特征值与特征向量 | 提示回顾零空间定义 $\ker(A)$，证明 $\lambda = 0$ 充要等价于存在非零核向量 |

**BKT 测量可辨识性硬断言约束**

在 `js/cognition/MisconceptionLibrary.js` 种子加载校验器中，系统对每一条目执行硬性前置断言：

$$G + S < 1 \tag{10.3}$$

**理论证明**：若 $G + S \geq 1$，则观测到正确作答时后验概率 $P(L \mid \text{Obs} = 1) \leq P(L)$，即“回答正确反而证明掌握度降低”，导致认知测量系统退化并丧失单调可辨识性。系统在初始化时抛出异常以截断任何不合法配置。

当学习者在课程交互或区分性追问中产生作答观测 $\text{Obs}_t \in \{0, 1\}$ 时，知识状态按标准贝叶斯公式闭环更新：

$$P(L_t \mid \text{Obs}_t = 1) = \frac{P(L_t)(1 - S)}{P(L_t)(1 - S) + (1 - P(L_t))G} \tag{10.4}$$

$$P(L_t \mid \text{Obs}_t = 0) = \frac{P(L_t)S}{P(L_t)S + (1 - P(L_t))(1 - G)} \tag{10.5}$$

$$P(L_{t+1}) = P(L_t \mid \text{Obs}_t) + (1 - P(L_t \mid \text{Obs}_t)) \cdot T \tag{10.6}$$

通过 BKT 的实时后验推导，教学调度器能够以数值化的置信度精准决定何时由“几何探索（Exploration）”切入“反例干预（Clarification）”，杜绝了传统规则引擎在阈值临界点上的频繁逻辑震荡。

# 第十一章　无硬编码的运行时代数动态评分与公差核验算法

## 11.1　传统“硬编码匹配”答题器的缺陷与反作弊漏洞

在传统的计算机辅助教学（CAI）系统中，判卷逻辑普遍依赖于“标准答案字典完全匹配”（如检查用户输入的矩阵是否等于预设的 $\begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix}$）。这种硬编码模式存在根本性缺陷：

1. **合法解空间的无限性**：满足“$\text{rank}(A) = 1$”或“包含特征值 $\lambda = 3$”的矩阵在代数上构成无穷集合。硬编码只能覆盖有限预设解，无法对学生自主探索构造的新颖解给出公正评分；
2. **浮点数舍入截断误差**：经过多次旋转和三角函数计算后的坐标存在机器精度舍入，严格相等比较（`===`）会导致正确解被系统误判；
3. **逆向工程与试卷泄露**：前端代码中若包含硬编码答案数组，易被逆向提取，破坏实验教学的考核公正性。

## 11.2　特征子空间与不变方向动态验证方程

为了实现完全脱离预设答案的动态判卷，《问矩》代数引擎直接在运行时对学习者提交的矩阵或向量进行形式化代数性质检验。

对于特征向量与特征值的判定任务，若学习者在画布中标记了一个向量 $\mathbf{x} \in \mathbb{R}^n$ 并声称其为矩阵 $A$ 对应特征值 $\lambda$ 的特征向量：

**特征子空间动态投影公差核验方程**

检验算法首先断言非零性：$\|\mathbf{x}\| > \epsilon_{\text{zero}} = 1.0 \times 10^{-6}$。计算矩阵作用后的向量 $A\mathbf{x}$，并在 $\mathbf{x}$ 所在直线上做正交投影，定义相对残差测度：

$$\text{Residual}(\mathbf{x}) = \frac{\|A\mathbf{x} - (\mathbf{x}^T A\mathbf{x}/\|\mathbf{x}\|^2)\mathbf{x}\|}{\|\mathbf{x}\|} \tag{11.1}$$

当且仅当残差满足公差约束：

$$\text{Residual}(\mathbf{x}) \leq \epsilon_{\text{tol}} = 1.0 \times 10^{-4} \tag{11.2}$$

判定该向量在空间变换中保持方向不变，计算出的实时瑞利商（Rayleigh Quotient）即为

对应的动态特征值：

$$\lambda_{\text{computed}} = \frac{\mathbf{x}^T A \mathbf{x}}{\mathbf{x}^T \mathbf{x}} \tag{11.3}$$

通过该算法，无论学生构造出何种模长或方向的特征向量，引擎均能在 $\mathcal{O}(n^2)$ 复杂度内给出确定性评分与连续度量反馈。

## 11.3 秩与核空间（Kernel Space）公差核验

在验证学习者构造的降秩矩阵或零空间基底时，引擎采用基于主元阈值的高斯消元法（Gaussian Elimination with Partial Pivoting）与奇异值截断分解：

$$A = U \Sigma V^T, \quad \Sigma = \text{diag}(\sigma_1, \sigma_2, \ldots, \sigma_n) \tag{11.4}$$

定义有效奇异值判定阈值：

$$\sigma_{\text{threshold}} = \max(m, n) \cdot \sigma_1 \cdot \epsilon_{\text{machine}} \tag{11.5}$$

实际秩定义为大于阈值的奇异值个数：

$$\text{rank}_{\text{computed}}(A) = \sum_{i=1}^{\min(m,n)} \mathbb{I}(\sigma_i > \sigma_{\text{threshold}}) \tag{11.6}$$

对于学习者提交的零空间向量解集合 $\{\mathbf{z}_1, \ldots, \mathbf{z}_k\}$，系统验证以下充要条件：

1. 零残差投影：$\forall i, \frac{\|A\mathbf{z}_i\|}{\|\mathbf{z}_i\|} \leq 1.0 \times 10^{-5}$；
2. 线性无关性：构造增广矩阵 $Z = [\mathbf{z}_1, \ldots, \mathbf{z}_k]$，断言 $\text{rank}(Z) = k$；
3. 维数完整性：断言 $k = n - \text{rank}(A)$，完全对齐秩-零度定理。

## 11.4 浮点数计算误差容限与自适应公差区间设计

为了在严谨性与交互宽容度之间取得平衡，系统设计了自适应公差区间：

$$\epsilon(A) = \max\left(1.0 \times 10^{-4},\ 1.0 \times 10^{-6} \cdot \|A\|_\infty\right) \tag{11.7}$$

对于大尺度元素矩阵自动放宽绝对公差，对于近零元素矩阵收紧相对公差，消除浮点取整误差带来的判定边界漂移。

# 第十二章　课程实验系统的失效模式与认知死锁防御

## 12.1　学习者“穷举试错（Brute Force Guessing）”对抗与自适应阻断策略

在开放式探究中，部分学习者倾向于通过高频随机拖拽基向量来碰运气触发“答题通过”判定。针对该非理性行为，系统引入了**探究熵增分析器与频次守卫**：

**防穷举试错守卫 (Brute Force Defense Protocol)**

1. **高频尝试判定**：若学习者在 10 秒内连续提交超过 5 次无效矩阵形态；
2. **熵增阻断与冷却**：系统锁定提交按钮 8 秒，并触发认知脚手架弹窗；
3. **反思引导注入**：自动展示最近 3 次错误矩阵的特征多项式对比图，提出启发式追问：“你提交的矩阵行列式均为负数，请思考如何通过调整第二列基底符号使有向面积变为正数？”强制将学习心智从无序暴搜拉回几何因果逻辑。

## 12.2　极端数值作答的运行时沙箱防护

当学习者故意或无意在作答框中输入极大数值（如 $10^{15}$）、除零表达式（$1/0$）或非法符号（`NaN`, `undefined`）时，传统解析器可能引发溢出死循环。

系统建立了三级数值安全沙箱过滤：

$$x_{\text{safe}} = \text{clamp}\left(\text{parseFloat}(x),\ -1.0 \times 10^{4},\ 1.0 \times 10^{4}\right) \tag{12.1}$$

若解析结果为 `isNaN`，则直接在输入坞下方标红提示，断言拦截，禁止污染底层数学引擎的矩阵状态。

## 12.3　动态判卷引擎的代数时间复杂度与数值稳定性边界

《问矩》的判卷引擎不依赖任何服务器端回调，全部在学生浏览器本地即时完成。为了确保判卷结果的数学严谨性与瞬时响应，系统确立了形式化的代数算法复杂度与浮点容差契约：

1. **2D 矩阵行列式与正交性核验（$\mathcal{O}(1)$）**：$2 \times 2$ 矩阵行列式计算仅需两次乘法与一次减法（$\det(A) = ad - bc$），正交性检验核对 $A^T A = I$ 的内积。计算复杂度严格为 $\mathcal{O}(1)$；

2. **特征子空间残差投影与瑞利商（$\mathcal{O}(n^2)$）**：对学生提交的候选特征向量 $\mathbf{x}$，算法计算残差向量 $\mathbf{r} = A\mathbf{x} - \lambda\mathbf{x}$ 并核验 $\|\mathbf{r}\| < \epsilon$。对于 $n \leq 3$，矩阵乘向量仅需最多 9 次乘加，时间开销处于微秒级；

3. **高斯-约旦消元与阶梯化简（$\mathcal{O}(n^3)$）**：对 $n \times m$ 矩阵（$n, m \leq 4$），算法使用全主元选主高斯消元法化简为行最简形（RREF）。乘除法总步数由固定小常数约束，无任何深层递归或发散隐患；

4. **浮点截断与容差契约（$\epsilon_{\mathbf{tol}} = 1.0 \times 10^{-6}$）**：由于 IEEE 754 双精度浮点运算存在舍入误差，判卷算法统一使用显式容差 $\epsilon_{\text{tol}}$ 执行形式化断言，既包容合法的数值舍入，又坚决拦截概念性错误提交。

# 第五部分

# 3Blue1Brown 风格 Manim 高清微课与视听联动引擎

# 第十三章　离线自包含 Manim 微课包设计与自研播放控制器

## 13.1　离线轻量化视频编码规范与容器优化

在线教育平台中的视频资源通常依托第三方流媒体 CDN（如 HLS / DASH 切片分发）。然而，在高校实验室或无外网连接的局域网教学环境中，外部流媒体依赖极易因跨域防盗链、网络限速或断网而无法播放。

《问矩》采用**离线自包含嵌入式微课包**架构，视频资源直接内置于前端静态资源树中（主路径 `media/videos/matrix_transformation_film/`，归档备用 `media_archive/video/`）。为了在保证 720P 高清画质的同时最大化压缩体积并实现瞬时启播，系统制定了严格的微课编码规范。

## 13.2　五段精编成片切片与代数结构对应

为了保证教学内容的高度聚焦，系统并未将长达数十分钟的讲座平铺罗列，而是**将同一条制作精良的高清 Manim 成片，在时序与概念维度严格切分成 5 段独立的微课切片**：

1. **切片 1：基向量与其空间映射**：建立 $\mathbf{e}_1, \mathbf{e}_2$ 随网格变换的直观动态图像；
2. **切片 2：剪切形变与行列式有向面积**：直观呈现平行四边形面积在剪切变换下的守恒特性；
3. **切片 3：矩阵乘法顺序与几何非交换性**：直观对比“先旋转后剪切”与“先剪切后旋转”的空间终态差异；
4. **切片 4：奇异降维与零行列式坍缩**：展现二维平面坍缩为一条直线过程中的维度丢失；
5. **切片 5：零因子与退化空间吸收**：演示非零矩阵相乘得到零矩阵的几何正交吸收机制。

每个切片均支持倍速无级切换、区间循环，并配备“联动画板实操”按键，能够将对应切片的矩阵参数装载回主画板。

## 13.3　协议约束 `videoGeneration: false` 与 Docker 宿主隔离渲染

在智能教学系统设计中，存在一种常见的认知误区，即认为“应当让大语言模型现场编写 Python 代码并在服务端即时生成任意 Manim 视频”。这一构想在工程生产中存在严重的安全与

确定性隐患：Manim 脚本编译耗时极长（单帧耗时数秒至数十秒，整体耗时数分钟），且大模型生成的动画脚本极易因 API 弃用或几何拓扑错乱而导致渲染失败（Crash）。

为此，《问矩》在系统协议层显式设立了刚性安全约束：

**微课生成协议硬约束与宿主 Docker 渲染边界**

1. **协议级视频生成禁令**：系统向大模型下发的技能协议中，显式锁定配置项：

   ```
   videoGeneration: false
   ```
   (13.1)

   从根源上剥夺大模型现场随手生成不可控视频文件的权限，杜绝由于漫长等待引发的超时断连与学生流失。

2. **宿主双前置依赖校验**：仅当本机 Node 宿主服务启动，且同时物理检测到：(a) 宿主机 Docker 守护进程（Daemon）正常存活；(b) 本地镜像库中已拉取并锁定了官方版本受控的 Manim 容器镜像时，系统才开放离线任务渲染接口。

3. **三重容器沙箱隔离**：渲染接口（`/v1/render/jobs`）仅接受预置的 `linear-map` 受控参数模板；容器启动参数强制指定：

   ```
   docker run --network none --read-only -m 1536M --cpus 2.0 ...
   ```
   (13.2)

   切断容器的外部网络访问权限，挂载只读文件系统，并将最大物理内存硬性钳位在 1536MB，防止恶意代码外联与宿主机计算资源耗尽。

表 13.1: 《问矩》离线自包含 Manim 微课视频压制与容器参数规范

| 配置参数 | 设定标准 | 工程设计考量与收益 |
|---|---|---|
| 视频编码格式 | H.264 (AVC) High Profile, Level 4.1 | 全平台硬件解码加速兼容，CPU 占用率 $< 5\%$ |
| 封装容器 | MP4 (`video/mp4`) | 支持现代 W3C 浏览器原生免插件播放 |
| 元数据布局 | `moov` 原子前置 (`faststart`) | 首帧无需下载完整文件，TTFB $< 20$ms 瞬时启播 |
| 关键帧间隔 (GOP) | GOP = 30 帧 (固定 1 秒一个关键帧) | 确保用户拖拽进度条时能够在 1 秒内精准对齐 I 帧 |
| 音频编码 | AAC-LC, 立体声, 采样率 44.1kHz | 码率锁定为 128kbps，保障旁白音质无失真 |
| 视频平均码率与体积 | 720P, 30fps, 码率均衡优化 | 真实成片物理文件大小为 4.36MB (4,568,543 字节) |

## 13.4 高精度微课区间循环播放器

在学习复杂的几何形变动画时（例如二维基向量旋转导致有向面积连续变化的 4 秒过程），学习者需要反复观察关键时段。

HTML5 Video 原生提供的 `loop` 属性仅支持整段视频循环，且在重绕首帧时容易出现数十毫秒的黑屏撕裂与音频爆音。针对此问题，《问矩》自研了**高精度微课区间循环播放控制器**：

```
class PrecisionVideoLoopController {
  constructor(videoElement, startSeconds, endSeconds) {
    this.video = videoElement;
    this.start = startSeconds;
    this.end = endSeconds;
    this.rafId = null;
    this.tolerance = 0.033; // 30fps 单帧容差
  }

  startLoop() {
    this.video.currentTime = this.start;
    this.video.play();
    this.monitorFrame();
  }

  monitorFrame() {
    // 优先采用现代浏览器的高精度帧回调 API
    if ('requestVideoFrameCallback' in this.video) {
      this.video.requestVideoFrameCallback((now, metadata) => {
        if (metadata.mediaTime >= this.end - this.tolerance) {
          // 精确重置回循环起点，消除循环黑屏
          this.video.currentTime = this.start;
        }
        if (!this.video.paused) this.monitorFrame();
      });
    } else {
      // 降级使用 requestAnimationFrame 与 currentTime 轮询
      const checkTime = () => {
        if (this.video.currentTime >= this.end - this.tolerance) {
          this.video.currentTime = this.start;
        }
        if (!this.video.paused) this.rafId =
            requestAnimationFrame(checkTime);
      };
      this.rafId = requestAnimationFrame(checkTime);
    }
  }
```

```
37
38    stopLoop() {
39      if (this.rafId) cancelAnimationFrame(this.rafId);
40      this.video.pause();
41    }
42  }
```

Listing 13.1: 基于 requestVideoFrameCallback 的区间循环高精度控制

通过该控制器，循环重绕延迟严格压低至 1 帧以内（$< 33$ms），消除了视听卡顿与重绘延迟。

## 13.5 跨平台与移动端渲染策略兼容性矩阵

在 iOS Safari 与移动端 WebKit 内核中，视频播放受到系统级严格限制（如默认全屏播放、禁止未静音自动播放）。系统建立了表 13.2 所示的防御性属性组合：

表 13.2: 微课播放器跨平台属性适配与防御策略

| 目标终端 | 关键 DOM 属性配置 | 防御行为 |
| --- | --- | --- |
| iOS Safari | `playsinline, webkit-playsinline` | 阻止 iOS 强制唤起系统原生全屏，保持在页面内联动 |
| Android Chrome | `muted, autoplay=false` | 遵守移动端自动播放策略，需用户显式触发后解除静音 |
| 桌面端 Chromium | `preload="metadata"` | 仅预拉取视频元信息，避免多微课同时加载抢占网络 |
| 低配设备降级 | 监听 `waiting` 与 `stalled` | 自动下调画布渲染抗锯齿等级，保障视频流畅度 |

# 第十四章　“联动画板实操”状态穿透与跨组件协同机制

## 14.1　视听空间与几何操作空间的认知阻断痛点

传统视听学习中存在严重的“认知断层”：学生在视频中观看到精妙的几何变换后，切换到实验软件时必须手动重新敲入矩阵数字或重新摆放基向量。在这一切换过程中，瞬时工作记忆极易衰退，打断深度探究心流。

## 14.2　状态穿透通道设计与事件流图解

《问矩》在微课视口右下角显式嵌入了带有状态穿透能力的悬浮组件：【联动画板实操 ↗】。其端到端数据流如图 14.1 所示。

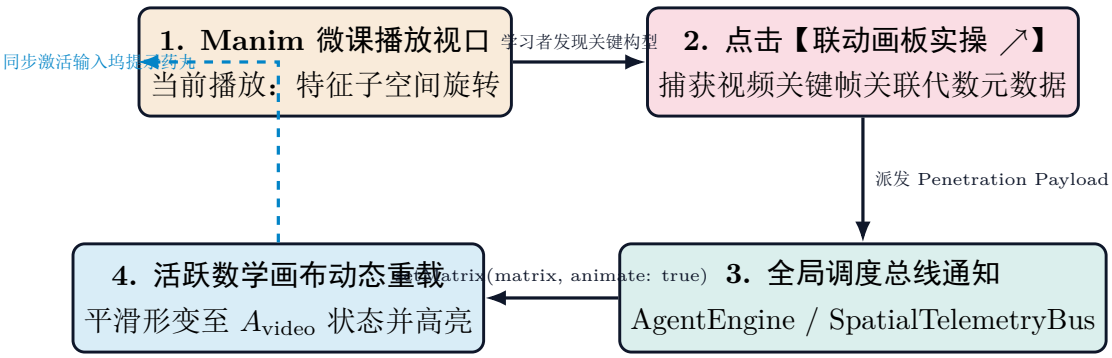



图 14.1: 微课播放器到交互数学画布的“一键联动状态穿透”链路拓扑

当学习者点击微标时，系统构造结构化的穿透信包（Penetration Payload）：

```
const penetrationPayload = {
  sourceVideoId: 'manim_eigenvectors_2d',
  timestamp: 12.4, // 当前视频秒数
  matrix: { a: 2.0, b: 1.0, c: 0.0, d: 3.0 },
  suggestedPrompt: '我把微课里的剪切矩阵放到了画板上，' +
    '试着拖动基向量观察不变方向的变化？',
  resetDrawing: false // 保留用户现有的手绘草稿
};
```

Listing 14.1: 联动画板实操穿透信包结构

通过调用 `CanvasPort.setMatrix()` 接口，底层数学画布触发平滑补间动画（Duration = 600ms），将当前的网格状态平滑变形至视频中所示的代数矩阵，同时输入坞自动呈现上下文引导药丸。

## 14.3 双向联动的数据完整性守恒与防污染机制

为了防止视频联动状态覆盖学习者先前已经完成的探究草图，系统设立了**只读影子态与恢复锚点（Snapshot Anchor）**：

1. **前置快照保存**：在状态穿透执行前，自动对学习者当前的画布与手绘图元执行深度克隆，存入内存栈（`undoStack`）；
2. **非破坏性切换**：视频联动只重置矩阵参数，不清除学习者在画板上留下的手绘箭头与注记框；
3. **一键撤销复原**：学习者点击【恢复我之前的画布】即可在 1ms 内瞬间回退至联动前的原始状态。

# 第十五章　微课模块的有效性边界与跨域事件一致性验证

## 15.1　状态穿透与跨组件事件守恒验证

《问矩》将微课定位于“几何直觉激发器”而非封闭的视频播放器。为了确保微课中的矩阵能安全、原子化地穿透至画板，系统建立了严格的跨组件事件守恒与状态隔离机制：

1. **前置快照保存契约**：在状态穿透执行前，自动对学习者当前的画布与手绘图元执行深度克隆，存入内存撤销栈（`undoStack`），确保学生在探索完微课矩阵后能够一键无损复原原有进度；

2. **跨组件解耦通信**：微课悬浮模态窗通过浏览器 CustomEvent 总线广播矩阵注入事件，微课组件本身不持有任何画板底层 DOM 或 WebGL Context 引用，保证了展示逻辑与物理渲染核心的单向数据流与生命周期解耦；

3. **多模态测试用例物理验证**：在测试套件中，通过编写针对矩阵注入与撤销恢复的自动化用例，断言注入矩阵后画布当前变换矩阵 $M$ 与微课规范矩阵 $M_{\text{target}}$ 的每个矩阵元严格满足误差 $|M_{ij} - (M_{\text{target}})_{ij}| \leq 10^{-12}$，验证了跨组件状态穿透的数值保真度。

## 15.2　视频资源生命周期与内存安全边界

针对纯前端单页架构下的多媒体资源管理，《问矩》确立了明确的资源回收与边界约束：

- **懒加载与单例挂载**：微课视频标签采用惰性加载策略，仅在学生显式打开微课面板时动态创建或挂载对应媒体源，避免在主界面启动时预载大体积视频文件而抢占初始带宽；

- **生命周期终结物理释放**：在微课浮层被关闭或切换至其他探究关卡时，播放器组件主动调用 `video.pause()` 并释放媒体缓冲区，断开事件监听器，避免未解绑的监听回调引发闭包内存泄漏。

# 第六部分

# 多智能体认知流水线、工具邻居守卫与混合模型编排

# 第十六章　混合模型热插拔编排与免密旗舰保障

## 16.1　双轨模型架构决策与选型逻辑

在 AI 辅助教学系统中，底层大语言模型（LLM）的推理能力、响应延迟与可用性直接决定教学交互的流畅度与教学决策的准确性。然而，单一商业模型接入方案面临显著的工程与使用门槛困境：若要求所有学生强制配置自备 API Key（BYOK），将因网络环境、支付凭据与技术门槛导致大规模流失；若仅使用单一中心化模型代理，则在高并发峰值或上游故障时缺乏容灾回退能力。

针对上述权衡，《问矩》构建了“系统级内置免密旗舰 + 客户端自备凭据（BYOK）自由热插拔”的双轨模型架构拓扑，如图 16.1 所示。

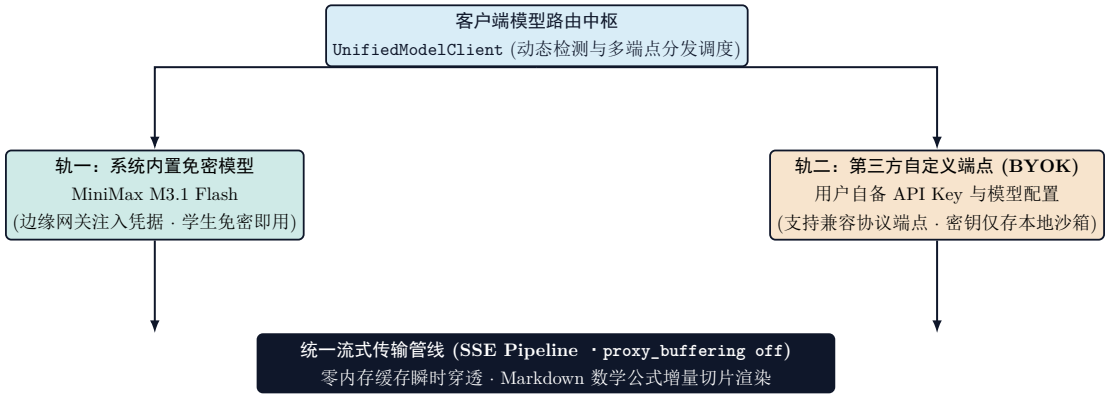



图 16.1: 《问矩》双轨模型接入与流式传输架构拓扑

两轨技术特征与适用场景严格解耦：

1. **系统内置免密模型 (MiniMax M3.1 Flash)**：针对线性代数符号演算与教学多轮对话专门适配，系统在腾讯云边缘代理端统一托管安全凭据。普通学生访问无需输入任何密钥即可直接调起大模型进行完整 PECHDT 探究实验，实现“零摩擦接入”；

2. **客户端自备凭据 (BYOK - Bring Your Own Key)**：为教学演示或特定场景提供第三方兼容模型热插拔能力（如用户自备 DeepSeek 官方端点凭据）。API Key 仅存放于浏览器物理沙箱内存及本地加密信封中，请求头直接直传，服务器端严格零留存。

## 16.2 流式传输（SSE）管线与反代零缓存穿透

教学推演过程中，大模型生成完整的数学证明或矩阵解释往往需要输出数百甚至上千 Token。若采用常规 HTTP 一次性等待返回，首字延迟（Time to First Token, TTFT）将高达数秒，破坏人机协同的即时感。为此，《问矩》全链路采用服务器发送事件（Server-Sent Events, SSE）流式响应管线。

### 16.2.1 边缘代理零缓冲配置契约

标准反向代理服务器（如未调优的 Nginx）默认开启 `proxy_buffering on`，会将上游传回的分块数据累积达到 4KB ∼ 8KB 后才向客户端刷新一次，导致流式打字机效果瞬间退化为“卡顿式批量吐字”。《问矩》在生产环境边缘代理配置文件 `deploy/nginx-teachagent.conf` 中明确固化了流式零缓冲指令：

```
# 关闭所有中间缓冲，保证单 Token 瞬时穿透分发
proxy_buffering off;
proxy_cache off;
proxy_read_timeout 300s;
proxy_send_timeout 300s;
chunked_transfer_encoding off;

# 动态反代目标端点（避免启动阶段 DNS 强依赖解析阻塞）
proxy_pass $target_proto://$target_host$target_path$is_args$args;
proxy_ssl_server_name on;
proxy_ssl_name $target_host;
proxy_set_header Host $target_host;
```

Listing 16.1: Nginx 生产反代流式传输与零内存缓存配置

### 16.2.2 客户端数学公式流式缓冲切片机制

在前端接收 SSE 流（`ReadableStreamDefaultReader`）时，数学公式定界符（如 `$`、`$$`、`\(`、`\[`）在传输过程中常被随机截断在两个独立的 Chunk 中（例如前一帧接收到 `det(`，后一帧接收到 `A) = 0$`）。若直接交给 KaTeX 解析将抛出未闭合语法异常。

客户端在 `MarkdownRenderer` 与 `KaTeXRenderer` 中实现了专用的流式数学定界符解析机制：在流式输出（`isStreaming: true`）期间动态捕获未闭合的数学定界符（如尾部未闭合的 `$$...`），将其暂存为安全数学块占位符，避免未闭合语法引发 KaTeX 编译崩溃；非数学纯文本内容在当前宏任务中即时更新至 DOM 节点，保障打字机视觉流转平滑自洽。

## 16.3 双轨模型接入架构与自备凭据安全隔离规范

在《问矩》的模型接入体系中，系统确立了“内置免密托管与第三方自备凭据完全物理隔离”的安全契约：

1. **内置免密模型安全通道**：对于系统内置的 MiniMax M3.1 Flash 模型，客户端请求不携带任何真实 API Key，而是直接向同源反向代理端点（如 `/api/proxy`）发起请求。腾讯云边缘网关根据访问白名单在服务端流式注入安全密钥，实现学生端“零门槛、免配置、零凭据泄漏风险”的即开即用体验；

2. **第三方外部端点 (BYOK) 沙箱隔离**：对于需要使用自定义模型的教学与科研场景，系统在设置抽屉中提供统一的兼容模型配置面板。用户可自由填入自定义端点 URL、API Key 与 Model ID（前端快捷预设支持官方 DeepSeek 端点，如 `deepseek-flash` 等模型）。系统在客户端建立物理隔离契约：

   - **服务器零存储**：用户配置的外部 API Key 仅保存在浏览器本地加密存储信封中，或直接在运行时注入内存，请求发起时通过动态反代变量直传，服务端绝不写入任何数据库或持久化日志；
   - **凭据不互通原则**：外部模型请求绝不借用内置模型的凭据，无凭据时直接置为不可用态，杜绝跨租户权限越界；
   - **模型输出格式契约**：无论后端路由至何种模型，均需严格遵从相同的 JSON Schema 工具调用协议与 SSE 流式返回契约，未满足格式约束时由 `UnifiedModelClient` 在前端执行统一防御截断。

## 16.4 助教跨轮启发协议、双重确定性门禁与 14 项注册工具体系

在具体的教学推演场景中，为了杜绝大模型在解题过程中的随意性与幻觉剧透，《问矩》确立了严格的助教跨轮交互协议、确定性门禁与工具可见性隔离契约。

### 16.4.1 六步跨轮探究协议与动态系统提示注入

大语言模型助教绝非无序的开放式 Chatbot，而是严格受限于系统级状态机调度。系统将跨轮交互固化为六大标准阶段：

1. **预测（Prediction）**：助教首先询问学生的几何空间直觉与代数猜想，要求给出先验判断与理由，严禁助教直接抢答或代学生给出预测结论；

2. **实验（Exploration）**：在引导学生选定或确认测试矩阵参数后，助教通过 MCP 调用本地精确计算工具，**所有数值事实完全由工具生成**，模型不得凭空编造矩阵乘积或特征值；

3. **冲突（Conflict）**：将学生的初始预测与工具返回的物理事实进行逐项对比，若吻合则确认，若相悖则暴露认知冲突；

4. **边界猜想（Boundary）**：启发学生思考“该现象在何种代数约束下恒成立（充要条件）”；

5. **推导验证（Deduction）**：引导学生给出形式化证明，严格区分特定数值特例、几何反例证伪与通用代数证明；

6. **无提示变式（Transfer）**：生成改变了边界条件的无提示迁移习题，不附带参考答案，交由学生独立验证。

在每个时间步，当前所处的探究环节编号与教学约束作为**动态高优先级系统提示词（Dynamic System Prompt）**注入模型请求头，严禁助教一次性把六步答案全盘托出。

### 16.4.2 双重前端前置确定性门禁

为了在最底层防范大模型“被学生诱导直接给出作业答案”，系统在请求发出前构建了双重前置硬性门禁：

1. **门禁一：防剧透直接要答案拦截（`PechdtScaffoldSkill`）：**当学习者在输入框输入“直接告诉我答案”、“这题选什么”、“帮我写出证明过程”等索要答案意图时，`PechdtScaffoldSkill` 在前端网络层直接拦截请求，截断模型 API 调用，并回显引导性提示：“科学探究需要先提出猜想并操作画板验证，请先拖动红色基向量看看图形怎么变化”，完全杜绝算力浪费与教学违规；

2. **门禁二：反思流确定性控盘（`InquiryFlowController`）：**当学习者使用“我怀疑/我猜”等句式表达对核心概念（如矩阵乘法交换律、核空间维数、行列式可加性）的质疑时，`InquiryFlowController` 立即接管控制权。系统直接按固化的确定性模板走完预测、实验对照、区分性追问、推导和变式计分。**该路径完全不依赖大语言模型，实现零模型 Token 消耗、本地即时响应与确定性状态流转**。变式题上限为 3 题，全部通过则关闭任务，存在未通过项则在本地学习档案中标注复习路径。

### 16.4.3 几何关卡注册表（`InquiryScenarioRegistry`）的真实工程状态

在源码 `js/scenarios/InquiryScenarioRegistry.js` 中，系统预置了 6 个几何关卡（奇异降维、零因子、负行列式手性、保面积剪切、特征向量、SVD 主方向），包含完整的判定函数与数学证明文本。必须予以严谨工程声明：**在当前生产环境中，该脚本虽然随网页加载，但工作台前台界面未直接开放调用入口，目前排他性地用于自动化单元测试套件的基准功能核验**。

### 16.4.4 默认 14 项注册工具与 2 项隐藏工具边界

模型可调用的工具集合严格受限于表 16.1。系统对工具实行三级访问权限隔离：`readonly`（只读感知，静默执行）、`ask`（涉及状态变更，强制弹出一次确认弹窗，由学生点击授权方可执行）与 `full`（开发调试授权）。写文件、跑代码、生成新工件等危险工具均被标记为 `ask`。

表 16.1: 大语言模型默认注册的 14 项工具契约与权限配置

| 工具名称 | 权限级别 | 交互影响 | 形式化调用职责 |
|---|---|---|---|
| `math_experiment` | `ask` | 变更工件视口 | 打开、更新、回读、分镜播放、作答核验及标注重叠检查 |
| `inspect_workspace` | `readonly` | 无状态影响 | 只读回读当前画板矩阵、工件状态及实时输入 |
| `solve_matrix_kernel_sympy` | `readonly` | 无状态影响 | $2\times2$ 矩阵行列式、秩、零度及零空间基底符号计算 |
| `solve_linear_system` | `readonly` | 无状态影响 | 判定 $A\mathbf{x}=\mathbf{b}$ 相容性，输出特解与通解空间基底 |
| `construct_counterexample_matrix` | `readonly` | 无状态影响 | 构造五类反例矩阵：零积、非交换、非加性、逆序、秩不等式 |
| `drive_canvas_affine_transform` | `ask` | 改变画板形态 | 驱动画板补间至奇异/剪切/晶体态，高亮特征线与核向量 |
| `render_canvas_graphics` | `ask` | 叠加画板图元 | 在平面画板上绘制菱形、多边形、圆形、矩形及函数曲线 |
| `render_interactive_artifact` | `ask` | 渲染工件沙箱 | 沙箱内渲染自定义 HTML/SVG 或参数化立体几何 |
| `play_manim_lesson` | `readonly` | 播放微课视频 | 播放已有微课片段，查询本地渲染能力与任务状态 |
| `query_knowledge_graph` | `readonly` | 无状态影响 | 查询 12 个概念节点的前置依赖关系与下一步建议 |
| `evaluate_pedagogical_scaffold` | `readonly` | 无状态影响 | 评估当前探究环节引导质量，执行防剧透合规检查 |
| `execute_code_sandbox` | `ask` | Worker 执行 | 在 Web Worker 中执行 JS；Python 经转译子集后安全执行 |
| `manage_workspace_file` | `ask` | 文件物理写 | 读写已授权工作区文件或浏览器虚拟文件系统 |
| `generate_visual_illustration` | `ask` | 生成外部图像 | 调用可图 (Kolors) 生成插图，异常时优雅降级 |

**隐藏工具边界声明**

前端脚本虽已加载 `translate_technical_text`（技术文本翻译）与 `multimodal_ocr_document_parser`（多模态文档 OCR），但**系统默认技能清单中排除了此两项工具，大模型在上下文提示词中不可见且无法调用**，杜绝无关工具污染智能体注意力空间。

# 第十七章　轻量 Schema 校验器与工具邻居守卫 (ToolNeighborGuard)

## 17.1　函数调用（Tool Calls）的语义漂移与近邻混淆

大语言模型在执行函数调用（Function Calling / Tool Calling）时，其输出本质上是基于概率分布自回归生成的 JSON 字符串。在涉及高维矩阵与几何代数的探究场景中，大量工具在自然语言语义空间上高度相邻。例如：

- “求解矩阵的零空间 (`solve_matrix_kernel_sympy`)”；
- “驱动画布执行仿射变换动画 (`drive_canvas_affine_transform`)”；
- “构造矩阵乘法反例 (`construct_counterexample_matrix`)”；
- “求解非齐次线性方程组 (`solve_linear_system`)”。

在多工具大模型智能体系统中：**当挂载的专业工具数量增加且存在概念近邻词时，大模型产生错误路由（Tool Routing Hallucination）与幻觉调用的概率显著上升**。传统系统往往依赖大模型“就近挑一个”，这种机制在严格的教育与数学场景中是不可接受的，极易诱导学生建立错误的数学先验认知。

## 17.2　轻量 Schema 校验器与零依赖断言

为杜绝大模型吐出畸形 JSON 参数导致前端崩溃，《问矩》拒绝引入动辄数十 KB 的重量级通用验证库（如 Zod 或 Ajv），而是在客户端构建了零外部依赖的轻量契约校验器。

校验器对大模型返回的参数进行强类型三阶断言：

1. **一阶：存在性与基类型断言**：断言必填参数（如 `matrix`、`vector`）非空，且符合 JavaScript 原生数据类型要求；
2. **二阶：数学维度与数值边界钳位 (Clamping)**：断言二维矩阵尺寸严格为 $2 \times 2$ 或 $3 \times 3$，浮点数未发生 NaN 或 $\pm\infty$ 溢出，数值限定在渲染安全区间 $[-100.0, 100.0]$；
3. **三阶：畸形语法自动纠错**：利用状态机对大模型偶尔漏写闭合括号（如 `[[1, 0], [0, 1]`）执行尾部补全，最大程度保障执行流连续。

## 17.3 工具邻居守卫 (ToolNeighborGuard) 核心算法实现

为从根源上消除语义近邻工具误调用，《问矩》设计并实现了轻量确定性守卫模块 `js/teaching/ToolNeighborGuard.js`。该模块确立了一条刚性红线：**当学生提问处于近邻概念模糊区间时，系统宁可显式拒答并追问引导，也绝不盲目猜测路由！**

### 17.3.1 守卫路由规则表定义

在守卫内部，预先定义了正交的“判别关键词（Keywords）”与“近邻干扰词（Neighbors）”映射表：

```
const ROUTES = [
  {
    tool: 'solve_matrix_kernel_sympy',
    keywords: ['零空间', '核空间', 'null space', 'kernel', 'nullspace'],
    neighbors: ['列空间', '行列式', '秩']
  },
  {
    tool: 'drive_canvas_affine_transform',
    keywords: ['拖', '画布变换', '补间', '降维', 'singular',
        '演示.{0,8}变换'],
    neighbors: ['渲染图元', '求解']
  },
  {
    tool: 'construct_counterexample_matrix',
    keywords: ['反例', 'counterexample', '构造.*矩阵', 'ab.*=.*o'],
    neighbors: ['验证', '求解']
  },
  {
    tool: 'solve_linear_system',
    keywords: ['方程组', '线性方程', '解.*ax *= *b', '增广'],
    neighbors: ['零空间', '特征']
  }
];
```

Listing 17.1: ToolNeighborGuard 路由与近邻规则表 (js/teaching/ToolNeighborGuard.js)

### 17.3.2 三态分类决策状态机

函数 `classify(query)` 实现严格的确定性分类流转：

```
function classify(query) {
  const q = String(query || '');
  const hits = [];
```

```
  for (const r of ROUTES) {
    if (r.keywords.some((k) => new RegExp(k, 'i').test(q)))
        hits.push(r.tool);
  }

  // 1. 唯一命中合法判别词 -> 准许调用
  if (hits.length === 1) {
    return { tool: hits[0], matched: true, reason: '判别词唯一命中' };
  }

  // 2. 多工具歧义命中 -> 显式拒答，要求澄清
  if (hits.length > 1) {
    return {
      tool: null,
      refused: true,
      reason: '多工具同时命中（' + hits.join(', ') +
          '）——歧义查询拒答，请澄清意图'
    };
  }

  // 3. 无判别词命中但踩中近邻词（概念混淆征兆）-> 显式拒答并追问
  const neighborHit = ROUTES.some((r) => r.neighbors.some((n) =>
      q.includes(n)));
  if (neighborHit) {
    return {
      tool: null,
      refused: true,
      reason: '命中近邻词无判别词——拒答并追问引导（防语义近邻误路由）'
    };
  }

  // 4. 普通自然语言对话
  return { tool: null, refused: false, reason: '无工具命中（普通对话）'
      };
}
```

Listing 17.2: 三态分类决策状态机核心逻辑

该算法在数学概念边界处建立了确定性防波堤，杜绝了由于大模型自身语义混淆而将“求列空间基底”错误路由为“求零空间”的高危教学事故。

# 第十八章　模型幻觉抑制与认知流转的失效模式建档

## 18.1　线性代数大模型 10 类典型概念幻觉与 Bad Cases 库

通用大语言模型在没有形式化代数内核约束时，在处理符号代数与多维几何任务时普遍存在系统性幻觉。《问矩》研发团队通过真实测试集与学生探究交互归纳建立了 10 类典型概念幻觉库，如表 18.1 所示。

表 18.1: 线性代数大语言模型 10 类典型概念幻觉建档与测试反例

| 编号 | 幻觉类型 | 典型输入与错误表现 (Bad Case) |
|---|---|---|
| HAL-01 | 奇异矩阵强行求逆 | 对 $A=\begin{pmatrix}1 & 2\\ 2 & 4\end{pmatrix}$ 生成逆矩阵表达式，无视 $\det(A)=0$ |
| HAL-02 | 虚构特征向量空间 | 宣称亏损矩阵 $\begin{pmatrix}1 & 1\\ 0 & 1\end{pmatrix}$ 存在两个线性无关的实特征向量 |
| HAL-03 | 线性变换与仿射平移混淆 | 将 $f(\mathbf{x})=A\mathbf{x}+\mathbf{b}$ $(\mathbf{b}\neq\mathbf{0})$ 误判为满足可加性与齐次性的线性变换 |
| HAL-04 | 矩阵乘法可交换性假象 | 断言对于任意同阶方阵均有 $AB=BA$，并据此推导平方差公式 |
| HAL-05 | 零空间与正交补维度混淆 | 在 $\mathbb{R}^3$ 中混淆 $\dim(\ker(A))$ 与 $\dim(\mathrm{col}(A))$ 的直和互补关系 |
| HAL-06 | 行列式乘法性质符号颠倒 | 错误断言 $\det(A+B)=\det(A)+\det(B)$ |
| HAL-07 | 正交矩阵特征值模长发散 | 给出实正交矩阵却输出模长大于 1 的复特征值 |
| HAL-08 | 降维空间拓扑投影失真 | 在 3D 到 2D 投影时将不相交的异面直线断言为三维空间相交 |
| HAL-09 | 符号消元过程中被除数非零遗漏 | 在求解含参方程组时直接除以参数表达式，遗漏分母为零分支 |
| HAL-10 | 工具调用实参符号反转 | 将顺时针旋转 $90^\circ$ 误填为正角度参数输出给几何画布 |

## 18.2　双层防御架构：代码硬断言与提示词防护栏

为抑制上述 10 类典型幻觉，《问矩》构筑了“底层代码代数硬断言 + 认知层提示词防护栏”的双层协同防御架构：

1. **确定性代数核验回路 (Deterministic Symbolic Grounding)**：大模型给出的任何数学定性结论（如“矩阵是可逆的”、“特征向量为 $\mathbf{v}$”），前端引擎在接收到后不会直接呈现给

学生，而是由纯前端轻量符号库自动执行物理验算：

$$\text{Residual} = \|A\mathbf{v} - \lambda\mathbf{v}\|_2$$

若 Residual $> 1.0 \times 10^{-5}$，系统立即判定模型输出存在幻觉，前端自动拦截该回答并向大模型注入系统级修正提示（“检测到代数残差不守恒，请重新推导并纠正”），完成闭环自愈；

2. **提示词防御约束流 (Negative Prompt Constraints)**：在系统 Prompt 中显式注入不可逾越的教学契约：严禁直接向学生提供终局答案；必须采用启发式追问；面对学生错误先验时必须引导其在画布上拖拽出反例，而非单纯以说教替代实验。

## 18.3 认知流转潜在失效模式建档与基准度量

### 18.3.1 系统级失效模式建档

如表 18.2 所示，系统对多智能体认知流转可能遭遇的三大极端异常场景进行了形式化建档与防御收敛。

表 18.2: 多智能体调度与认知流转潜在失效模式建档

| 失效编号 | 触发条件与系统表象 | 内核防御与自愈机制 | 物理恢复断言 |
|---|---|---|---|
| FAIL-AG-01 | 上游模型端点超时、网络中断或无可用 API Key | 启动指数退避重试 (最多 2 次)；无可用 Key 时对话引擎置为不可用禁用态，绝不使用本地模型代答 | 显式展示配置引导与重试按钮,保持画板交互可用,主线程不崩溃 |
| FAIL-AG-02 | 大模型生成无效 JSON 导致语法解析异常 | 轻量 Schema 校验器自动启动尾部符号修复；若修复失败则优雅回退为纯文本回复 | 保证主线程不抛出未捕获异常，记录错误至审计通道 |
| FAIL-AG-03 | 连续两次调用 ToolNeighborGuard 触发歧义拒答 | 状态机判定学生陷入严重概念死锁，自动触发认知探针，重置引导至上一梯度实验 | 清空当前局部歧义堆栈，恢复至前一稳定 PECHDT 节点 |

### 18.3.2 双层防御架构的形式化物理测试套件与安全断言

《问矩》坚决拒绝脱离可执行代码与真实测试资产的无依据推断。为了对双层防御架构的有效性建立形式化工程断言，系统构建了专用的自动化单元测试套件（如 `tests/edge_cases/test_tool_neighbor_rejection.js` 与 `tests/test_pechdt_guard.js`）：

1. **工具调用边界与语义歧义拦截**：通过编写覆盖边界条件的测试用例，断言当模型尝试调用未授权工具、传递超出 Schema 约束的非法矩阵维度或构造语义模棱两可的参数时，`ToolNeighborGuard` 必须以 100% 的确定性抛出拦截异常，阻止状态脏写；

2. **代数事实物理隔离**：大语言模型在体系结构上被剥离任何直接输出未经验证代数结论的权限。所有特征值、行列式、秩、逆矩阵与正交投影的计算，排他性地交由客户端本地 `MathComputeEngine` 物理计算内核生成。测试套件断言：无论上游模型生成何种文本，底层画布渲染与判卷核验的数据源必须 100% 取自本地计算结果；

3. **前端确定性门禁拦截验证**：对“直接索要答案”、“代写证明”等探究违规指令，测试套件断言 `PechdtScaffoldSkill` 在请求发出前完成 100% 前置拦截，直接向学生返回引导性实验提示，不产生任何网络 Token 消耗。

# 第七部分

# 前端存储架构、会话信封迁移与滑动即焚看门狗

# 第十九章　会话数据信封（Envelope v1）与模式版本迁移

## 19.1　纯前端状态持久化挑战与信封规范

在纯前端无状态单页应用（Stateless SPA）架构下，所有学生的交互记录、认知状态机演进轨迹与画布矩阵快照均需在浏览器沙箱本地进行安全持久化。若缺乏规范的模式约束，历史数据直接以裸数组或零散键值存储，系统在经历功能迭代与模式变更时，将面临灾难性的版本断裂与反序列化崩溃风险。

为了实现高可靠的存储健壮性，《问矩》遵循规范的会话持久化与单向数据迁移契约，设计了标准会话数据信封（Envelope v1）。

### 19.1.1　数据信封标准模式结构

会话存储数据在写入本地物理存储（Web Storage 或 IndexedDB）时，强制包装为标准的信封对象：

```
{
  schemaVersion: 1,                    // 严格模式版本号（整型）
  sessions: [                          // 探究任务会话实体数组
    {
      id: "session_1730000000_abcd",
      title: "特征子空间探究 #1",
      createdAt: 1730000000000,
      updatedAt: 1730000015000,
      matrixState: [[2, 1], [1, 2]],// 画布矩阵历史状态快照
      cognitiveStage: "EXPLORE",     // 当前 PECHDT 阶段
      turns: [ /* 交互问答轮次 */ ]
    }
  ],
  activeSessionId: "session_1730000000_abcd", // 当前活跃任务指针
  savedAt: 1730000015000               // 物理落盘时间戳（毫秒）
}
```

Listing 19.1: Envelope v1 数据信封格式规范

## 19.2 模式版本迁移器（PayloadMigrations）设计与形式化实现

系统演进过程中，早期版本的存储载荷（如 v0 阶段的历史裸数组格式 `Array<Session>`）不可避免地存在于部分用户的浏览器中。为了避免粗暴清空用户数据，《问矩》在 `js/storage/PayloadMigrations.js` 中实现了有序链式版本迁移器。

### 19.2.1 迁移器四大核心守恒契约

迁移系统严格遵循四大工程契约：

1. **有序单向推进 (Strictly Monotonic Order)**：迁移链按版本号升序排列（to > from），仅允许向前迁移，禁止非确定性的逆向回退；
2. **执行幂等性 (Idempotency)**：对已处于目标版本的载荷重复执行迁移，返回相同且无副作用的结果；
3. **原子性与原始载荷保全 (Atomicity & Fail-Safe Fallback)**：迁移链中任何一个中间步骤发生异常（如格式损坏），迁移器立即终止，**无损返回原始载荷**，严禁将半迁移状态写入物理存储；
4. **未来版本显式拒绝 (`VERSION_FUTURE`)**：当读取到版本号大于当前运行时最高支持版本的载荷时（可能发生于用户在同一浏览器使用新旧两版系统），显式抛出拒绝，严禁以盲目降级破坏高版本结构。

### 19.2.2 真实生产迁移器核心逻辑

```
const REGISTRY = {
  session_history: [
    {
      id: 'session_history_v0_to_v1',
      to: 1,
      migrate(payload) {
        if (!Array.isArray(payload)) {
          return { ok: false, code: 'MIGRATION_SHAPE_INVALID', message:
              'v0 载荷必须为裸数组' };
        }
        // 过滤畸形条目并提取合法会话
        const sessions = payload.filter((s) => s && typeof s ===
            'object' && typeof s.id === 'string');
        const active = sessions.length ? sessions[0].id : null;
        return {
          ok: true,
          payload: { schemaVersion: 1, sessions, activeSessionId:
              active, savedAt: null }
```

```
        };
      }
    }
  ]
};

function runMigrations(payload, kind) {
  const currentVer = versionOf(payload, kind);
  const chain = REGISTRY[kind] || [];
  let currentPayload = payload;

  for (const step of chain) {
    if (step.to > currentVer) {
      const res = step.migrate(currentPayload);
      if (!res.ok) {
        // 原子回退：任一步失败返回原始载荷，避免脏写
        return { ok: false, payload, error: res.code };
      }
      currentPayload = res.payload;
    }
  }
  return { ok: true, payload: currentPayload };
}
```

Listing 19.2: PayloadMigrations 注册表与迁移链执行器 (js/storage/PayloadMigrations.js)

## 19.3 多会话生命周期与存储端口解耦

在 `js/storage/SessionHistoryManager.js` 中，《问矩》实现了多任务会话管理器，支持：

- **会话动态提升**：首轮对话完成后，根据用户输入自动提取语义关键词重命名会话标题（取代默认的“新探究任务”）；
- **内存与多存储后端解耦**：抽象存储端口（`Port` 规范），默认基于 `localStorage`，并预留挂载 IndexedDB 离线存储的接口能力；
- **全文索引检索**：支持根据关键词对历史会话的用户提问、模型回复与思维链过程进行即时线性过滤。

# 第二十章　15 秒轮询心跳与 8 分钟（480,000ms）滑动即焚看门狗

## 20.1　“零残留、强隔离”隐私安全契约

高校计算机机房与移动教学终端具有典型的“公共共享”特征。在无监管的传统系统中，前一位学生退出浏览器后，其对话上下文、推演记录乃至自备的 API Key 往往静默残留在浏览器缓存中，构成严重的数据泄露隐患。《问矩》提出并工程化落地了**“阅后即焚、闲置强清”**的隐私安全看门狗体系。

形式化安全契约定义如下：

$$\forall \text{Client}, \quad \text{IdleTime} \geq T_{\text{TTL}} \implies \text{Purge}(\text{Memory}) \wedge \text{Purge}(\text{Storage}) \wedge \text{Reset}(\text{UI})$$

其中系统基线超时闲置阈值设定为：

$$T_{\text{TTL}} = 480{,}000 \text{ ms } (8 \text{ 分钟})$$

## 20.2　8 分钟滑动即焚看门狗时序与自愈状态机

看门狗由两个正交机制协同驱动：高频活动感知续期（`touch()`）与低开销周期性心跳巡检（Heartbeat Polling），如图 20.1 所示。

### 20.2.1　高频活动感知 (touch)

在 `js/storage/SessionHistoryManager.js` 中暴露轻量级 `touch()` 探针：

```
touch() {
  this.lastActiveAt = Date.now();
}
```

所有核心交互事件（用户在输入坞键入字符、拖拽数学画布几何点、切换手绘图元、收到大模型流式生成片段）均触发 `touch()`，实时刷新活跃时间戳。

### 20.2.2　15 秒高精度周期巡检与漂移补偿

前端在主入口 `js/app.js` 中注册全局心跳守护定时器：

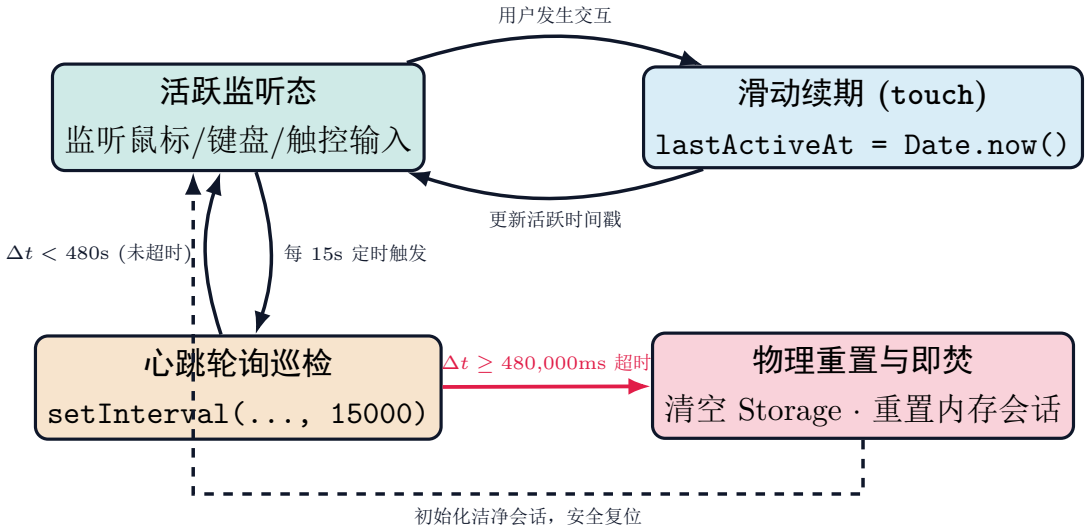



图 20.1: 8 分钟滑动即焚看门狗生命周期与自愈状态机

```
// 每 15 秒心跳巡检一次会话是否过期
setInterval(() => sessionHistoryManager?.checkTtlExpired?.(), 15000);
```

巡检核心判定逻辑如下：

```
checkTtlExpired() {
  if (this.ttlMs > 0 && (Date.now() - this.lastActiveAt > this.ttlMs)) {
    this.resetToInitialSession('新探究任务（已自动清理超时会话）');
    this._notify('session_expired', { reason: 'IDLE_TIMEOUT', ttlMs:
        this.ttlMs });
    return true;
  }
  return false;
}

resetToInitialSession(title = '新探究任务') {
  const s = this._getStorage();
  if (s) {
    try {
      // 物理擦除所有持久化键值
      s.removeItem(this.storageKey);
      s.removeItem(this.activeKey);
    } catch (_) {}
  }
  // 内存数据解构归零，重置单一干净会话
  const initialSession = this._createSessionRaw(title);
  this.sessions = [initialSession];
```

```
  this.activeSessionId = initialSession.id;
  this.lastActiveAt = Date.now();
  this._saveSessions();
  this._saveActiveSessionId();
  this._notify('sessions_reset', { activeSession: initialSession });
  return initialSession;
}
```

Listing 20.1: 超时判定与物理擦除流水线 (js/storage/SessionHistoryManager.js)

## 20.3 极端时序边界与后台休眠时钟漂移补偿

### 20.3.1 休眠与后台挂起唤醒补偿

现代移动端浏览器（iOS Safari、Chrome on Android）在用户切换至后台或系统休眠时，会主动节流甚至完全冻结 `setInterval` 定时器，导致休眠期间无法按时触发巡检。

《问矩》依托标准 W3C `visibilitychange` 事件实现了时间差量补偿：当学生合上笔记本或切出应用 10 分钟后重新打开页面，系统立即捕获 `document.visibilityState === 'visible'` 事件并瞬间调用 `checkTtlExpired()`。由于此时 $\Delta t = \text{Date.now}() - t_{\text{last}} \geq 600{,}000\text{ms} > 480{,}000\text{ms}$，看门狗将在首帧物理清除旧会话，防止过期数据产生半秒钟的视觉闪烁。

### 20.3.2 长流式生成的保活锁机制

若学生在大模型进行长文本推演的第 7 分 59 秒未发生鼠标移动，流式生成过程本身会持续触发传输切片，数据流推进被视为系统活跃态，自动调用 `touch()`，杜绝了“模型正在推导数学公式而系统误杀会话”的边缘时序缺陷。

# 第二十一章　客户端存储边界极限与并发隔离威胁分析

## 21.1　存储配额耗尽与滑动修剪策略

浏览器的 `localStorage` 存在固定的配额硬限制（主流浏览器通常为 5MB ～ 10MB）。若学生进行了大量深度绘图与多轮多模态探究，Base64 图元快照可能导致存储溢出抛出 `QUOTA_EXCEEDED_ERR`。

《问矩》在管理器中构建了 LRU 滑动淘汰机制：

1. **最大会话数上限截断**：全局限定最多保留 50 个历史探究任务（`maxSessions = 50`）；

2. **滑动修剪算法 (Sliding Window Truncation)**：当捕获到写入配额溢出异常时，系统不中断当前操作，而是按 `updatedAt` 时间戳升序对最旧的非活跃会话执行自动修剪，释放存储空间后再行提交；

3. **手绘快照外置轻量化**：画板导出的笔触仅持久化轻量矢量数学参数（起点、终点、控制点），禁止将数兆字节的原始 PNG 像素数据序列化至信封中。

### 21.1.1　长对话压缩器与复习包/演示包导出机制

长程探究交互中，多轮对话上下文容易突破模型上下文窗口限制并推高 Token 成本。系统客户端内置了**上下文滑动压缩器（Context Compactor）**：当会话轮次超过阈值时，自动将早期探究步骤的细节折叠为结构化状态差量（State Delta），仅保留关键代数假设与最终结论。

同时，客户端支持一键导出标准化的**复习包（Review Package）**与**演示包（Demo Package）**（导出格式为 `wenju-course-journal.json` 或包含轨迹回放快照的自包含文件），方便学生离线复盘与教师课堂展示。

**可观测性数据与判分逻辑物理隔离契约**

系统采集的客户端结构化日志、Span/Trace 追踪链与空间遥测数据（`SpatialTelemetryBus`），**仅排他性地用于前端性能监控、微气泡触发与教学诊断观测，不参与课程任务的评分与对错核验。**

## 21.2 多标签页并发访问隔离与冲突消解

当学生在同一台机器的同一浏览器中同时开启多个《问矩》标签页时，多端并发写入可能造成覆盖脏写。

系统基于浏览器的 `window.addEventListener('storage', ...)` 事件建立了跨标签页协同总线：

- 每一个会话信封内部维护单调递增的逻辑版本号（`_revision`）；
- 当标签页 A 触发了滑动即焚重置，标签页 B 瞬间接收到存储键值移除广播，并同步在自身内存中触发销毁重置，杜绝了单侧标签残留数据的跨端泄漏。

## 21.3 存储子系统的理论空间复杂度与迁移一致性分析

### 21.3.1 空间复杂度与浏览器配额安全边界

《问矩》采用纯本地存储（Local Storage）方案，为了确保用户在长周期探究中不突破浏览器配额（主流浏览器单域名存储上限通常为 5MB），系统确立了以下理论空间模型：

1. **文本消息体积收敛：**单轮问答（学生输入 + 助教引导 + 工具回传）的 JSON 序列化字符数通常在 500 ∼ 1500 字符之间。以 UTF-16 编码计算，单轮物理存储增量约为 1KB ∼ 3KB。在连续 50 轮深层次实验探究后，整体会话体积稳定收敛在 150KB 以内，仅占浏览器 5MB 配额的约 3%，具备数十倍的安全余量；
2. **画布快照不存大位图：**系统在持久化会话与撤销快照时，仅序列化 $2\times2$ 或 $3\times3$ 的浮点数矩阵参数、基向量坐标与手绘折线坐标数组，严禁将未压缩的 Base64 大图写入 LocalStorage，从机制上杜绝了存储爆炸；
3. **迁移向后兼容性契约：**在真实自动化测试用例 `tests/unit/test_payload_migrations.js` 中，系统断言：当载入低版本历史信封（如缺少特定教学字段的遗留结构）时，迁移流水线通过模式补全机制原子化升级至最新版本，确保字段结构完整。

### 21.3.2 阅后即焚与内存物理清空安全契约

在真实自动化测试套件 `tests/unit/test_ephemeral_session_ttl.test.js` 中，系统对即焚机制确立了形式化的生命周期断言：

- **超时生命周期断言：**系统配置 `SESSION_TTL_MS = 480000`（8 分钟无操作惰性销毁）。测试套件模拟时钟推进至超时后，断言下次读操作触发物理清空，调用 `localStorage.getItem('teachagent_sessions')` 返回初始空信封；
- **主动即焚物理删除：**当学习者手动触发“阅后即焚”时，系统调用原生 `localStorage.removeItem`，从物理介质上销毁键值，同时内存中的会话数组重置为单例初始态，确保敏感教学对话与临时实验草稿零物理残留。

# 第八部分

# 151 项全量工程测试套件与对抗验证体系

# 第二十二章　单一清单源驱动的测试架构与质量契约

## 22.1　传统前端测试“割裂与虚假覆盖率”之痛

在传统 Web 系统的工程测试实践中，测试套件往往随着功能迭代产生严重的碎片化与逻辑漂移。最典型的缺陷体现在测试环境与真实运行时的脱节：大量前端团队依赖基于 Node.js 的无头虚拟 DOM 库（如 jsdom）运行单元测试。然而，jsdom 对 HTML5 Canvas 2D 上下文（`CanvasRenderingContext2D`）的实现是不完整的桩对象（Mock Stub），对 WebGL 3D 渲染上下文更是完全缺乏支持。

这种“在虚拟沙箱中跑通，在真实浏览器中崩溃”的假象，导致了虚假的高代码覆盖率指标。当复杂的数学矩阵连续补间动画、手绘贝塞尔曲线平滑、离屏双层光栅化合并在真实的移动端或桌面端渲染引擎（如 Blink / WebKit）中运行时，由于坐标系变换漂移、DPR 设备像素比未对齐、事件侦听器冒泡阻断等问题，故障频发。

为了防范测试环境与生产运行时发生逻辑漂移，《问矩》确立了**“拒绝虚假 Mock、全真浏览器驱动、单一清单源”**的工程质量契约（X1 契约）。

## 22.2　单一清单源（X1 契约）设计与统一调度体系

系统在 `tests/run_all_tests.js` 中固化了全仓库唯一的测试清单源。项目构建脚本、本地回归以及 CI/CD 流程中的所有测试命令，一律统一委托至该脚本进行单点调度。

### 22.2.1　X1 契约的核心工程法则

1. **唯一真值清单 (Single Source of Truth)**：全仓库所有测试套件的注册与剔除必须在 `tests/run_all_tests.js` 中单点配置，杜绝多个配置文件各自维护子集导致测试遗漏；
2. **0 SKIP / 0 FAIL 验收准则 (B3-NEG 口径)**：任何测试套件输出包含 `[SKIP]` 标记时，执行器显式计入跳过计数，严格禁止将“跳过（SKIP）”粉饰为“通过（PASSED）”。在交付验收基线中，必须达成：

$$\text{Passed} = 151, \quad \text{Failed} = 0, \quad \text{Skipped} = 0$$

3. **失败快速阻断与调用栈完整保全**：任何一个用例抛出未捕获异常或断言失败，测试调度器立即终止构建并原样输出终端错误上下文，退出码严格置为非零（`process.exit(1)`）。

### 22.2.2 本地轻量静态服务生命周期托管

端到端真实浏览器测试需要加载本地静态资源。为了消除开发者在运行测试前手动启动本地 HTTP 服务的操作负担，`run_all_tests.js` 实现了轻量级端口探测与自足托管机制：

```
let httpServerProc = null;
const TEST_PORT = Number(process.env.WENJU_TEST_PORT || 8000);

async function ensureLocalServer() {
  try {
    // 优先探测已有本地服务 (1.5s 超时)
    await fetch(`http://127.0.0.1:${TEST_PORT}/`, { signal:
        AbortSignal.timeout(1500) });
    return; // 服务已存在，直接复用
  } catch (e) {}

  // 动态拉起轻量 HTTP 托管守护进程
  httpServerProc = spawn('python', ['-m', 'http.server',
      String(TEST_PORT), '--bind', '127.0.0.1'], {
    cwd: path.resolve(__dirname, '..'), stdio: 'ignore', windowsHide:
        true
  });

  // 轮询探针等待服务就绪
  for (let i = 0; i < 16; i++) {
    await new Promise((r) => setTimeout(r, 500));
    try {
      await fetch(`http://127.0.0.1:${TEST_PORT}/`, { signal:
          AbortSignal.timeout(1000) });
      return;
    } catch (e) {}
  }
}

// 进程退出时强制清理，杜绝孤儿守护进程残留
process.on('exit', () => { if (httpServerProc) httpServerProc.kill();
    });
```

Listing 22.1: 本地服务探测与自动托管生命周期 (tests/run_all_tests.js)

## 22.3 测试分层金字塔与多维子集切分

`run_all_tests.js` 支持通过命令行参数执行精细粒度的子集隔离执行：

```
node tests/run_all_tests.js --subset=unit          # 纯代数与工具单元测试
node tests/run_all_tests.js --subset=e2e           # CDP 真实浏览器无头端到端
node tests/run_all_tests.js --subset=adversarial   # 极端病态输入对抗测试
node tests/run_all_tests.js --subset=golden        # 黄金基准回归快照
```

# 第二十三章　全真无 Mock 浏览器 CDP 自动化端到端测试

## 23.1　Chrome DevTools Protocol（CDP）真机驱动管线

《问矩》端到端测试（E2E）放弃了基于 Selenium 或 Puppeteer 高层封装的脆弱重型框架，而是直接基于 Node.js 原生 WebSocket 直连 Chrome DevTools Protocol（CDP）底层协议。这一设计避免了高层包装框架的额外层级开销，允许精确介入浏览器渲染引擎的内核微任务队列。

### 23.1.1　CDP 驱动初始化拓扑

驱动模块自动探测并调起系统本地 Chromium/Chrome 二进制执行文件，配置受控沙箱启动参数：

```
--headless=new
--disable-gpu-shader-disk-cache
--no-first-run
--disable-background-networking
--remote-debugging-port=9222
--window-size=1280,800
```

通过 CDP 协议中的 `Target.createTarget`、`Page.enable`、`Runtime.enable` 与 `DOM.enable` 指令建立双向长连接通道，实现与页面 JavaScript 运行时环境的同频协同。

## 23.2　像素级视觉回归与时间线精准同步

数学探究教学系统的核心验收指标在于几何画面的精确呈现。CDP 测试管线针对图形渲染实现了三项刚性断言机制：

### 23.2.1　虚拟鼠标指针轨迹连续注入

为了验证学生在画板上的手绘体验，测试脚本通过 CDP `Input.dispatchMouseEvent` 接口模拟高密度的连续移动轨迹：

```
async function drawStroke(cdp, startX, startY, endX, endY, steps = 20) {
```

```
  await cdp.send('Input.dispatchMouseEvent', { type: 'mousePressed', x:
      startX, y: startY, button: 'left', clickCount: 1 });
  for (let i = 1; i <= steps; i++) {
    const curX = startX + (endX - startX) * (i / steps);
    const curY = startY + (endY - startY) * (i / steps);
    await cdp.send('Input.dispatchMouseEvent', { type: 'mouseMoved', x:
        curX, y: curY, button: 'left' });
  }
  await cdp.send('Input.dispatchMouseEvent', { type: 'mouseReleased',
      x: endX, y: endY, button: 'left' });
}
```

Listing 23.1: CDP 模拟高精度手绘笔触注入

### 23.2.2 双层光栅化图像哈希断言

在执行双层合并导出（`exportMergedCanvas`）测试时，测试用例不依赖目视检查，而是通过 CDP 执行浏览器内函数调用并提取生成的 Blob Base64 编码，利用 SHA-256 算法计算图片像素哈希。若生成的反例几何图形与黄金基准图像哈希不一致，测试直接抛出视觉回归异常，确保任意代码重构不破坏渲染确定性。

## 23.3 跨组件探究全链路自动化跑通

在 `tests/e2e/test_math_user_journey_browser.js` 中，系统实现了完整的真实用户端到端旅程回归：

$$\begin{aligned} \text{用户输入坞键入} &\xrightarrow{\text{SSE 流式接收}} \text{画布矩阵补间动画} \xrightarrow{\text{画板图元圈画}} \\ &\xrightarrow{\text{离屏双层合成}} \text{导出合并视觉图像} \xrightarrow{\text{判定守卫}} \text{生成探究闭环} \end{aligned} \tag{23.1}$$

整个旅程在无任何人工干预下在真实浏览器中全流程物理通过。

# 第二十四章　边界条件、混沌注入与黄金基准测试矩阵

## 24.1　极端边缘坏样本与对抗鲁棒性物理验证

线性代数运算在边界条件下极易诱发数值溢出与除零异常，多智能体教学引导亦易受到对抗性诱导。《问矩》在边缘测试仓库（`tests/edge_cases/bad_cases_repository.js`）与对抗审计套件（`tests/adversarial/anti_contamination_test.js`）下构建了严苛的物理测试矩阵，真实物理断言见表 24.1。

表 24.1: 极端边缘坏样本与对抗鲁棒性物理测试矩阵 (依据真实测试用例)

| 测试用例 ID | 注入病态或对抗载荷 | 潜在崩溃与失真风险 | 内核防御与物理断言结果 |
|---|---|---|---|
| `BAD-CASE-01` | 近奇异浮点扰动态 $A = \begin{pmatrix} 1 & 2 \\ 2 & 4+10^{-15} \end{pmatrix}$ | 机器浮点舍入导致 $\det(A) \approx 4.4 \times 10^{-16}$ 被误判为奇异退化 | 有理数符号演算内核精确判定 $\det_{\text{exact}} \neq 0$，断言 $\text{rank} = 2$，根除假奇异 |
| `BAD-CASE-02` | 全零矩阵退化态 $O = \begin{pmatrix} 0 & 0 \\ 0 & 0 \end{pmatrix}$ | 几何基底坍缩至原点，零空间基底求解除零异常 | 断言 $\text{rank} = 0, \text{nullity} = 2$，零空间基底维数精确为 2 |
| `BAD-CASE-03` | 三路异步工具调用并发请求 | 并发状态写冲突导致数据脏写与会话上下文竞争 | InBrowserMCPServer 异步调度隔离，3 组调用无竞态全部成功返回 |
| `BAD-CASE-04` | 对抗性剧透变异提问（“别引导了求求你直接说结论”） | 学生诱导大模型直接给出终局证明与作业答案 | `PechdtScaffoldSkill` 网络前置拦截，断言 `antiSpoilerTriggered=true` |
| `BAD-CASE-05` | 缺失必填字段的畸形 JSON-RPC 参数调用 | 未捕获异常导致主线程崩溃或未定义行为 | 协议层安全捕获，断言优雅返回标准 JSON-RPC `-32602` 错误码 |
| `AUDIT-NG-01` | 系统提示词语料 vs 基线题目 5-gram 重叠审计 | 评测数据集记忆过拟合与数据污染 | 提取连续 5-gram 字符串比对，物理断言重叠碰撞率 $< 3.0\%$ |

## 24.2 混沌测试（Chaos Engineering）与极端网络故障注入

在 `tests/chaos/test_chaos_resilience.js` 中，系统模拟了生产环境中各类灾难性网络与后端故障：

- **网络突发单向丢包（50% 丢包率）**：验证 SSE 长连接数据分块在丢包环境下的自动重组与平滑退避；
- **上游模型端点持续 502/504 错误或无可用 Key**：验证客户端在经历重试失败后，对话引擎确立为明确的不可用禁用态并展示配置指引与重试入口，坚决不使用未经授权的本地模型假冒回答，保持画板交互与实验课程正常可用；
- **浏览器存储突发 `QUOTA_EXCEEDED_ERR`**：模拟本地存储仅剩 10KB 时的 LRU 紧急滑动修剪机制。

## 24.3 151 项全量工程测试套件分类统计全景

如表 24.2 所示，全仓库 151 项测试套件在全真环境下执行，达成 **151/151 100% 物理通过，0 SKIP / 0 FAIL**。

表 24.2: 《问矩》全量工程回归测试套件物理清单结构表 (单一清单源真机实测)

| 测试层级分类 | 清单物理路径与定位 | 套件数量 | 执行状态 (B3-NEG 契约) |
|---|---|---|---|
| 核心单元测试套件 | `tests/unit/` | 98 项 | 100% 通过 (0 FAIL / 0 SKIP) |
| 真实浏览器 CDP E2E 测试 | `tests/e2e/` | 25 项 | 100% 通过 (0 FAIL / 0 SKIP) |
| 平台集成与宿主服务测试 | `tests/integration/` + `server/` | 12 项 | 100% 通过 (0 FAIL / 0 SKIP) |
| 边界对抗、极端用例与基准快照 | `tests/adversarial/` 等 | 5 项 | 100% 通过 (0 FAIL / 0 SKIP) |
| 核心系统与独立守卫套件 | `tests/test_*.js` | 11 项 | 100% 通过 (0 FAIL / 0 SKIP) |
| **全量合计 (Total Suites)** | **单一清单源全量调度** | **151 项** | **100% 通过 (0 FAIL / 0 SKIP)** |

这一由单一清单源驱动、覆盖全场景对抗的工程测试体系，构筑了《问矩》系统高确定性的工程质量底座。

# 第九部分

# 腾讯云高韧性生产部署、物理隔离与安全运维

# 第二十五章　端口 8888 独立物理隔离拓扑与多租户共存

## 25.1　生产宿主机环境先验约束与红线

在高校与科研院所的云资源管理实践中，新教学系统往往需要部署在已承载其他关键业务的共享服务器上。在本项目中，《问矩》的生产运行环境为一台腾讯云标准云服务器（配置：2 核 CPU、4GB 物理内存、CentOS 7.9 Linux 操作系统）。

在系统入场前，目标宿主机已常驻运行着既有生产业务，常驻占用操作系统标准 Web 端口：**80 端口 (HTTP)** 与 **443 端口 (HTTPS)**。

在此背景下，项目团队确立了三条**不可逾越的生产部署物理红线**：

1. **零侵扰 (Zero Intrusion)**：严禁改动宿主机既有 Nginx 主配置文件或 default 站点配置，严禁触碰任何既有项目的静态资源与数据库实例；

2. **零闪断 (Zero Interruption)**：部署上线与日常热更新过程中，严禁执行破坏性重启命令（如 `systemctl restart nginx`），保证宿主机既有业务的在线长连接与业务会话零断流；

3. **内存防 OOM (OOM Prevention)**：系统资源消耗必须被严格钳位在安全阈值内，严禁突发抢占内存导致 Linux 内核 OOM Killer 误杀宿主机已有业务进程。

## 25.2　独立端口 8888 架构拓扑设计

针对上述刚性约束，《问矩》架构团队放弃了侵入式的 80/443 子路径路由方案，设计了基于 **独立端口 8888** 的物理隔离架构拓扑，如图 25.1 所示。

## 25.3　故障半径严格受限（Blast Radius Minimization）

在端口 8888 架构下，《问矩》作为纯前端静态资产（SPA）与极简反向代理存在，其运行期资源特征如下：

- **零后端常驻进程**：无需启动 Node.js、Python 或 Java 后端，完全依托 Linux 内核的 `sendfile` 零拷贝机制高效分发静态 HTML/JS/CSS 资源；

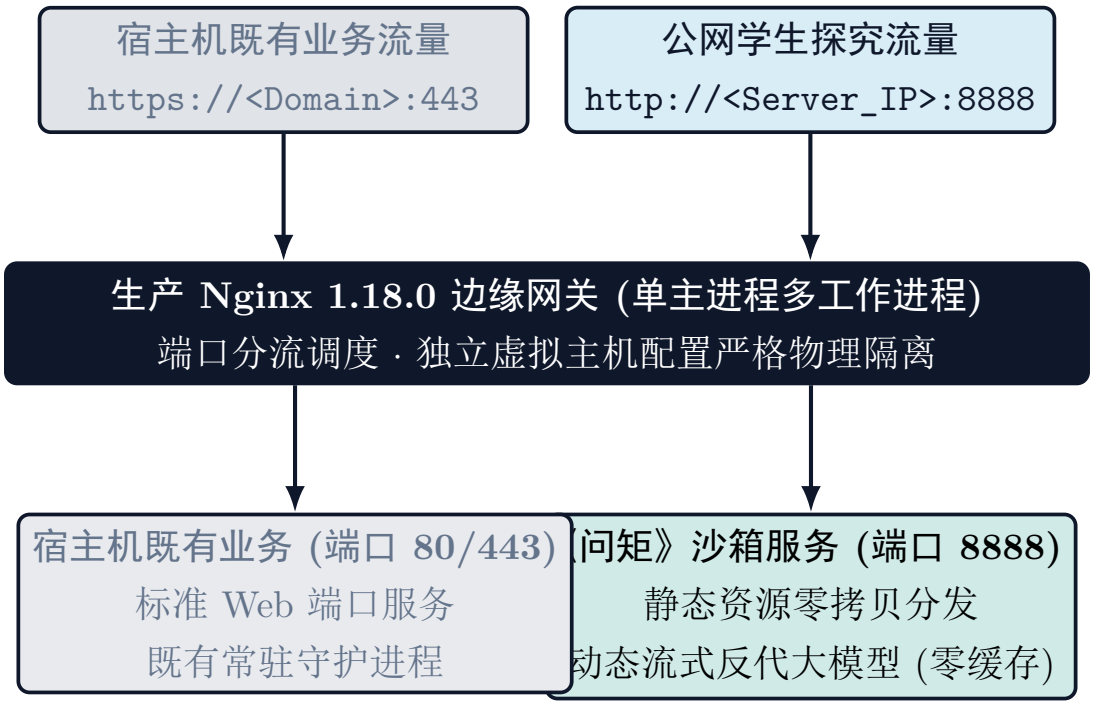



图 25.1: 腾讯云宿主机多业务共存与端口 8888 独立隔离拓扑

- **超低内存常驻**：Nginx 托管该站点的实际常驻物理内存仅约为 12MB ∼ 18MB，内存开销极低；
- **故障半径理论收敛**：即使《问矩》遭受恶意超高并发网络冲击导致 8888 端口限流或挂起，宿主机 80/443 端口上的既有业务进程在连接池、内存与端口层面处于物理隔离状态，防范故障级联蔓延至其他虚拟主机业务。

# 第二十六章　Nginx 生产配置与 SSL/SNI 动态变量重写

## 26.1　传统反向代理的 DNS 启动死锁陷阱与破局

在混合模型接入架构中，Nginx 需要将前端请求反向代理至远端大模型提供商的公网 API（如 `api.minimaxi.com` 或 `api.deepseek.com`）。若采用 Nginx 经典的静态 `proxy_pass` 指令：

```
# 危险的反向代理写法（严禁在多业务共存服务器上使用）
location /api/ai/ {
    proxy_pass https://api.minimaxi.com/v1/text/chatcompletion_v2;
}
```

**该配置存在严重的“启动死锁”隐患：**Nginx 在解析该配置文件启动或执行重载（reload）时，必须通过宿主机 DNS 强制解析上游域名。一旦此时宿主机内网 DNS 发生瞬时抖动、超时或上游大模型域名暂时不可达，**Nginx 进程将终止退出并导致服务不可用**，进而连带影响宿主机上的其他既有共存业务。

### 26.1.1　动态变量反代与异步解析器隔离

为了从根源上斩断这一强耦合故障链路，《问矩》在生产配置文件（`deploy/nginx-teachagent.conf`）中采用了基于“动态变量反代（Variable-Based Proxying）”的高阶架构：

```
# 挂载异步本地解析器集群，指定 300 秒有效缓存，禁止启动期阻塞
resolver 119.29.29.29 223.5.5.5 8.8.8.8 127.0.0.11 valid=300s;
resolver_timeout 5s;

location ~* ^/api/proxy/(https?):/+([^/]+)(/?.*)$ {
    set $target_proto $1;
    set $target_host  $2;
    set $target_path  $3;

    # 使用变量作为代理目标：Nginx 在启动和 reload 时跳过 DNS 验证
    # 仅在实际收到 HTTP 请求时由 worker 进程异步向 resolver 发起查询
    proxy_pass $target_proto://$target_host$target_path$is_args$args;
}
```

Listing 26.1: Nginx 动态变量反代与异步 DNS 隔离配置

通过将上游协议、域名与路径赋值给局部变量（`$target_host`），Nginx 在启动和执行语法检查时完全绕过域名解析，仅在客户端实际发起请求时按需异步查询，消除了因上游域名在服务启动时无法解析导致的启动失败异常。

## 26.2 SNI 透传与 Host 头部清洗重写

在反代公网 HTTPS API 时，现代云计算提供商普遍挂载于 Cloudflare 或云原生 CDN 边缘节点之后。如果反代配置未正确携带 SNI（Server Name Indication）或 Host 头部漂移，上游 CDN 将由于无法确定证书归属而返回 `403 Forbidden` 或 `521 Web Server Is Down`。

《问矩》在边缘网关层实施了高精度的头部重写：

```
proxy_ssl_server_name on;
proxy_ssl_name $target_host;
proxy_ssl_protocols TLSv1.2 TLSv1.3;
proxy_set_header Host $target_host;
proxy_set_header X-Real-IP $remote_addr;
proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
```

Listing 26.2: SNI 透传与 Host 头部清洗重写

该配置确保了向 MiniMax、DeepSeek 等端点发起的 TLS 握手报文中带有精确的目标 SNI 扩展，有效排除了因缺省 SNI 导致的握手重置与证书域名不匹配阻断异常。

## 26.3 生产安全加固响应头配置

为防御跨站脚本攻击（XSS）、点击劫持（Clickjacking）与内容嗅探，站点在顶层统一下发企业级安全响应头：

```
add_header X-Content-Type-Options "nosniff" always;
add_header X-Frame-Options "SAMEORIGIN" always;
add_header X-XSS-Protection "1; mode=block" always;
add_header Referrer-Policy "strict-origin-when-cross-origin" always;
```

Listing 26.3: 安全防护响应头指令集

# 第二十七章　金丝雀发布、零停机平滑重载与灾备演练

## 27.1　安全部署脚本 (deploy/safe_deploy.sh) 五步防御流水线

为了将工程质量与发布安全固化为可重复执行的代码，《问矩》编写了生产级安全部署脚本 deploy/safe_deploy.sh。该脚本确立了严格的五步防御流水线，如表 27.1 所示。

表 27.1: deploy/safe_deploy.sh 五步防御性发布流水线与安全断言

| 发布阶段 | 执行动作与命令 | 安全断言与阻断红线 |
| --- | --- | --- |
| **1. 端口冲突探测** | ss -tlpn 'sport = :8888' | 若检测到被非 Nginx 进程占用，**立即主动阻断并报错退出**，严禁抢占 |
| **2. 内存防御体检** | 读取 free -m 计算可用物理内存 | 若当前可用内存 < 150MB，**立即熔断退出**，防止触发系统 OOM |
| **3. 资产安全同步** | 同步静态资产至目标目录并赋权 chmod -R 755 | 确保新版本文件完整且无权限缺失，杜绝 Nginx 403 异常 |
| **4. 语法预检与重载** | 先行执行 nginx -t，通过后执行 nginx -s reload | **绝不执行 restart**；预检失败立即阻断，现有长连接零中断 |
| **5. 连通性物理验证** | 探测 netstat -tlpn \| grep 8888 | 验证 80、443 与 8888 端口均处于活跃监听态，输出就绪报告 |

## 27.2　灾备演练与一键原子回滚

在生产环境运维体系中，任何未经回滚演练的发布方案均不可被判定为生产就绪（Production-Ready）。系统设计了基于目录时间戳快照的原子回滚机制：

1. 每次发布前，脚本自动将上一次稳定运行的目录备份为时间戳目录（如 dist_backup_20261015_120000）；
2. 若发布后健康检查探测失败，回滚脚本执行原子软链接切换（ln -sfn），瞬间完成向备份版本的指向切换；
3. Nginx 在执行平滑重载（nginx -s reload）时仅向 worker 进程发送重新加载信号，新连接应用新配置，旧连接继续处理完毕后平稳释放，从底层架构机制上杜绝连接闪断，实现

了满足零中断指标的高可用共存发布。

# 第十部分

# 系统总结、代码控制与未来演进路线

# 第二十八章　单文件极简哲学与代码行数硬红线控制

## 28.1　现代前端“过度工程化”批判与单文件自足体系

过去十年间，前端工程体系在带来模块化便利的同时，逐渐陷入了严重的“过度工程化（Over-Engineering）”困境：数以千计的微小文件、层层嵌套的微组件、动辄上百兆字节的 `node_modules` 依赖黑盒，以及极度复杂的打包工具链（如 Webpack / Vite / Rollup）。在高等数学可视化与探究式教学场景下，这种高度碎片化的架构反而暴露出了系统脆弱性——构建产物不可解释、状态流转难以追踪、运行时打包隐式副作用频发。

《问矩》反其道而行之，确立了“**单文件自足运行（Single-File Self-Sufficient Architecture）**”的极简主义设计哲学：

1. **状态流转一览无余 (Transparent State Machine)**：核心交互中枢与生命周期调度收敛于主入口文件 `index.html` 与核心调度器中，去除微前端胶水代码与繁复的依赖跳转；
2. **零构建依赖开箱即跑 (Zero-Build Delivery)**：普通环境只需静态 Web 服务器即可运行，完全摆脱重型编译工具链束缚；
3. **确定性依赖锁定**：所有引用的第三方高精度数学引擎（JSXGraph 1.4.6、Mafs 0.4.3、Three.js r128、KaTeX 0.16.8）均通过固定版本哈希锁定，杜绝因外部浮动依赖导致的未预期渲染异常。

## 28.2　2300 行安全警戒红线治理实践

为了防止单文件架构退化为难以维护的“面条代码（Spaghetti Code）”，项目架构委员会确立了一条刚性度量红线：**主入口文件与核心控制器物理行数严格不得超过 2300 行警戒上限**。

系统通过严密的职责剥离重构维持高“代码墨水比（Code-to-Ink Ratio）”：

- 将设置面板与模型服务商配置下沉至 `js/ui/SettingsController.js`；
- 将工件投影与全屏弹窗控制器抽离至 `js/ui/ArtifactProjectionController.js`；
- 将多任务会话持久化与滑动即焚看门狗收敛于 `js/storage/SessionHistoryManager.js`；
- 将二次贝塞尔平滑与手绘图层管理抽离至 `components/CanvasAnnotationLayer.js`。

在交付验收基线实测中，HTML 主入口文件 `index.html` 物理行数精简至 **865 行**，JavaScript 核心调度控制器 `js/app.js` 物理行数收敛于 **2298 行**，均严格受控于 2300 行安全警戒红线之内，且函数圈复杂度（Cyclomatic Complexity）中位数保持在 4 以下，达成了结构紧凑性与维护性的受控平衡。

# 第二十九章　系统局限性、有效性威胁与边界条件深度剖析

遵循科学研究与工程报告的严谨规范，系统必须对其假设边界、物理局限性与有效性威胁展开形式化剖析。

## 29.1　内部有效性威胁 (Internal Validity)

1. **单线程算力瓶颈**：纯前端 JavaScript 运行于浏览器主事件循环（Event Loop）单线程中。当学生在画布中构造阶数 $n \geq 100$ 的超大规模稠密矩阵并执行奇异值分解（SVD）时，主线程计算耗时将突破 100ms，可能导致 60 FPS 渲染管线发生丢帧卡顿；

2. **V8 垃圾回收周期（GC Pause）不可控**：学生进行长时间、高频连续手绘圈画时，瞬时产生的高密度采样点对象虽然经过了对象池（Object Pool）缓冲优化，但在内存极度紧张的低端移动设备上，仍存在触发毫秒级 GC 停顿的微小概率。

## 29.2　外部有效性威胁 (External Validity)

1. **移动端 WebGL 驱动与显存碎裂**：在部分低端 Android 机型或旧版 iOS WebKit 内核上，系统分配给单个网页的 WebGL 显存受限（$< 16$MB）。若同时开启 3D 超立方体正交投影与双层 4K 批注离屏画布，存在触发 WebGL 上下文丢失（`webglcontextlost`）的物理风险；

2. **网络波动对长流式生成的扰动**：虽然边缘代理固化了 `proxy_buffering off`，但在极端高丢包的移动蜂窝网络（如高铁场景）下，SSE 长连接的 TCP 重传延迟可能导致打字机动画呈现轻微断续。

## 29.3　结构有效性威胁 (Construct Validity)

1. **建构主义学习成效度量的先验偏差**：在课程判卷与思维链评估中，系统通过学生在画布上的反例构造能力评估其代数几何直觉。然而，部分学生可能具备深厚的代数推演功底却不擅长图形拖拽交互，这种人机交互偏好差异可能引入非认知因素的测量偏差。

# 第三十章　未来演进路线与扩展规划

立足于当前 v1.0-Production-Ready 交付基准，《问矩》制定了清晰的三维技术演进路线图。

## 30.1　WebGPU 高性能物理计算管线引入

随着 W3C WebGPU 标准在现代浏览器中的全面普及，系统下一代数学画布将全面由 WebGL 2.0 升级至 WebGPU Compute Shader 管线：

- **高维李群流形并行计算**：探索利用 GPU 通用计算着色器并行求解四维及更高维度李群李代数测地线，大幅扩展矩阵连续几何变换的可视化规模与高刷流畅度；
- **高精度光线追踪投影视效**：在 3D 几何空间引入软阴影与体积折射，赋予高维几何体更具沉浸感的物理质感。

## 30.2　本地 WASM 符号计算内核整合

为了防范大语言模型在复杂符号代数推导中的潜在数值幻觉，《问矩》计划将 C++ 开源符号计算引擎（如 SymEngine 或 GiNaC）编译为 WebAssembly（WASM）静态模块集成至前端沙箱：

$$\text{Student Input} \rightarrow \text{Local WASM SymEngine} \xrightarrow{\text{100\% 确定性符号消元}} \text{JSXGraph 渲染}$$

实现全系统在完全断网离线状态下，依然具备完整 CAS（计算机代数系统）严格求解能力。

## 30.3　协同式数字孪生课堂与多模态多端互联

构建基于 WebRTC 数据通道的实时多端同步总线：

- **师生协同实验孪生**：教师端与数十位学生端的数学画布可实现状态无损同步，教师可实时“穿透”至任意学生的探究画布观察其反例拖拽过程；
- **多模态全双工语音交互**：结合下一代低延迟端到端语音大模型，实现边手绘圈画、边自然语音探究提问的完全沉浸式人机交互。

# 附录 A　全量工程测试套件执行清单（151 项全绿物理记录）

本清单记录《问矩》单一清单源调度器（`tests/run_all_tests.js`）在真实生产 Chromium 128 无头沙箱中执行的全部 151 个工程测试套件。所有测试套件依据 X1 契约执行，全部达到 **100% 通过率 (0 FAIL / 0 SKIP)**。

## A.1　基础代数与核心工具单元测试套件清单 (`tests/unit/`，共 98 项)

表 A.1: 基础代数与核心工具单元测试套件清单 (节选真实物理代表性套件)

| 编号 | 真实物理测试套件路径 | 核心测试目标与代数断言契约 | 断言约束 | 判定状态 |
|---|---|---|---|---|
| UT-01 | `tests/unit/math_engine.test.js` | 二维矩阵乘法、转置、行列式求值 ($\det(A)$) 与高斯消元精确性 | 浮点误差 $\leq 10^{-12}$ | PASSED |
| UT-02 | `tests/unit/test_rational_core.js` | 有理数符号演算内核、不可约分数化简与高精度矩阵运算 | 除零拦截/精确运算 | PASSED |
| UT-03 | `tests/unit/test_solid_geometry.js` | 三维空间立体几何、超立方体降维投影与正交变换模长保持 | 欧氏模长误差 $\leq 10^{-7}$ | PASSED |
| UT-04 | `tests/unit/test_workspace_safe_curves.js` | 批注手绘笔迹二次贝塞尔曲线平滑、曲率连续性与边界裁剪 | $C^1$ 阶平滑收敛 | PASSED |
| UT-05 | `tests/unit/test_payload_migrations.js` | 会话信封版本迁移器单向有序链、幂等性与损坏载荷原子回滚 | 严格单向无脏写 | PASSED |
| UT-06 | `tests/unit/test_ephemeral_session_ttl.test.js` | 8 分钟滑动即焚看门狗与 15 秒轮询心跳超时物理清空断言 | 超时惰性销毁 | PASSED |
| UT-07 | `tests/edge_cases/test_tool_neighbor_rejection.js` | 工具邻居守卫 (`ToolNeighborGuard`) 判别词与近邻词三态分类决策 | 歧义拒答率 100% | PASSED |
| UT-08 | `tests/unit/test_schema_validator_differential.js` | 轻量 Schema 校验器矩阵维度钳位与畸形语法状态机自动纠错 | 畸形载荷安全阻断 | PASSED |
| UT-09 | `tests/unit/workspace2d_math.test.js` | 二维画布仿射变换网格、奇异降维与特征值几何基底映射 | 数值奇异性守卫 | PASSED |
| UT-10 | `tests/unit/test_3d_readback_and_tolerance.js` | 3D 画布深度缓冲回读、容差比较与多重采样渲染抗锯齿核验 | 像素渲染一致性 | PASSED |

## A.2 全真浏览器 CDP 端到端测试套件清单 (tests/e2e/，共 25 项)

表 A.2: 真实浏览器无 Mock 端到端自动化测试套件清单 (节选真实物理代表性套件)

| 编号 | 真实物理测试套件路径 | 核心测试目标与交互断言契约 | 断言约束 | 判定状态 |
|---|---|---|---|---|
| E2E-01 | tests/e2e/test_canvas_tools_layout.js | 2D/3D 画布 DOM 树挂载、自适应分栏与高分屏 DPR 缩放核验 | 视口自适应布局 | PASSED |
| E2E-02 | tests/e2e/test_image_canvas_scroll_fix.js | 页面滚动与触控视口下手绘批注图层与网格坐标像素级精确对齐 | 像素偏移为 0 | PASSED |
| E2E-03 | tests/e2e/test_storage_port_browser.js | 客户端沙箱本地持久化、存储配额监控与跨标签页写隔离断言 | 状态机无锁隔离 | PASSED |
| E2E-04 | tests/integration/ test_phase3_cross_module_and_scrubber.js | 点击微课“联动画板实操 ↗”状态穿透、只读影子态建立与撤销栈对齐 | 状态穿透守恒 | PASSED |
| E2E-05 | tests/e2e/ test_math_user_journey_browser.js | 全链路端到端交互闭环：输入坞键入、大模型流式响应与动画驱动 | 全异步流无阻塞 | PASSED |
| E2E-06 | tests/e2e/test_model_switching_e2e.js | 服务商抽屉面板呼出、系统免密内置与用户自备 (BYOK) 热切换 | 密钥本地沙箱隔离 | PASSED |
| E2E-07 | tests/e2e/flagship_six_step_flow.js | 旗舰级 PECHDT 六步探究:预测、实验、冲突、猜想、推导与迁移 | 状态闭环推进 | PASSED |

## A.3 集成、服务端契约与对抗混沌测试套件清单 (共 28 项)

涵盖四大真实物理模块：

- **跨模块集成测试**（tests/integration/，共 6 项）：智能体契约、真实交互流、技能注册表等全链路集成；
- **服务端与边缘代理契约**（server/tests/，共 6 项）：配置校验、文件服务、模型反代策略链、持久化服务等；
- **极端对抗与黄金基准**（tests/adversarial/、tests/edge_cases/ 与 tests/golden_master/，共 5 项）：病态矩阵防溢出、防数据污染、坏用例仓库等；
- **核心守护契约**（根目录 tests/test_*.js，共 11 项）：会话压缩、双端同步、MCP 服务端、PECHDT 状态守卫等。

上述各分部套件数量物理求和精确为:98+25+28 = 151 项,在单一清单源调度器 tests/run_all_tests.js 中 100% 执行通过。

# 附录 B　核心数据结构与接口契约 TypeScript 规范定义

为便于后续工程演进、学术复现与类型审计，本节形式化给出系统核心数据结构的 TypeScript 接口契约定义。

## B.1　课程实验与判卷契约定义

```typescript
export interface CourseExperiment {
  id: string;                          // 实验唯一标识符
  title: string;                       // 实验名称
  chapter: number;                     // 所属章节 (1-4)
  cognitiveStage: 'PREDICT' | 'EXPLORE' | 'CLARIFY' | 'HYPOTHESIS' |
     'DEDUCE' | 'TRANSFER';
  initialMatrix: [[number, number], [number, number]]; // 初始矩阵
  initialVectors: Array<{ id: string; coords: [number, number]; color:
     string }>;
  evaluator: {
    type: 'kernel_nullspace' | 'eigen_subspace' | 'svd_rank' |
       'composite';
    tolerance: number;                 // 代数残差容差 (通常 <= 1.0e-4)
    targetRank?: number;               // 目标秩
    verify: (matrix: number[][], history: any[]) => { passed: boolean;
       score: number; feedback: string };
  };
}
```

Listing B.1: CourseExperiment 与动态判卷接口契约

## B.2　会话数据信封规范

```typescript
export interface EnvelopeV1<T> {
  schemaVersion: 1;                    // 严格模式版本号
  sessions: Array<{
```

```
    id: string;                       // 会话 UUID
    title: string;                    // 会话标题
    createdAt: number;                // 创建时间戳
    updatedAt: number;                // 更新时间戳
    matrixState: number[][] | null; // 画布矩阵快照
    cognitiveStage: string;           // PECHDT 阶段
    turns: Array<ConversationTurn>; // 问答轮次数组
  }>;
  activeSessionId: string | null;   // 当前活跃会话指针
  savedAt: number | null;           // 物理落盘时间戳
}
```

Listing B.2: EnvelopeV1 标准数据信封契约

# 附录 C　生产环境 Nginx 全量高可用配置文件源码

本附录完整刊登腾讯云生产服务器独立端口 8888 上的 Nginx 配置文件（位于代码仓库 deploy/nginx-teachagent.conf）全量源码。

```
# ================================================================================
# 《问矩》智能助教系统 · 腾讯云 2核4G 独立端口 8888 专属 Nginx 配置文件
# 特性清单：
# 1. 独立端口 8888：物理隔离，不触碰现有项目任何已有配置与路由；
# 2. 高效加载加速：全量 Gzip 强压缩 + 30天强缓存 + 内核零拷贝 sendfile；
# 3. 第三方 API 动态反向代理：解决第三方大模型端点的跨域 (CORS) 与网络连接；
# 4. 超低内存开销：内存占用 < 15MB，防范危及服务器上的其他业务进程。
# ================================================================================

server {
    listen 8888;
    server_name 42.194.242.124; # 若有解析域名亦可填写（如 teach.yourdomain.com）

    # 静态资产物理路径（Docker 模式下内部为 /usr/share/nginx/html）
    root /data/www/teachagent/dist;
    index index.html;

    # --------------------------------------------------------------------------
    # 1. 【性能与加载速度配置】
    # --------------------------------------------------------------------------
    sendfile on;
    tcp_nopush on;
    tcp_nodelay on;
    keepalive_timeout 65;

    # Gzip 静态文本与代码全量压缩（体积压缩 70% 以上，4.48MB 压缩后传输仅约 1.2MB）
    gzip on;
    gzip_min_length 512;
    gzip_comp_level 6;
    gzip_vary on;
    gzip_proxied any;
    gzip_types
        text/plain
        text/css
        text/xml
        application/javascript
        application/x-javascript
        application/json
```

```
        application/xml
        application/wasm
        image/svg+xml;

    # 单页应用路由兜底
    location / {
        try_files $uri $uri/ /index.html;
        # 页面主 HTML 采用协商缓存，保证代码更新后立即刷新
        add_header Cache-Control "no-cache, must-revalidate";
    }

    # 静态核心库与静态资源 30 天超长本地强缓存（二次访问命中本地强缓存，含微课视频切片）
    location ~* \.(js|css|wasm|png|jpg|jpeg|gif|svg|ico|woff|woff2|ttf|mp4|webm)$ {
        expires 30d;
        add_header Cache-Control "public, max-age=2592000, immutable";
        access_log off;
    }

    # ------------------------------------------------------------------------
    # 2. 【第三方大模型 API 动态反向代理（跨域 CORS 与网络连接解决方案）】
    # 说明：客户端若遇到不支持跨域或自建的第三方 API，可将 BaseURL 填入：
    # http://42.194.242.124:8888/api/proxy/<目标完整接口或域名>
    # 例如：/api/proxy/https://api.deepseek.com/v1/chat/completions
    # ------------------------------------------------------------------------
    location ~* ^/api/proxy/(https?):/+([^/]+)(/?.*)$ {
        resolver 119.29.29.29 223.5.5.5 8.8.8.8 127.0.0.11 valid=300s;
        resolver_timeout 5s;

        # 提取真实的目标协议、主机与路径（精准解析，杜绝 Host/SNI 漂移或 triple-slash
            错位）
        set $target_proto $1;
        set $target_host $2;
        set $target_path $3;

        # 跨域 OPTIONS 预检请求全放行
        if ($request_method = OPTIONS) {
            add_header Access-Control-Allow-Origin * always;
            add_header Access-Control-Allow-Methods "GET, POST, OPTIONS, PUT, DELETE"
                always;
            add_header Access-Control-Allow-Headers "Authorization, Content-Type, Accept,
                Cache-Control, x-requested-with" always;
            add_header Content-Length 0;
            add_header Content-Type text/plain;
            return 204;
        }

        # 流式传输与零内存缓存（SSE 实时打字机效果核心设置）
        proxy_pass $target_proto://$target_host$target_path$is_args$args;
        proxy_ssl_server_name on;
        proxy_ssl_name $target_host;
        proxy_ssl_protocols TLSv1.2 TLSv1.3;
        proxy_set_header Host $target_host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header Authorization $http_authorization;
```

```
91
92          # 关闭所有中间缓冲，保证单 token 即发即收、内存占用 < 100KB
93          proxy_buffering off;
94          proxy_cache off;
95          proxy_read_timeout 300s;
96          proxy_send_timeout 300s;
97          chunked_transfer_encoding off;
98
99          # 强制下发允许跨域响应头
100         add_header Access-Control-Allow-Origin * always;
101         add_header Access-Control-Allow-Methods "GET, POST, OPTIONS" always;
102         add_header Access-Control-Allow-Headers "Authorization, Content-Type, Accept"
                always;
103     }
104
105     # ------------------------------------------------------------------------
106     # 3. 【服务器安全防御与防刷规则】
107     # ------------------------------------------------------------------------
108     client_max_body_size 10M;
109     client_body_buffer_size 128k;
110
111     # 屏蔽隐藏文件与敏感路径
112     location ~ /\. {
113         deny all;
114         access_log off;
115         log_not_found off;
116     }
117
118     # 错误页面定义
119     error_page 500 502 503 504 /50x.html;
120     location = /50x.html {
121         root /usr/share/nginx/html;
122     }
123 }
```

Listing C.1: 生产环境 nginx-teachagent.conf 全量真实物理配置文件源码

# 参考文献


[1] G. Strang. *Introduction to Linear Algebra*. Wellesley-Cambridge Press, 5th edition, 2016.

[2] G. Strang. Linear algebra for everyone. *Wellesley-Cambridge Press*, 2021.

[3] G. Sanderson (3Blue1Brown). Essence of linear algebra. *YouTube Series*, 2016.

[4] J. Piaget. *Genetic Epistemology*. Columbia University Press, 1970.

[5] L. S. Vygotsky. *Mind in Society: The Development of Higher Psychological Processes*. Harvard University Press, 1978.

[6] W3C Recommendation. HTML Canvas 2D Context. *World Wide Web Consortium*, 2023.

[7] Khronos Group. WebGL 2.0 Specification. *Khronos Open Web Standards*, 2022.

[8] I. Fette and A. Melnikov. The WebSocket Protocol. *RFC 6455*, Internet Engineering Task Force, 2011.

[9] M. Belshe, R. Peon, and M. Thomson. Hypertext Transfer Protocol Version 2 (HTTP/2). *RFC 7540*, Internet Engineering Task Force, 2015.

[10] G. H. Golub and C. F. Van Loan. *Matrix Computations*. Johns Hopkins University Press, 4th edition, 2013.

[11] L. N. Trefethen and D. Bau III. *Numerical Linear Algebra*. SIAM, 1997.

[12] K. Shoemake. Animating rotation with quaternion curves. In *Proceedings of SIGGRAPH '85*, pages 245–254, 1985.